NPTEL LLM Notes

Authors: Ambikesh & Diksha

Introduction to Large Language Models (LLMs)

Prof. Tanmoy Chakraborty, Prof. Soumen Chakraborti
IIT Delhi, IIT Bombay

About the Course:

This course introduces the fundamental concepts underlying **Large Language Models (LLMs)**. It begins with an overview of challenges in **Natural Language Processing (NLP)** and explains how language modeling can be approached using **deep learning**. It explores the **Transformer architecture**, its **pre-training objectives**, and advances in **LLM alignment, prompting, efficient adaptation, hallucination, bias, and ethics**. The goal is to prepare students to understand and investigate modern LLM research.

Audience:

Undergraduate and postgraduate students in **CSE, EE, ECE, IT, Mathematics**, and related fields.

Prerequisites:

Mandatory: Machine Learning, Python Programming
Optional: Deep Learning

Reference Courses:

- Introduction To Machine Learning: https://nptel.ac.in/courses/106105152
- Python for Data Science: https://nptel.ac.in/courses/106106212

Industry Support:

Industries involving machine learning applications, such as **Google, Microsoft, Adobe, IBM, Accenture, JP Morgan, Wipro, Flipkart, Amazon**, etc.

Introduction to Large Language Models

Overview

Large Language Models (LLMs) are advanced AI systems capable of understanding and generating human-like text using deep learning. They build upon decades of progress in **Natural Language Processing (NLP)** and neural network architectures.

Real-life Analogy

When your phone suggests the next word while texting, it uses a small language model. An LLM (e.g., ChatGPT) does the same but at a much larger scale — trained on huge corpora to produce coherent, context-aware text across tasks.

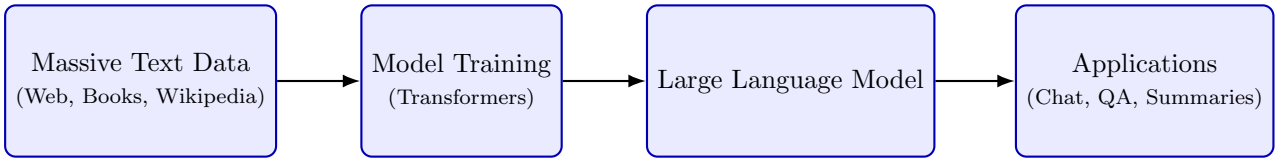


Figure 1: Pipeline of a Large Language Model from data to applications.

Quick example (overview)

Example: Email Autocomplete

Scenario: You type "Dear Professor," and email autocomplete suggests the next phrase. **Why it works:** The model computes $P(\text{next words} \mid \text{Dear Professor,})$ from patterns seen during training and offers high-probability continuations.

Mathematical Foundation

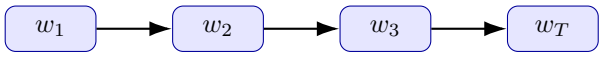
A language model assigns a probability to a sequence of tokens:

$$P(w_1, w_2, \ldots, w_T)$$

Using the chain rule:

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1})$$

Each factor is the model's conditional prediction for the next token.



Each arrow: conditional probability $P(w_t \mid w_{<t})$.

Figure 2: Chain-rule view: each token depends on previous tokens.

Example & explanation (probabilities)

Example: Predicting a short sentence

Vocabulary $V = \{\text{the, monsoon, rains, have, arrived}\}$.
Probability of grammatical sequence:

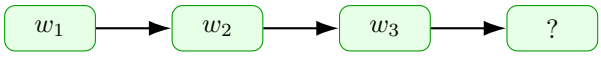
$$P(\text{the monsoon rains have arrived}) = \prod_{t} P(w_t \mid w_{<t}) \approx 0.2$$

Non-grammatical order receives a much smaller product probability (e.g., 0.001).
Takeaway: The product of many conditional probabilities captures fluency and coherence.

Text Generation

LLMs often generate text in an **auto-regressive** manner: predict one token at a time conditioned on prior tokens:

$$P(x_i \mid w_1, \ldots, w_t)$$



Model picks highest-probability next token at each step.

Figure 3: Auto-regressive generation: stepwise next-token prediction.

Real-life example (generation)

Example: Chatbot Answering a Question

Prompt: "Explain photosynthesis in simple words." **Model flow:** The model produces each subsequent word maximizing $P(\text{next} \mid \text{context})$ and stops when termination criteria are met. **Why useful:** Produces clear explanations, summaries, or conversational replies on demand.

Growth of LLMs

Empirical trends show bigger models + more data improve general capabilities (up to limits like compute & data quality).

Model	Parameters	Key Feature
GPT-1	117M	Generative pretraining
GPT-2	1.5B	Open-ended generation
GPT-3	175B	Few-shot generalization
PaLM	540B	Large-scale training
GPT-4	∼1.7T	Multimodal reasoning
Gemini Ultra	∼1.5T	Unified multimodal learning

Practical note

Note on scaling

Bigger models can store more implicit knowledge and often perform better on reasoning tasks, but they require more compute and careful safety checks (e.g., bias evaluation).

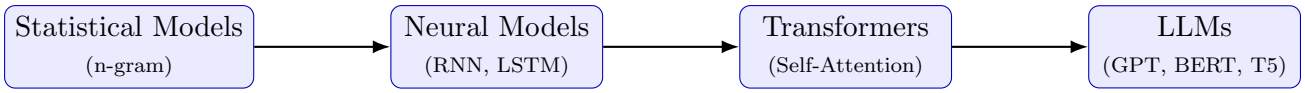


Figure 4: Evolution of Language Models: from statistical counts to transformer-based LLMs.

Evolution of Language Models

Short examples of each stage

Examples by era

- **Statistical (n-gram):** Predict next word by counting co-occurrences. (e.g., language models used in early spell-checkers)
- **Neural (RNN/LSTM):** Use hidden states to capture sequences (e.g., early machine translation systems).
- **Transformers:** Use self-attention to model long-range context efficiently (e.g., BERT for question answering).
- **LLMs:** Scale transformers and pretrain on large corpora for generative tasks (e.g., ChatGPT answering diverse prompts).

Emergent Capabilities

Key Capabilities

- **In-context learning:** Perform new tasks from examples in the prompt (no retraining).
- **Few-shot learning:** Learn patterns from a few demonstration examples.
- **Compositional reasoning:** Chain steps to solve multi-step problems.
- **Generalization:** Apply learned patterns to unseen contexts.

Example: In-context learning

Practical Example

Provide two examples of converting Celsius to Fahrenheit in the prompt, then ask to convert another value. The model uses the examples to infer the conversion rule and produces correct results without gradient updates.

Risks and Challenges

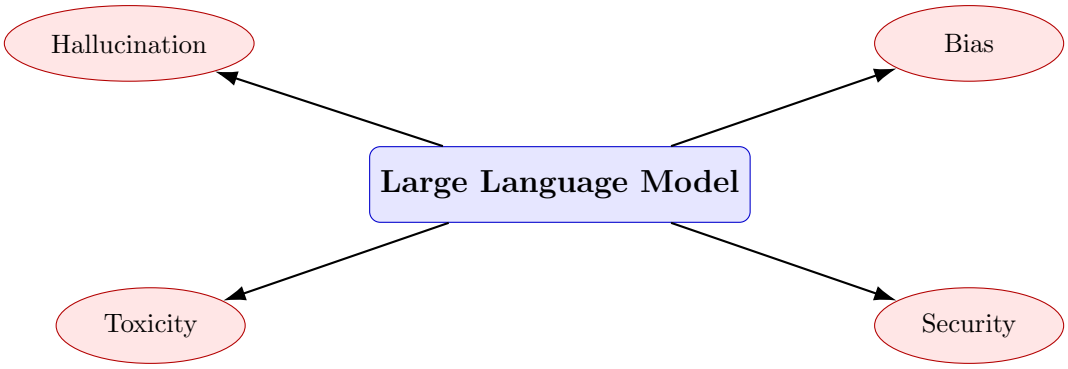


Figure 5: Main risks associated with LLM outputs and deployment.

Concrete examples & mitigations

Problem + Mitigation

Hallucination: Model fabricates facts (e.g., invents citations).

Mitigation: Use retrieval-augmented generation (RAG) and source citations.

Bias: Stereotyped associations in outputs (gender/ethnicity).

Mitigation: Dataset curation, fairness tests, and post-processing filters.

Toxicity: Generates offensive content.

Mitigation: Safety fine-tuning and content filters.

Security: Prompt injection or model misuse.

Mitigation: Input sanitization, access controls, adversarial testing.

Core Areas of LLM Research

Module	Focus	Main Topics
Basics	NLP and Deep Learning	Embeddings, attention
Architecture	Transformer Design	Positional encoding, tokenization
Learnability	Adaptation & Alignment	Prompting, fine-tuning, PEFT
Knowledge	External Access	RAG, Knowledge graphs
Ethics	Responsible AI	Bias, safety, transparency

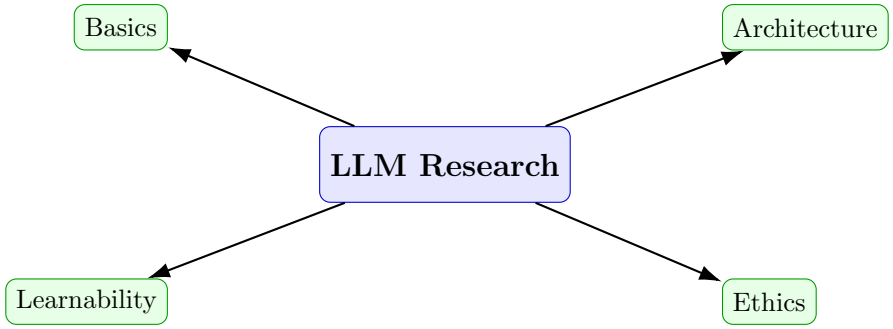


Figure 6: Major research directions that inform modern LLM development.

Summary

- LLMs model conditional probabilities of token sequences and generate text auto-regressively.
- Transformers enabled efficient long-range context and are the core LLM architecture.
- Larger models generally improve capabilities but bring cost and safety trade-offs.
- Practical deployment requires retrieval, evaluation, and mitigation for hallucination and bias.

Introduction to Natural Language Processing

Overview

Natural Language Processing (NLP) is the study of computational methods for understanding, analyzing, and generating human language. It combines linguistics, computer science, and machine learning to process text or speech.

Real-life Analogy

When you talk to Siri, Alexa, or Google Assistant, they convert your speech to text, interpret its meaning using NLP models, and then respond intelligently.

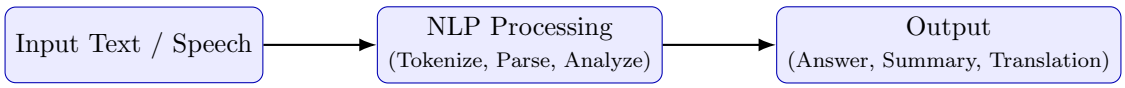


Figure 7: General NLP pipeline: input → analysis → output.

Ambiguity in Language

Language is naturally ambiguous. The same sentence can have multiple interpretations.

- *I saw a girl with a telescope.* — Who owns the telescope?
- *Mary had a little lamb.* — Pet or meal?
- *I ate rice with Rahul.* — Companion or instrument?

Real-world Example

In customer-support chatbots, "I need a new card" could mean a *credit card*, *SIM card*, or even a *birthday card*. Context and intent detection modules in NLP help disambiguate such requests.

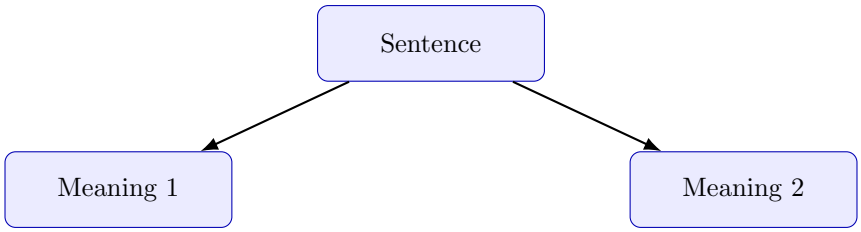


Figure 8: Ambiguity: one sentence → multiple interpretations.

Why NLP is Challenging

- Lexical and structural ambiguity
- Dependence on context and world knowledge
- Idioms and informal expressions
- Complex grammar and multiword constructs

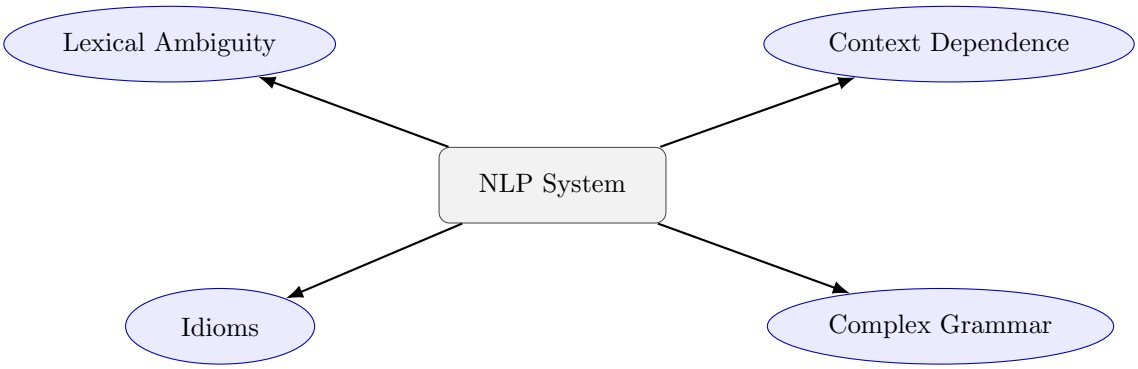


Figure 9: Challenges faced by NLP systems.

Example: Human vs. Machine Understanding

Humans understand that "Can you pass the salt?" is a polite request, not a yes/no question. Machines must infer intent beyond literal words — that's pragmatics and context.

Linguistic Layers in NLP

- **Morphology:** Word formation, e.g., $dogs = dog + s$.
- **Syntax:** Sentence structure — grammar rules $G = \langle V, T, P, S \rangle$.
- **Semantics:** Meaning of words and phrases.

- **Pragmatics:** Intended meaning in context.
- **Discourse:** Logical flow across sentences.

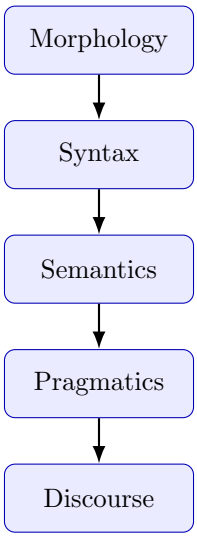


Figure 10: Linguistic hierarchy in NLP analysis.

Real-world Example

Voice assistants process language through these layers: speech → words (morphology), sentence parsing (syntax), meaning extraction (semantics), understanding intent (pragmatics), and conversation coherence (discourse).

Core NLP Tasks

- Part-of-Speech Tagging — grammatical labeling.
- Parsing — checking sentence structure.
- Named Entity Recognition — identifying people, places, etc.
- Word Sense Disambiguation — correct meaning selection.
- Sentiment Analysis — detecting emotion or polarity.
- Machine Translation — language conversion.
- Summarization — information condensation.
- Question Answering — factual extraction.

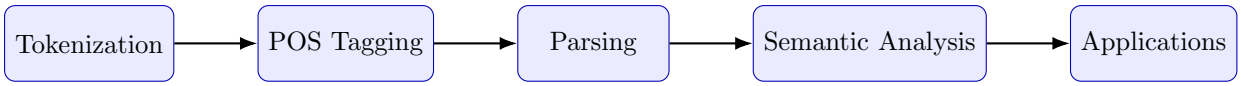


Figure 11: Simplified NLP pipeline: from text segmentation to understanding.

- Dialogue Systems — interactive communication.

Real-world Examples of Tasks

POS Tagging: Grammar checking in MS Word.
NER: News apps detecting person or location names.
Sentiment Analysis: Monitoring customer feedback on Twitter.
Translation: Google Translate uses transformer-based NLP.
Summarization: Meeting-note apps condensing transcripts.

Summary

NLP forms the foundation of LLMs. Modern transformer-based systems jointly learn morphology, syntax, semantics, and pragmatics in one neural architecture. Remaining challenges include ambiguity, bias, factual consistency, and contextual accuracy.

Key Takeaway

NLP bridges human communication and machine understanding — every LLM builds upon these linguistic layers and tasks to understand and generate meaningful language.

Statistical Language Models

Overview

Statistical Language Models (SLMs) estimate how probable a word sequence is. They are key components in **speech recognition**, **machine translation**, **text generation**, and **spelling correction**. An SLM computes:

$$P(W) = P(w_1, w_2, \ldots, w_n)$$

and uses it to decide which sentences sound more "natural" in a language.

Real-life Example

When your phone's autocorrect suggests "good morning" instead of "good monsoon," it uses a statistical model trained to know which word sequences are more probable in English.

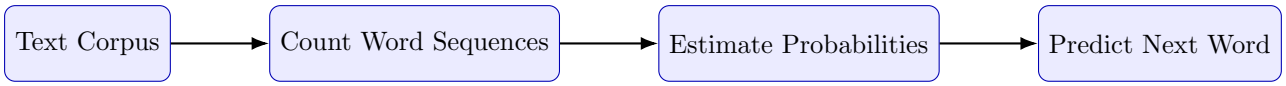


Figure 12: Basic flow of a Statistical Language Model.

—

Goal of a Language Model

The model computes probabilities such as:

$$P(W) = P(w_1, w_2, \ldots, w_n) \quad \text{or} \quad P(w_n \mid w_1, \ldots, w_{n-1})$$

Intuitive Understanding

If you have the partial phrase "The monsoon", a good LM should assign higher probability to "season" than "keyboard" — because "season" is statistically more likely to follow.

—

Probability of a Sentence

For a sentence like "The monsoon season has begun",

$$P(W) = P(\text{The})\, P(\text{monsoon} \mid \text{The})\, P(\text{season} \mid \text{The, monsoon}) \ldots$$

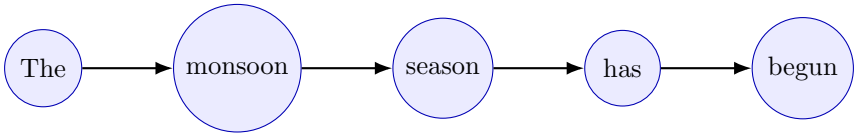


Figure 13: Sentence probability estimated via conditional dependencies.

—

The Chain Rule and the Markov Assumption

The chain rule expresses joint probabilities as products of conditionals:

$$P(W) = \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})$$

Because long dependencies are hard to model, we use the **Markov assumption**:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-k}, \ldots, w_{i-1})$$

Only the last few words matter (limited context).

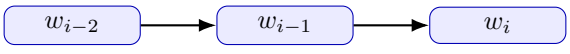


Figure 14: Markov assumption: next word depends on a few previous words.

Example

Typing "New" → your phone predicts "York," not "Phone," because it considers the immediate context "New" (bigram model).

—

N-gram Language Models

An N-gram model predicts a word from the previous $(N-1)$ words:

$$P(w_i \mid w_{i-N+1}, ..., w_{i-1})$$

- **Unigram:** $P(w_i)$
- **Bigram:** $P(w_i \mid w_{i-1})$
- **Trigram:** $P(w_i \mid w_{i-2}, w_{i-1})$

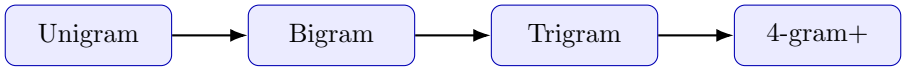


Figure 15: Progression of N-gram models. Higher N = larger context.

—

Estimating N-gram Probabilities

Using Maximum Likelihood Estimation (MLE):

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

Example: Frequency Counts

If "the cat" occurs 50 times and "the" occurs 500 times,

$$P(\text{cat} \mid \text{the}) = 50/500 = 0.1$$

The model thus predicts "cat" after "the" with 10% likelihood.

—

The Data Sparsity Problem

Unseen word combinations yield zero probability. Example training vs. test data:

- Train: "enjoyed the movie", "enjoyed the food"
- Test: "enjoyed the concert"

Since "concert" never appeared → $P(\text{concert} \mid \text{enjoyed the}) = 0$.

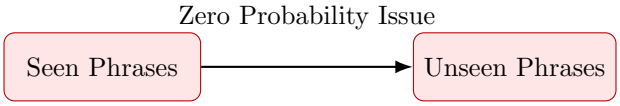


Figure 16: Data sparsity: unseen sequences receive zero probability in SLMs.

—

Smoothing Techniques

Smoothing redistributes probability mass to unseen events.

(a) Laplace (Add-One) Smoothing

$$P_{\text{Add-1}}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + |V|}$$

Adds one to all counts.

(b) Add-k Smoothing

$$P_{\text{Add-k}}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i) + k}{c(w_{i-1}) + k|V|}$$

Uses smaller $k < 1$ for finer control.

(c) Unigram Prior Smoothing

$$P_{\text{UnigramPrior}}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i) + mP(w_i)}{c(w_{i-1}) + m}$$

(d) Back-off and Interpolation

$$P_{\text{interp}} = \lambda_1 P(w_i \mid w_{i-2}, w_{i-1}) + \lambda_2 P(w_i \mid w_{i-1}) + \lambda_3 P(w_i)$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$.

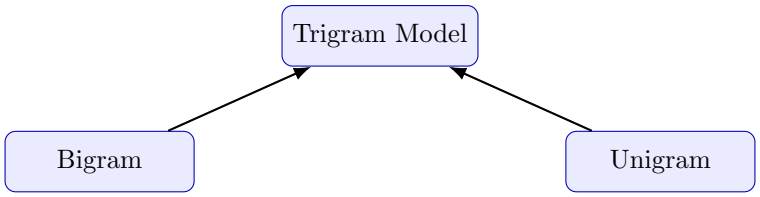


Figure 17: Back-off and interpolation combine multiple N-gram levels.

Example: Real Application

In predictive keyboards, smoothing ensures that even rare phrases like "quantum entanglement" get nonzero probabilities, avoiding blank predictions.

—

Advanced Smoothing Methods

(a) Good–Turing

$$c^* = \frac{(c+1)N_{c+1}}{N_c} \quad \text{and} \quad P^*(\text{unseen}) = \frac{N_1}{N}$$

(b) Absolute Discounting

$$P_{\text{AD}}(w_i \mid w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1})P(w_i)$$

(c) Kneser–Ney

$$P_{\text{KN}}(w_i \mid w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1})P_{\text{cont}}(w_i)$$

Insight

Kneser–Ney smoothing is most accurate because it adjusts probabilities based on how many different contexts a word appears in.

—

Evaluation of Language Models

(a) Intrinsic Evaluation — Perplexity

$$PP(W) = P(w_1, w_2, \ldots, w_n)^{-1/n}$$

Lower perplexity → better prediction.

(b) Extrinsic Evaluation

Measures how the LM improves tasks like translation or speech recognition.

Real-life Example

Google Translate trains its LM to reduce perplexity on bilingual text, improving fluency of generated translations.

—

Limitations of Statistical Language Models

- Fixed-size context (limited memory)
- Data sparsity and zero probabilities
- Expensive computation for large N
- Lack of semantic understanding

Figure 18: Core limitations of SLMs motivating neural approaches.

—

Transition to Neural Language Models

Neural models replaced discrete counts with continuous embeddings.

- Learn distributed representations (word vectors)
- Capture long-range dependencies
- Handle unseen combinations gracefully
- Enable transfer learning and fine-tuning

Figure 19: Evolution from SLMs to Neural and Transformer-based LMs.

Real-world Shift

Google's 2006 statistical translation system was replaced by **Neural Machine Translation (NMT)** in 2016, drastically improving fluency and contextual accuracy.

—

Summary

- SLMs use probability to model natural language sequences.
- N-gram models approximate context using the Markov assumption.
- Smoothing handles unseen data; Kneser–Ney performs best.
- Evaluation via perplexity measures predictive power.
- Neural models overcame SLM limitations using embeddings and deep networks.

Key Takeaway

Statistical Language Models laid the probabilistic groundwork for modern NLP. Their evolution toward neural networks marks the bridge from traditional to deep learning–based language understanding.

Deep Learning Basics for Language Modeling (Tensors and PyTorch)

Overview

Deep learning models rely on efficient numerical computation. **PyTorch** offers a flexible tensor library supporting automatic differentiation and GPU acceleration — ideal for training neural language models. This chapter introduces **tensors**, tensor operations, and the fundamentals of **gradient computation** in PyTorch.

Real-life Analogy

Think of tensors as "containers" for numbers — like spreadsheets for AI. They store all inputs, weights, and activations inside a neural network. PyTorch automates their math, much like Excel formulas but millions of times faster.

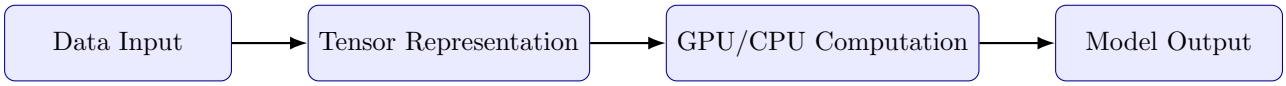


Figure 20: Pipeline of tensor-based computation in PyTorch.

—

1. Introduction to Tensors

A **tensor** is a multi-dimensional array — the core data structure in deep learning. It generalizes scalars, vectors, and matrices to any number of dimensions.

Tensor Type	Example	Dimension
Scalar	$a = 5$	0-D
Vector	[3, 7, 9]	1-D
Matrix	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	2-D
Tensor	`torch.rand(2,3,4)`	3-D or higher

Example: From Scalar to Tensor

Scalar: A single temperature reading (e.g., $25°C$). **Vector:** Weekly temperatures $[25, 27, 30, 29]$. **Matrix:** Temperature grid across cities and days. **Tensor:** Extends this grid to multiple sensors, days, and conditions.

—

2. Creating Tensors in PyTorch

Tensors can be created directly from Python lists or NumPy arrays.

Code Example: Creating Scalars, Vectors, and Matrices

```
import torch

# Scalar (0D)
scalar = torch.tensor(1)

# Vector (1D)
vector = torch.tensor([6, 8, 0, 1, 2])

# Matrix (2D)
matrix = torch.tensor([[0, 1, 7],
                       [4, 2, 4]])

print("Vector Shape:", vector.shape)
print("Matrix Shape:", matrix.size())
```

—

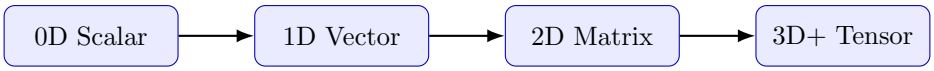


Figure 21: Tensor dimensional hierarchy.

3. Tensor Initialization Methods

PyTorch can create tensors filled with zeros, ones, random values, or empty memory.

Code Example: Common Initialization Methods

```
size = (3, 4)
torch.empty(size)     # uninitialized
torch.rand(size)      # random 0{1
torch.zeros(size)     # all zeros
torch.ones(size)      # all ones
```

Real-life Example

In an LLM, random tensor initialization defines the model's initial weights. During training, these numbers are updated to minimize prediction errors.

—

4. Tensor Data Types and Devices

Tensors can specify data type and hardware device.

Code Example: Dtype and GPU Usage

```
tensor_fp16 = torch.rand(2, 2, dtype=torch.float16)

# Convert dtype
tensor_double = tensor_fp16.type(torch.double)

# Move to GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tensor_gpu = torch.ones((3, 7)).to(device)
```

—

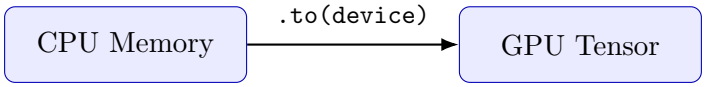


Figure 22: Transferring tensors between CPU and GPU in PyTorch.

5. Converting Between NumPy and Tensors

PyTorch integrates seamlessly with NumPy — both share memory.

Code Example: Tensor–NumPy Interoperability

```
import numpy as np

arr = np.array([[9, 3], [0, 4]])
tensor_from_np = torch.from_numpy(arr)
arr_back = tensor_from_np.numpy()
```

Real-life Example

When preprocessing text data using NumPy arrays, you can directly feed the same data into a PyTorch model without re-copying — saving memory and time.

—

6. Indexing and Slicing

Tensors support Python-style indexing and slicing.

Code Example: Accessing Tensor Elements

```
tensor2 = torch.tensor([[0, 1, 7],
                        [4, 2, 4]])

element = tensor2[1, 0]     # single value
row = tensor2[0]            # first row
column = tensor2[:, 2]      # third column
tensor2[1, 2] = 9           # modify element
```

7. Tensor Operations

Tensors allow vectorized arithmetic and linear algebra operations.

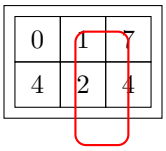


Figure 23: Matrix-style indexing in tensors.

Code Example: Arithmetic and Matrix Operations

```
a = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
b = torch.tensor([[5, 6], [7, 8]], dtype=torch.float32)

add = a + b
sub = a - b
matmul = torch.matmul(a, b)
elemwise = a * b
a_T = a.T
```

Example: Real Application

In transformer networks, `torch.matmul()` performs attention score computation — multiplying query and key matrices to model relationships between words.

—

8. Reshaping and Concatenation

Tensors can be reshaped or combined for model input batching.

Code Example: Reshaping and Concatenation

```
x = torch.arange(1, 10)
x_reshaped = x.view(3, 3)

x1 = torch.ones((2, 3))
x2 = torch.zeros((2, 3))
x_concat = torch.cat((x1, x2), dim=0)
```

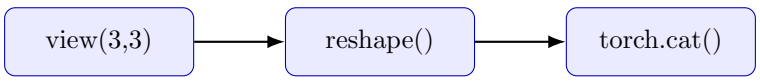


Figure 24: Reshaping and concatenation operations.

—

9. Automatic Differentiation (Autograd)

PyTorch's `autograd` module computes gradients automatically — essential for backpropagation.

Code Example: Gradient Computation

```
x = torch.tensor(2.0, requires_grad=True)
y = x**3 + 4*x**2 + 5*x + 6
y.backward()
print(x.grad)  # dy/dx = 33
```

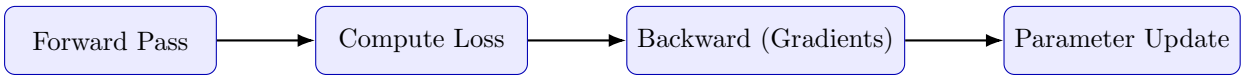


Figure 25: Backpropagation pipeline in neural training.

—

10. Gradient Flow Example

Gradients propagate through a computation graph:

Code Example: Backpropagation Through Graph

```
a = torch.tensor(3.0, requires_grad=True)
b = torch.tensor(4.0, requires_grad=True)

c = a**2 + b**2
d = torch.sqrt(c)
d.backward()
print(a.grad, b.grad)
```

Real-life Analogy

During model training, gradients act like GPS directions guiding each parameter to move in the direction that reduces the total loss.

—

11. Importance of Tensors in Language Modeling

Tensors store every internal value in an LLM — from token embeddings to attention weights and logits. Matrix multiplication and gradient flow drive learning and text generation.

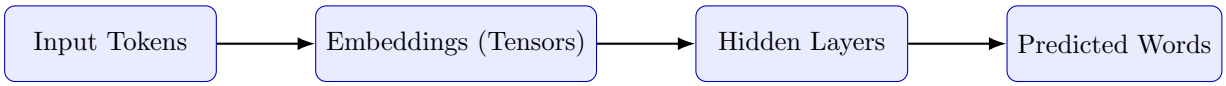


Figure 26: Tensors flowing through a neural language model.

—

Summary

- Tensors are multi-dimensional data containers used in every deep learning operation.
- PyTorch enables tensor creation, manipulation, and GPU acceleration.
- `autograd` automates gradient computation for model training.
- All internal computations in LLMs — embeddings, attention, gradients — rely on tensors.

Key Takeaway

Mastering tensors and PyTorch operations is the first step to understanding how large language models perform numerical reasoning and learn from data.

Word Representation and Embedding Models

Introduction

In Natural Language Processing (NLP), text must be represented numerically so that models can process it mathematically. The goal of word representation is to convert words into vectors while preserving meaning and relationships. Representation methods evolved from sparse encodings (like one-hot vectors) to dense, low-dimensional embeddings such as **Word2Vec**, **GloVe**, and **FastText**.

Real-life Analogy

Just as GPS coordinates locate places in space, embeddings locate words in a semantic space. Words with similar meanings (like "king" and "queen") appear close together — just as two nearby cities have similar coordinates.

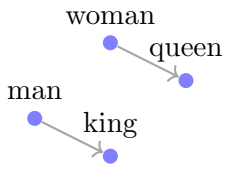


Figure 27: Word embeddings form a vector space where relationships (e.g., gender, tense) become geometric directions.

—

One-Hot Encoding

A **one-hot vector** represents each word with a unique index in a vocabulary of size V. All entries are 0 except one position, which is 1.

Word	One-hot Representation
apple	[1, 0, 0, 0]
banana	[0, 1, 0, 0]
mango	[0, 0, 1, 0]
orange	[0, 0, 0, 1]

Limitations:

- High dimensionality for large vocabularies.
- No concept of similarity — "apple" and "orange" are orthogonal.
- Fails to generalize to unseen words.

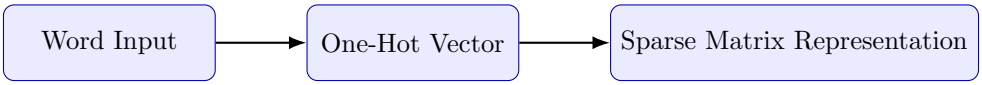


Figure 28: One-hot encoding converts discrete words into sparse numerical vectors.

—

Distributed Representations

Distributed (dense) embeddings map each word to a real-valued vector. Words with similar meaning appear close together in the vector space.

$$king - man + woman \approx queen$$

Real Example

In Google's Word2Vec model, "Paris – France + Japan ≈ Tokyo". The embedding space captures both syntactic and semantic relationships.

—

Count-Based Approaches

Co-occurrence Matrix

A co-occurrence matrix counts how often each word appears in the context of another. Example (context window = 1):

	cat	sat	mat
cat	0	1	0
sat	1	0	1
mat	0	1	0

TF–IDF Weighting

Reduces the effect of common words and emphasizes informative ones:

$$TF\text{-}IDF(w,d) = TF(w,d) \times \log \frac{N}{DF(w)}$$

Real-life Example

In a news corpus, "economy" appears frequently in finance articles but rarely elsewhere, giving it high TF–IDF weight — unlike common words like "the" or "is."

—

Prediction-Based Embeddings (Word2Vec)

Word2Vec learns embeddings by predicting context words using a neural network.

- **CBOW:** Predict current word from context.
- **Skip-Gram:** Predict context from current word.

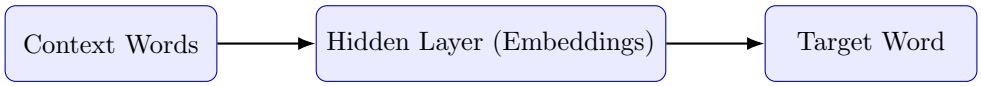


Figure 29: Word2Vec (CBOW): predicting a word from its context.

—

Negative Sampling

Instead of computing over the entire vocabulary, Word2Vec updates only a few "negative" examples at each step to improve efficiency.

Real Example

If "king" appears with "queen," the model also samples unrelated words like "car" or "table" as negatives, helping embeddings distinguish context relevance.

—

FastText

FastText extends Word2Vec by including subword information. Each word is represented as a bag of character n-grams.

Example

Word: "playing" → subwords = ⟨pla, play, layi, ayin, ying, ing⟩ Captures morphology: play → played → playing share overlapping subwords.

—

Global Vectors (GloVe)

GloVe combines the global co-occurrence statistics with local learning objectives.

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

Intuition

If "ice" and "steam" co-occur with "solid" and "gas" respectively, their vector difference reflects that semantic contrast.

—

Comparison of Embedding Methods

Method	Type	Context Used	Strength
Word2Vec	Prediction-based	Local	Efficient, scalable
GloVe	Hybrid (count + prediction)	Global	High semantic accuracy
FastText	Subword prediction	Local + morphology	Handles unseen words

—

Tokenization Strategies

Introduction

Tokenization divides text into smaller units (words or subwords) that serve as input for models. Modern subword methods like **Byte Pair Encoding (BPE)**, **WordPiece**, and **Unigram LM** solve problems of rare and unseen words.

Real-life Analogy

Just like breaking a long password into known patterns helps you recall it, subword tokenization breaks rare words into recognizable pieces that the model already understands.

—

Why Subword Tokenization?

- Reduces out-of-vocabulary (OOV) words.
- Captures word roots and affixes.
- Keeps vocabulary size manageable.

Example: “unhappiness” → “un” + “happi” + “ness”

—

Byte Pair Encoding (BPE)

Originally a compression algorithm, BPE merges frequently occurring symbol pairs.

Algorithm Summary

1. Start with character-level tokens. 2. Count adjacent pair frequencies. 3. Merge the most frequent pair. 4. Repeat until reaching the vocabulary size.

Example:

$$\text{t, h, e, r, e, s, u, n} \rightarrow \text{(t,h)} \rightarrow \text{th} \rightarrow \text{(th,e)} \rightarrow \text{the}$$

Result: "there sun" → [the, re, sun]

—

WordPiece Tokenization

Used in models like **BERT**. Merges subwords based on maximum likelihood gain.

$$\text{score(pair)} = \frac{\text{count(pair)}}{\text{count(first token)}}$$

Example: "playing" → [play, ##ing], "played" → [play, ##ed]

Observation

"##" denotes continuation of a subword — the model knows "play" and "ing" belong together.

—

Unigram Language Model Tokenization

Used in **SentencePiece**. Tokenization is treated as a probabilistic process.

$$P(W) = \prod_{i=1}^{N} P(t_i)$$

Advantages

- Can produce multiple valid segmentations.
- Handles languages with complex morphology.
- Used in GPT and T5 pipelines.

—

Comparison of Tokenization Methods

Method	Type	Key Idea	Used In
BPE	Deterministic	Merge frequent pairs	GPT, GPT-2
WordPiece	Likelihood-based	Maximize probability gain	BERT
Unigram LM	Probabilistic	Remove least likely tokens	SentencePiece, T5

—

Summary

- Word embeddings map words into vector space; tokenization breaks text into processable units.
- Together, they define how models interpret, learn, and generalize across languages.
- Modern tokenization bridges vocabulary efficiency with semantic accuracy.

Key Takeaway

Embeddings capture meaning; tokenization defines structure. Both are foundational for all modern LLM architectures.

Neural Language Models and Attention

Introduction

Neural Language Models (NLMs) replace discrete, count-based predictions with *learned* continuous functions that map contexts to next-token probabilities. They address core limitations of statistical models (sparsity, fixed context, lack of semantics) by learning dense embeddings and parameterizing conditional distributions with neural networks.

Why neural LMs matter

Neural LMs let systems generalize to unseen text (e.g., "quantum entanglement" after seeing "quantum mechanics" and "entanglement"), capture semantic relations, and support downstream fine-tuning for many NLP tasks.

Mathematical foundation (reminder)

Given tokens $x^{(1)}, \ldots, x^{(T)}$, an autoregressive model estimates

$$P(x^{(1:T)}) = \prod_{t=1}^{T} P\big(x^{(t)} \mid x^{(1:t-1)}\big).$$

Training minimizes negative log-likelihood (cross-entropy):

$$J(\theta) = -\sum_{t=1}^{T} \log P\big(x^{(t)} \mid x^{(1:t-1)}; \theta\big).$$

Observation

Everything below—fixed-window nets, RNNs, LSTM/GRU, attention—are different architectures that implement this conditional prediction with different inductive biases.

Fixed-window (feed-forward) neural LM

Idea: Use a fixed-length context (e.g., last k tokens). Each token $\rightarrow$ embedding $\mathbf{e}$; concatenate $[\mathbf{e}_{t-k}, \ldots, \mathbf{e}_{t-1}]$, pass through dense layers, softmax to predict next token.

$$\mathbf{h} = \phi\big(W[\mathbf{e}_{t-k}; \ldots; \mathbf{e}_{t-1}] + b\big), \quad P(x_t) = \text{softmax}(U\mathbf{h} + c)$$

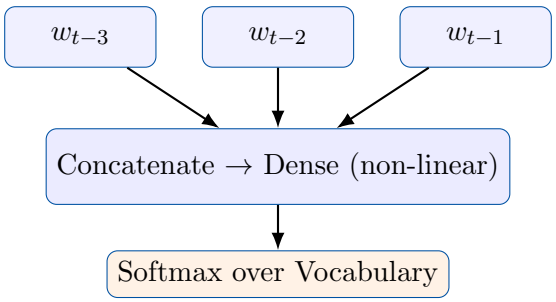


Figure 30: Fixed-window (feed-forward) neural language model — embeddings concatenated into dense layers.

Pros: simple, learns embeddings; **Cons:** fixed context, parameter blow-up as k grows, positional asymmetry.

Recurrent Neural Networks (RNNs)

RNNs maintain a hidden state h_t that summarizes past tokens:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \qquad y_t = \text{softmax}(W_{hy}h_t + b_y).$$

They are naturally suited for variable-length sequences and share parameters over time.

Backpropagation Through Time (BPTT)

Gradients for weights shared across time are sums of contributions from every timestep. The chain of Jacobians can make gradients shrink (vanish) or grow (explode).

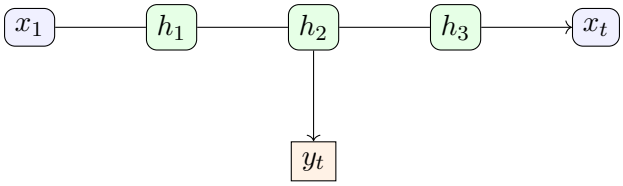


Figure 31: RNN unrolled through time — hidden state h_t propagates information.

$$\frac{\partial J}{\partial W_{hh}} = \sum_t \sum_{k \le t} \frac{\partial J_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}.$$

Vanishing / Exploding gradients — intuition and cure

If $\|W_{hh}\| < 1$ repeatedly multiplying Jacobians shrinks gradients exponentially; if $\|W_{hh}\| > 1$ they explode.

Practical analogy

Think of passing a whisper along a long chain of people: if each whisperer reduces volume slightly, the message vanishes after many people; if each amplifies slightly, it explodes into noise.

Architectural solutions: gated RNNs (LSTM, GRU) and attention.

Long Short-Term Memory (LSTM)

Motivation: Vanilla RNNs struggle with long-term dependencies because gradients vanish or explode through repeated multiplications. **LSTMs (Hochreiter & Schmidhuber, 1997)** solve this by introducing a *cell state* c_t — a memory pathway regulated by *gates* that decide what to keep, forget, and output.

$$
\begin{aligned}
i_t &= \sigma(W_i[x_t, h_{t-1}] + b_i) && \text{(Input gate)} \\
f_t &= \sigma(W_f[x_t, h_{t-1}] + b_f) && \text{(Forget gate)} \\
o_t &= \sigma(W_o[x_t, h_{t-1}] + b_o) && \text{(Output gate)} \\
\tilde{c}_t &= \tanh(W_c[x_t, h_{t-1}] + b_c) && \text{(Candidate state)} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{(Updated memory)} \\
h_t &= o_t \odot \tanh(c_t) && \text{(Hidden state / output)}
\end{aligned}
$$

Why LSTM Helps

If $f_t \approx 1$ and $i_t \approx 0$, then $c_t \approx c_{t-1}$ — maintaining nearly constant error flow and reducing vanishing gradients. The gates enable selective memory updates: the network dynamically decides what to remember and what to forget.

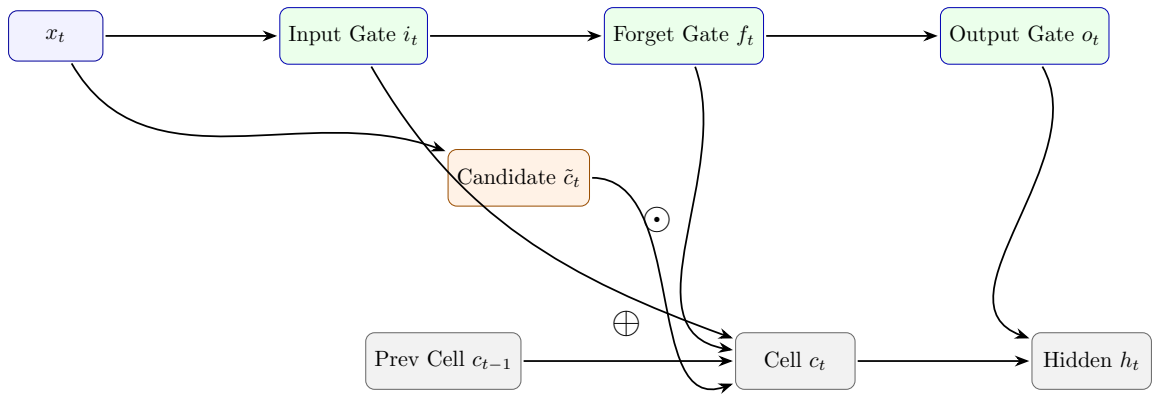


Figure 32: LSTM cell architecture — gates regulate information flow, maintaining long-term dependencies through the cell state c_t.

Real-life Analogy

Think of an LSTM as a long-term memory diary:

- The **input gate** decides what new events to record.
- The **forget gate** erases unimportant memories.
- The **cell state** stores accumulated knowledge.
- The **output gate** decides what to recall now.

Just like humans, it selectively remembers key details and forgets noise.

Gated Recurrent Unit (GRU)

Motivation: LSTM networks are powerful but computationally heavy — they maintain multiple gates and a separate cell state. **Gated Recurrent Units (GRUs)** simplify this structure by merging the **input** and **forget** gates into a single **update gate** z_t, and by combining the hidden and cell states into one unified state h_t.

$z_t = \sigma(W_z[x_t, h_{t-1}] + b_z)$	(Update gate — how much to keep from the past)
$r_t = \sigma(W_r[x_t, h_{t-1}] + b_r)$	(Reset gate — how much past to ignore)
$\tilde{h}_t = \tanh\left(W_h[x_t,\ r_t \odot h_{t-1}] + b_h\right)$	(Candidate activation)
$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	(Final hidden state / output)

How GRU Works:

- The **reset gate** (r_t) controls how much of the previous state (h_{t-1}) to forget before computing the new candidate $\tilde{h}_t$.
- The **update gate** (z_t) determines how much of the new candidate $\tilde{h}_t$ replaces the old hidden state.
- When z_t is small, GRU keeps most of the old memory (h_{t-1}); when z_t is large, it updates quickly with new information.

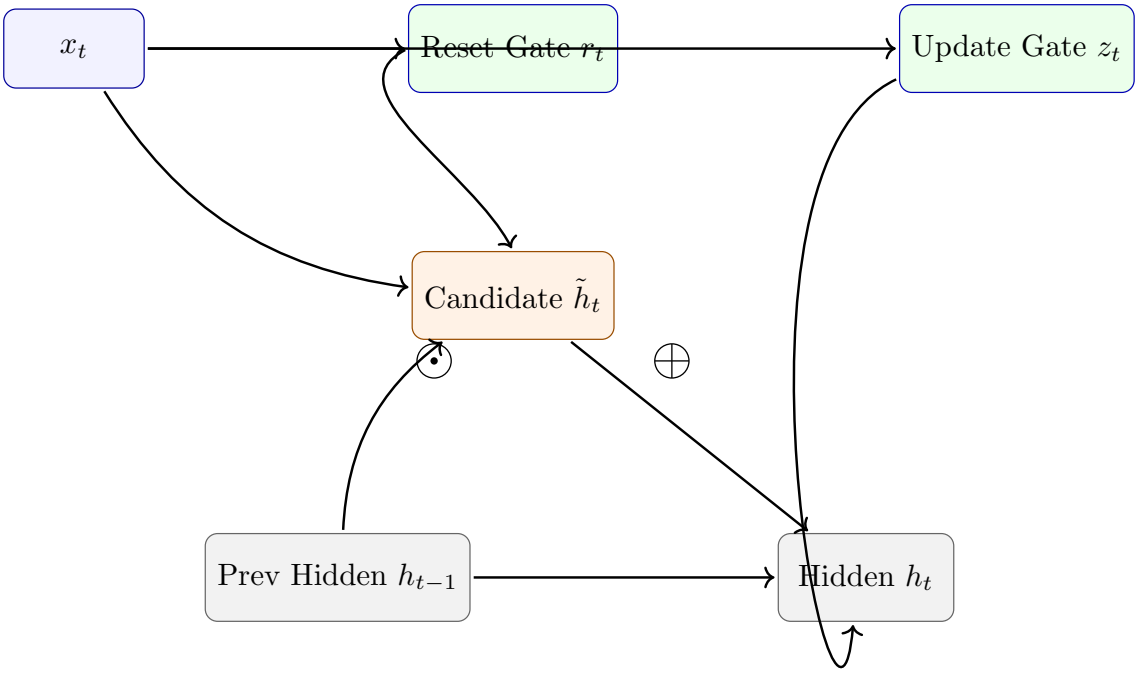


Figure 33: GRU cell architecture — reset and update gates regulate memory flow, merging LSTM functionality in a simpler structure.

Tradeoff: GRUs train faster and require fewer parameters than LSTMs, making them preferred in applications where memory length is moderate (e.g., speech or short text). LSTMs may outperform GRUs on tasks needing very long-term dependencies.

Real-life Analogy

Imagine GRU as an employee balancing old and new work:

- **Reset gate (r_t):** decides whether to discard outdated project notes.
- **Update gate (z_t):** determines how much of the new task information to integrate.
- The final memory (h_t) blends experience (h_{t-1}) with current knowledge ($\tilde{h}_t$).

GRU thus works efficiently with fewer "management layers" than an LSTM.

Bidirectional and Stacked RNNs

Bidirectional RNN (BiRNN). A Bidirectional RNN runs two RNNs over the sequence: one forward (left-to-right) and one backward (right-to-left). For each position t the model concatenates (or sums) the forward and backward hidden states to obtain a richer representation that contains both past and future context:

$$\overrightarrow{h}_t = \mathrm{RNN}_{\mathrm{fwd}}(x_{1:t}), \qquad \overleftarrow{h}_t = \mathrm{RNN}_{\mathrm{bwd}}(x_{T:t}),$$

$$h_t = \left[\overrightarrow{h}_t\,;\,\overleftarrow{h}_t\right] \quad \text{(concatenation)}$$

This is especially useful for sequence labeling tasks (POS tagging, NER) where information from both sides of a token improves decisions.

Real-life Analogy

Reading a word with bi-directional context is like understanding a word in a sentence by glancing both at the words before it and the words after it — you get the complete local meaning.

Stacked (Multi-layer) RNNs. Stacked RNNs place multiple recurrent layers on top of each other. The hidden sequence from layer ℓ becomes the input to layer $\ell + 1$:

$$h_t^{(1)} = \mathrm{RNN}^{(1)}(x_{1:t}), \quad h_t^{(2)} = \mathrm{RNN}^{(2)}(h_{1:t}^{(1)}), \quad \ldots$$

Lower layers typically learn short-range patterns; higher layers capture more abstract or longer-range features.

Why use them?

- **BiRNN:** captures both prior and future contexts for each token — improves token-level tasks.
- **Stacked RNN:** builds hierarchical representations, often improving model capacity without increasing per-step context size.

Practical notes and caveats:

- **Latency:** Bidirectional models cannot be used in streaming left-to-right generation without lookahead because they require future tokens. For online generation tasks, use unidirectional or chunked approaches.
- **Parameter growth:** Stacking increases parameters and compute; balance depth with available data and compute budget.

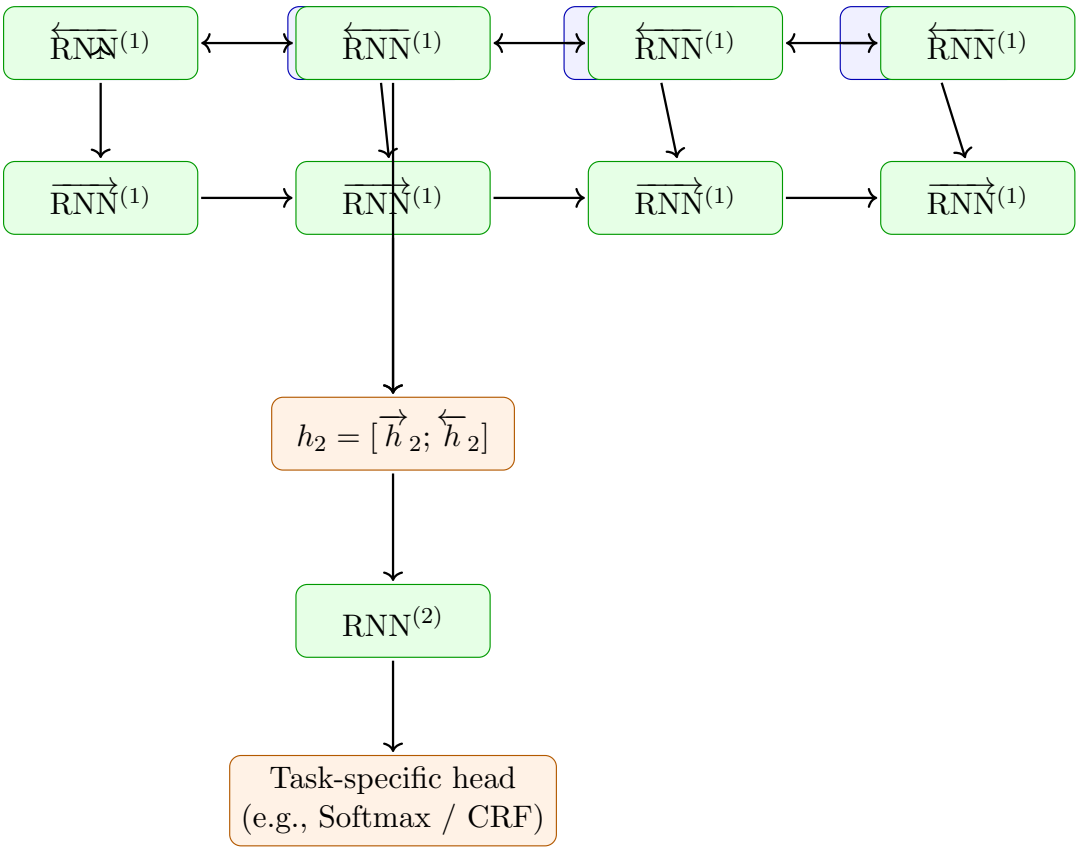


Figure 34: Bidirectional (forward + backward) RNNs produce per-token representations by concatenating $\overrightarrow{h}_t$ and $\overleftarrow{h}_t$. Stacked layers (e.g., $\text{RNN}^{(2)}$) build hierarchical features used by downstream task heads.

- **Initialization and regularization:** Use dropout between layers and proper initialization (e.g., Xavier/He) to stabilize training.

When to choose which

Use BiRNNs when full-sequence information is available (e.g., POS tagging, NER, sentence classification). **Use Stacked RNNs** when you need greater representation power but can afford extra compute (e.g., complex sequence-to-sequence encoders).

EXTRA(Encoder, Decoder, and Autoencoders)

Encoder Networks

Concept: An **Encoder** is a neural network that processes an input sequence or vector and compresses it into a dense, informative representation — typically a hidden state or latent vector z. This vector captures the most salient features of the input in a lower-dimensional space.

$$z = f_{\text{enc}}(x_1, x_2, \ldots, x_T)$$

In Seq2Seq models, the encoder is often an RNN (or LSTM/GRU) that reads the input sequentially and outputs a final hidden state summarizing the entire input.

Intuition

The encoder "understands" the input and compresses its meaning into a fixed-length internal thought vector, much like how humans summarize a long paragraph into key points.

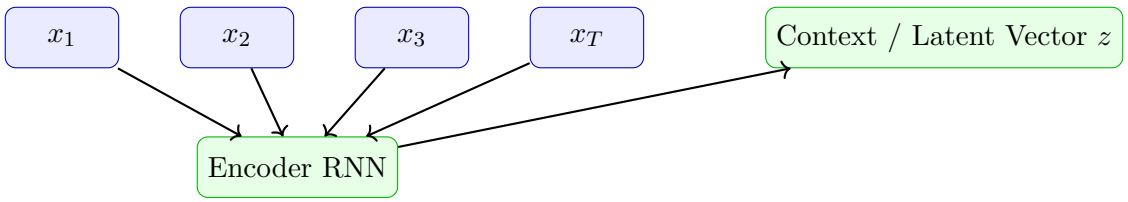


Figure 35: Encoder compresses input sequence into a compact latent vector z summarizing its meaning.

—

Decoder Networks

Concept: A **Decoder** takes a latent representation z (or encoder context vector) and reconstructs or generates an output sequence. It models conditional probability:

$$P(y \mid z) = \prod_t P(y_t \mid y_{<t}, z)$$

The decoder is often an RNN (or Transformer block) that receives z as input and generates tokens step-by-step.

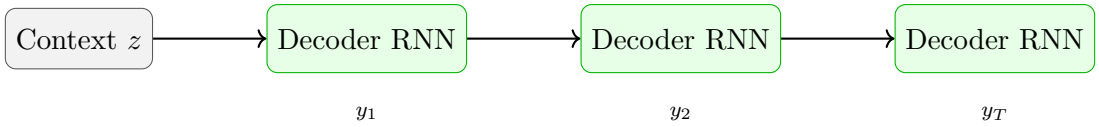


Figure 36: Decoder generates the output sequence token-by-token from context vector z.

Real-life Analogy

If the encoder acts like "understanding a sentence," the decoder is like "expressing that understanding" — for instance, translating thoughts (latent vector) back into words.

—

Autoencoders

Definition: An **Autoencoder** is a neural network designed to learn efficient data encodings. It consists of:

- **Encoder:** compresses the input into a lower-dimensional latent representation z.
- **Decoder:** reconstructs the input from this compressed latent space.

$$z = f_{\text{enc}}(x), \quad \hat{x} = f_{\text{dec}}(z)$$

Training Objective: Minimize reconstruction error between input x and reconstructed $\hat{x}$:

$$J(\theta) = \|x - \hat{x}\|^2$$

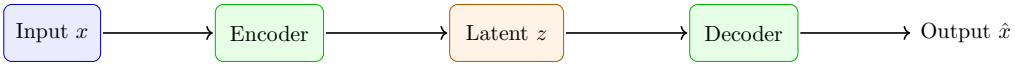


Figure 37: Autoencoder structure — the encoder compresses input into a latent representation z, and the decoder reconstructs it back to $\hat{x}$.

Why Autoencoders Matter

- Learn unsupervised representations without labeled data.
- Reduce dimensionality and remove noise from signals.
- Used in image compression, anomaly detection, and pretraining.

Variants:

- **Denoising Autoencoder:** learns to reconstruct clean input from noisy data.
- **Sparse Autoencoder:** encourages sparsity in latent activations (like feature selection).
- **Variational Autoencoder (VAE):** introduces probabilistic latent variables, enabling generative modeling.

Real-life Analogy

Think of an autoencoder as a human learning summary skills: You hear a long paragraph (**Encoder**), remember only the essence (**Latent** z), and then retell it in your own words (**Decoder**) — preserving meaning while compressing details.

—

Key Differences

Aspect	Encoder–Decoder (Seq2Seq)	Autoencoder
Input–Output	Different domains (e.g., English → French)	Same domain (input ≈ output)
Goal	Generate or translate sequence	Reconstruct input efficiently
Supervision	Supervised (requires target output)	Unsupervised (no target labels)
Context vector	Used for decoding new data	Used for self-reconstruction
Applications	Translation, summarization, chatbots	Compression, anomaly detection, representation learning

—

Key Takeaway

Encoders compress, decoders expand, and autoencoders learn to do both together. These architectures form the conceptual foundation of modern Transformers and Large Language Models(EXTRA TILL HERE).

Sequence-to-Sequence (Seq2Seq) Models

Concept: A Sequence-to-Sequence (Seq2Seq) model maps an input sequence $x = (x_1, \ldots, x_T)$ to an output sequence $y = (y_1, \ldots, y_{T'})$. It consists of two main components:

- An **Encoder** RNN that reads and encodes the input sequence into a single fixed-length vector (the "context" or "thought" vector).
- A **Decoder** RNN that generates the target sequence token by token, conditioned on that context.

Mathematically:

$$P(y \mid x) = \prod_{t=1}^{T'} P(y_t \mid y_{<t}, x)$$

Intuition: The encoder summarizes the entire input (like remembering the meaning of a sentence), and the decoder reconstructs or translates it word by word.

Real-life Analogy

Imagine listening to a sentence in French (*encoder phase*) and then repeating it in English (*decoder phase*). The mental summary you keep before starting to speak is the **context vector**.

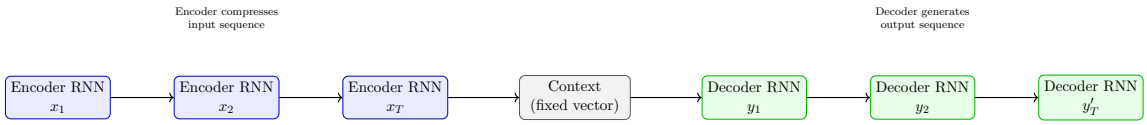


Figure 38: Basic Seq2Seq encoder–decoder architecture (without attention). The encoder compresses input into a single vector, which the decoder uses to generate the target sequence.

Limitation: The single context vector becomes a **bottleneck** for long sentences—important details can be lost since the decoder receives only a single summary vector regardless of input length. This problem motivated the introduction of the **Attention Mechanism**.

Decoding Strategies (Practical Methods)

When generating text, the decoder predicts one word at a time from a probability distribution $P(y_t \mid y_{<t}, x)$. Different decoding strategies control how the next word is chosen.

- **Greedy Decoding:** Select the highest-probability token at each step. *Fast but can lead to locally optimal (not global) sequences.*
- **Beam Search:** Keeps the top-k candidate sequences ("beams") at every step and expands them. *Balances exploration with efficiency.*
- **Stochastic Sampling:** Randomly samples next tokens according to their probabilities (useful for creativity).

Beam Search Intuition

Beam search avoids early mistakes by maintaining k high-probability partial outputs at each decoding step. When $k = 1$, it becomes greedy decoding; larger k values yield better global coherence but increase computation.

Practical Insights:

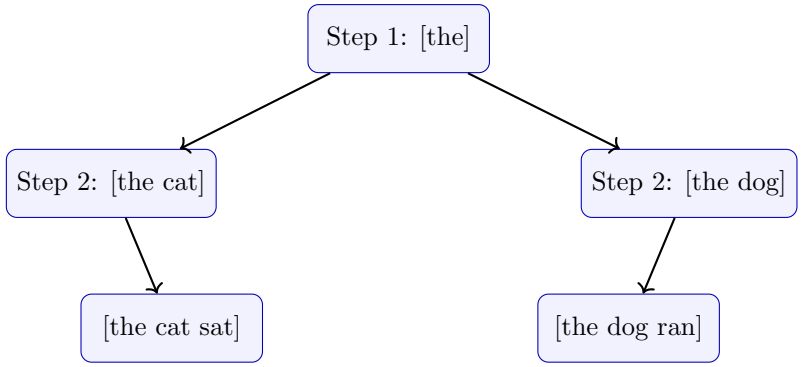


Figure 39: Beam search expands multiple candidate sequences simultaneously, balancing quality and speed.

- Use **beam search** ($k = 5$–10) for translation and summarization tasks.
- Use **sampling** (Top-p, Top-k, Temperature) for creative text like story generation or dialogue.
- For deterministic responses, lower temperature ($\tau \approx 0.7$); for diversity, increase it ($\tau \approx 1.5$).

Attention: Motivation and Mechanism

Problem: In the vanilla Seq2Seq model, the encoder compresses the entire input into one fixed-length vector, which becomes a **bottleneck**. For long sentences, this fixed-size vector loses important information from distant tokens.

Solution: Attention allows the decoder to "look back" at all encoder hidden states during decoding — dynamically deciding which input words are most relevant to the current output.

Intuitive View

At each decoding step, the model decides where to "pay attention" in the input sequence. It computes alignment weights that tell it how strongly each encoder word contributes to the current output token.

Mathematical formulation: Given encoder hidden states $h_1, h_2, ..., h_T$ and decoder hidden state s_t at time step t:

$$e_{t,i} = \text{score}(s_t, h_i) \quad \text{(relevance between decoder and each encoder state)}$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})} \quad \text{(normalized attention weights via softmax)}$$

$$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i \quad \text{(context vector as weighted sum of encoder states)}$$

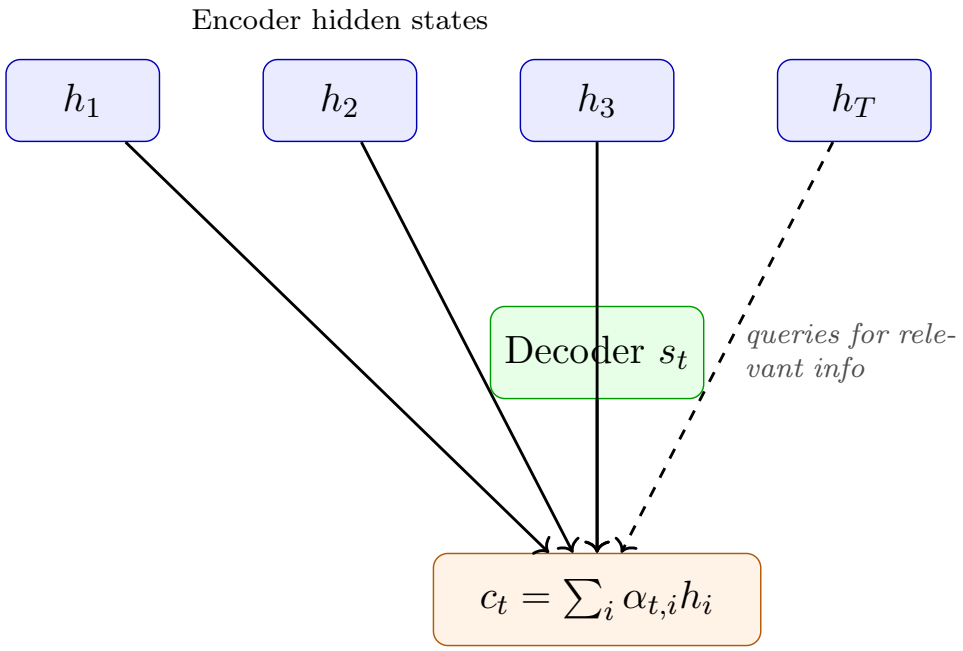


Figure 40: Attention mechanism — the decoder queries encoder states and combines them (weighted by $\alpha_{t,i}$) to form the dynamic context vector c_t.

Score functions (alignment models)

The **score function** determines how similarity between s_t and h_i is measured.

$$\text{Dot:} \quad e_{t,i} = s_t^\top h_i$$

$$\text{Scaled Dot:} \quad e_{t,i} = \frac{s_t^\top h_i}{\sqrt{d}}$$

$$\text{Additive (Bahdanau):} \quad e_{t,i} = v_a^\top \tanh(W_s s_t + W_h h_i)$$

Interpretation

Each $e_{t,i}$ quantifies how well the encoder word i matches what the decoder currently needs. The softmax converts these scores into attention weights $\alpha_{t,i}$, forming a probability distribution over input words.

Decoder computation with attention

At each decoding step:

$$s_t = f(s_{t-1}, y_{t-1}, c_t)$$

$$P(y_t \mid y_{<t}, x) = \text{softmax}(W_o[s_t; c_t] + b_o)$$

The attention-weighted context c_t dynamically guides the decoder to focus on the most relevant parts of the source sequence.

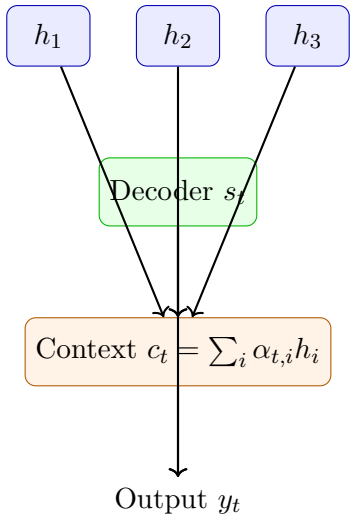


Figure 41: Figure 38: Flow of computation in attention — the decoder combines its state s_t with context c_t to generate the next output y_t.

Step-by-step process (Bahdanau attention)

1. Encoder processes all inputs $x_1, \ldots, x_T$ to produce hidden states $h_1, \ldots, h_T$.
2. Decoder receives previous output y_{t-1} and hidden state s_{t-1}.
3. Compute energy scores $e_{t,i} = v_a^\top \tanh(W_s s_{t-1} + W_h h_i)$.
4. Normalize scores with softmax to obtain attention weights $\alpha_{t,i}$.
5. Compute context vector $c_t = \sum_i \alpha_{t,i} h_i$.
6. Update decoder hidden state: $s_t = f(s_{t-1}, y_{t-1}, c_t)$.
7. Predict next token: $P(y_t|y_{<t}, x) = \text{softmax}(W_o[s_t; c_t] + b_o)$.

Real-life Analogy

Think of attention as reading a book while translating: you don't memorize the entire sentence first — instead, at each translation step, you glance back at the most relevant words in the source text.

Summary and Key Insights

- Attention dynamically weights encoder outputs — solving the fixed-context bottleneck.
- It improves translation, summarization, and question answering by allowing focus on relevant words.
- Attention forms the foundation for the **Transformer** model through **self-attention**, where every token attends to all others.

Transformer Architecture-1

Motivation: Beyond Recurrent Models

Recurrent Neural Networks (RNNs) and LSTMs process tokens sequentially, which limits parallelization and struggles with long-term dependencies. Attention mechanisms allowed models to focus on relevant input tokens, but still required recurrence to capture order. The question arises — can we design a sequence model using **only attention**?

Core Idea

Transformers eliminate recurrence entirely, replacing it with **self-attention layers** that can attend to all positions in a sequence simultaneously.

Self-Attention Mechanism

Self-attention enables every token in a sentence to compute a representation that depends on all other tokens.

- Each token creates three vectors — **Query (Q)**, **Key (K)**, and **Value (V)** — through learned linear projections.
- Attention scores are calculated as scaled dot-products between queries and keys.
- Softmax normalizes these scores into attention weights.
- The final output is the weighted sum of all value vectors.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

Here, Q = Query matrix, K = Key matrix, V = Value matrix, and d_k = dimension of the key vectors (used for scaling).

Why Scaling by $\sqrt{d_k}$?

Without scaling, dot products between large-dimensional vectors can become very large, causing the softmax to saturate and gradients to vanish. Dividing by $\sqrt{d_k}$ stabilizes the gradient and improves convergence.

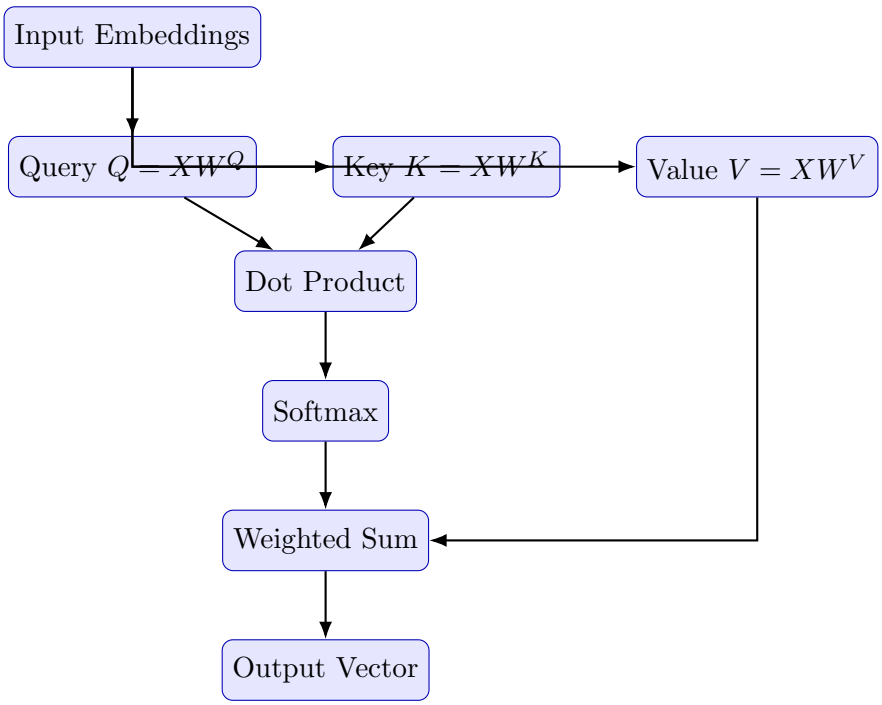


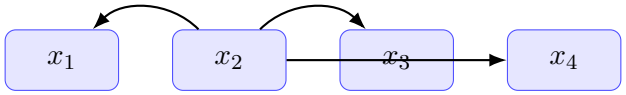
Figure 42: Flow of computations inside a single self-attention head.

Why Self-Attention?

- Removes the sequential bottleneck of RNNs — enabling full parallelism.
- Provides direct paths between all words, improving long-range dependency modeling.
- Learns contextual meaning by dynamically weighting token relationships.

Real-life Analogy

Imagine reading a paragraph — instead of remembering previous words one by one, you can instantly glance back at any earlier word to interpret meaning. That's what self-attention does for tokens.



Each token attends to all others

Figure 43: Self-Attention: each token computes weighted attention over all tokens.

From Self-Attention to Transformers

Although self-attention captures relationships, it lacks:

1. **Positional Information** — order of words is ignored.
2. **Multiple Representation Subspaces** — only one attention view.
3. **Nonlinearity** — each attention layer is linear in its inputs.
4. **Masked Decoding** — future information must be hidden during generation.

Key Design Additions

To overcome these issues, the Transformer introduces:

1. **Positional Encoding**
2. **Multi-Head Attention**
3. **Feed-Forward Nonlinear Layers**
4. **Masked Self-Attention (for Decoding)**

Positional Encoding: Why It's Needed

Self-attention treats inputs as a "bag of words" — permutation invariant. For sentences like "*The sun rises in the east*" vs. "*Rises the sun in the east*", meaning changes, but self-attention cannot tell the difference.

Problem

Self-attention has no notion of order — all words look equally distant.

Sinusoidal Positional Encoding

We inject sequence order by adding sinusoidal signals to word embeddings:

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad \text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{2}$$

- Provides both absolute and relative position information.
- Works for sequences longer than seen during training.
- Ensures continuity between nearby positions.

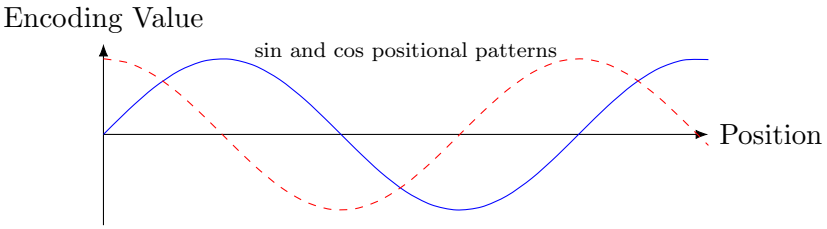


Figure 44: Sinusoidal positional encoding encodes sequence order using continuous wave patterns.

Multi-Head Attention

- A single attention head may focus too narrowly on one dependency.
- Multi-head attention uses multiple (Q, K, V) projections, enabling different heads to focus on different relationships (e.g., subject, verb, object).

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{3}$$

Each head is computed as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{4}$$

Analogy

Multiple heads act like different "readers" interpreting the same sentence from varied perspectives — one might focus on grammar, another on meaning.

Feed-Forward Networks (FFN)

After each attention layer, a position-wise feed-forward network introduces nonlinearity:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \tag{5}$$

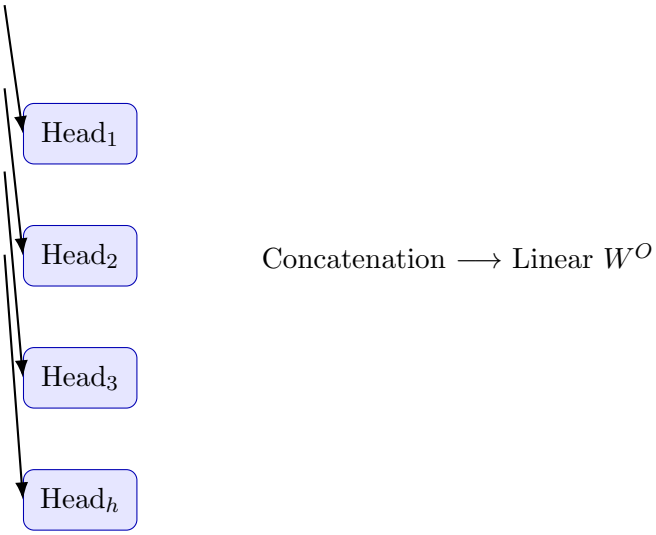


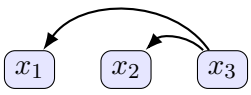
Figure 45: Multi-head attention: multiple subspaces learn different relations.

- Same weights are applied independently to each position.
- Adds expressive power and deeper feature transformation.

Masked Self-Attention

During text generation (decoding), each position should only attend to previous positions — not future ones.

$$\alpha_{t,i} = \begin{cases} \text{softmax}\left(\frac{Q_t K_i^\top}{\sqrt{d_k}}\right), & \text{if } i \leq t \\ 0, & \text{if } i > t \end{cases} \tag{6}$$



Only previous tokens are visible during decoding.

Figure 46: Masked self-attention: prevents future information leakage.

Summary and Key Insights

- Transformers use self-attention instead of recurrence.
- Positional encodings inject order into sequences.
- Multi-head attention enables diverse contextual views.

- Feed-forward networks add nonlinearity and capacity.
- Masked decoding ensures causal generation for language modeling.

Transformer Architecture-2

Overview

In Part 1, we saw how self-attention replaces recurrence. In this section, we build the **complete Transformer** by combining several crucial components:

1. Positional encodings — introduce token order.
2. Multi-head attention — capture multiple relations.
3. Feed-forward layers — add non-linearity.
4. Residual and normalization layers — ensure stable deep training.
5. Encoder–decoder structure — enable tasks such as translation or summarization.

High-Level View

Each encoder/decoder block can be imagined as a **modular circuit** combining: attention ⇒ normalization ⇒ feed-forward ⇒ residual addition. Stacking many such modules forms the full Transformer.

Absolute Positional Encoding (Fixed Sinusoids)

Self-attention itself is order-agnostic. To encode order, the original Transformer (Vaswani et al., 2017) used **sinusoidal positional encodings** added to token embeddings.

$$\mathrm{PE}_{(pos,2i)} = \sin\Big(\frac{pos}{10000^{2i/d_{model}}}\Big), \qquad \mathrm{PE}_{(pos,2i+1)} = \cos\Big(\frac{pos}{10000^{2i/d_{model}}}\Big) \tag{7}$$

- pos — token position, i — embedding dimension index.
- The mixture of sines and cosines allows interpolation between positions.
- Different frequencies enable the model to distinguish fine and coarse distances.

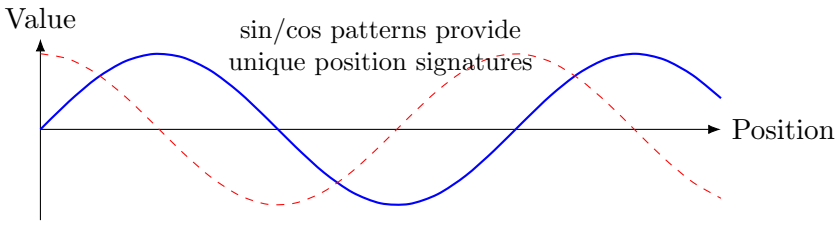


Figure 47: Sinusoidal positional encodings at different frequencies.

Real-life Analogy

Imagine each position emitting a pair of sine and cosine waves of different frequencies — the combination acts like a unique **barcode** for that word's location in the sequence.

Learned Positional Embeddings

Instead of fixed formulas, the model can **learn** position vectors:

$$E_{pos} = \text{Embedding}(pos)$$

These are parameters trained alongside word embeddings.

Pros:

- Adapts to dataset-specific sequence patterns.
- Often improves convergence on shorter sequences.

Cons:

- Cannot extrapolate to sequences longer than those seen during training.

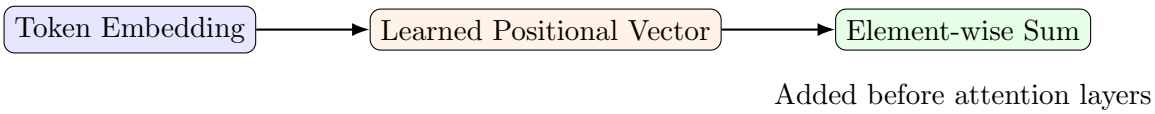


Figure 48: Combining token and positional embeddings.

Relative Positional Encoding

For long text, relative distance is more meaningful than absolute index. Relative encoding modifies attention weights:

$$A_{ij} = \frac{Q_i(K_j + P_{i-j})^\top}{\sqrt{d_k}}$$

where P_{i-j} is a learned embedding of the relative offset between tokens i and j.

Why Relative Positions?

The phrase "New York City" should retain similar relationships wherever it appears. Relative encodings make the model **translation-invariant**.

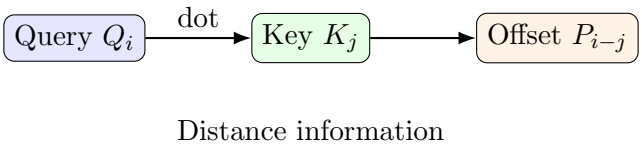


Figure 49: Relative positional information adjusts key representations.

Rotary Positional Encoding (RoPE)

RoPE expresses position as a rotation in embedding space.

$$\text{RoPE}(x, pos) = \begin{bmatrix} x_{2i} \cos\theta_{pos} - x_{2i+1} \sin\theta_{pos} \\ x_{2i} \sin\theta_{pos} + x_{2i+1} \cos\theta_{pos} \end{bmatrix} \quad \text{where } \theta_{pos} = pos/10000^{2i/d_{model}}$$

- Preserves relative distances through rotational invariance.
- Multiplicative — no addition to embeddings required.
- Used in GPT-Neo, LLaMA, PaLM.

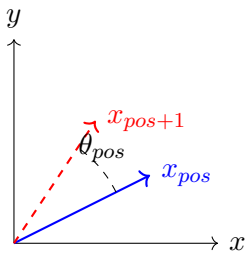


Figure 50: RoPE rotates embeddings according to position index.

Geometric Intuition

Two tokens at the same relative distance produce the same angle difference — so attention between them remains unchanged even if the sequence is shifted.

Residual Connections

Deep attention stacks are prone to gradient vanishing. Residual (skip) connections solve this by letting information bypass the transformation:

$$x_{l+1} = \text{Layer}(x_l) + x_l$$

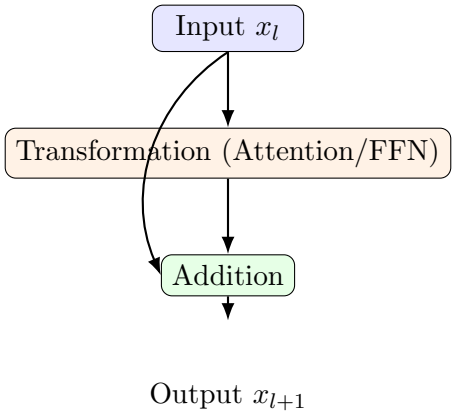


Figure 51: Residual path allows gradient flow across layers.

Training Benefit

Residual links act like express highways for gradients, helping the network converge faster and maintain previously learned features.

Normalization Layers

Layer activations can drift over time (covariate shift). **Normalization** rescales features to maintain stable distributions.

Batch Normalization:

- Normalizes using statistics across the mini-batch.
- Works well in vision models, but not for variable-length sequences.

Layer Normalization:

- Computes statistics across feature dimensions of each token vector:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma + \epsilon} \cdot \gamma + \beta$$

- Independent of batch size, ideal for Transformers.
- Applied before or after each sub-layer depending on architecture variant.

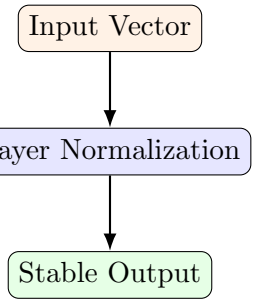


Figure 52: Layer normalization rescales features per token.

Comparison

Batch Norm: across samples in batch sensitive to batch size. **Layer Norm:** across features in a token robust for sequences.

Encoder–Decoder Composition

Each encoder layer outputs contextual embeddings; the decoder uses two attentions — one on itself (masked) and one on encoder outputs.

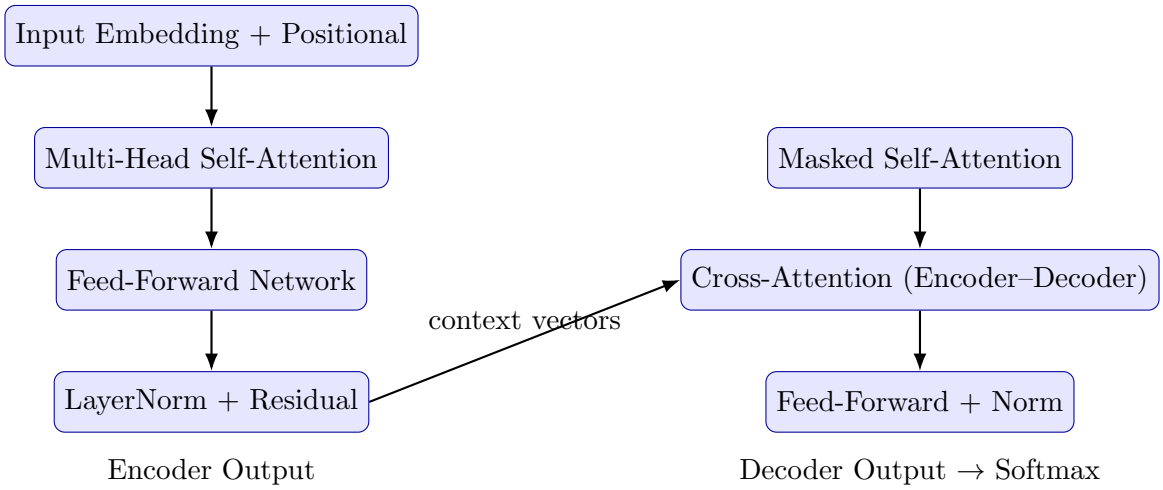


Figure 53: Encoder–decoder data flow inside the Transformer.

Flow Summary

Encoder: learns bidirectional context of input.
Decoder: generates target sequence step-by-step using masked and cross-attention.

Putting It All Together

Each Transformer block can be summarized as:

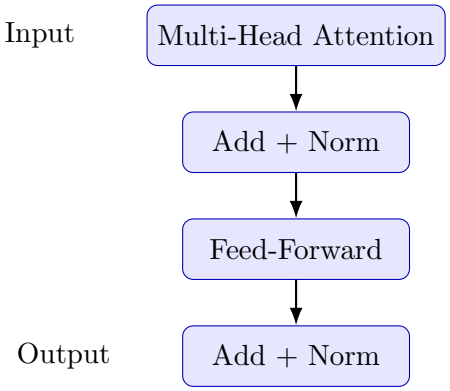


Figure 54: Internal layout of one Transformer encoder block.

Key Insight

Attention tells *where to look*, Feed-Forward tells *how to process*, Normalization keeps values stable, Residuals preserve information flow.

Summary and Key Insights

- Transformers replace recurrence with parallel self-attention.
- Positional information (absolute, relative, rotary) restores order sensitivity.
- Multi-head structure enables multiple semantic relationships.
- Residual and normalization layers ensure deep-network stability.
- Encoder–decoder stacking generalizes across NLP, vision, and speech.

Transformer Implementation in PyTorch

Overview

After understanding the Transformer architecture, we now explore how it is implemented in **PyTorch**. This lecture connects equations to code — showing how queries, keys, and values are computed, attention matrices are formed, and how entire encoder–decoder layers are built.

Learning Objectives

- Implement self-attention and multi-head attention in PyTorch.
- Construct encoder and decoder layers using `nn.Module`.
- Understand tensor dimensions and matrix operations.
- Combine all modules into a full Transformer model.

Scaled Dot-Product Attention

Code Example: Scaled Dot-Product Attention

```
import torch
import torch.nn.functional as F

def attention(Q, K, V, mask=None):
    # Q, K, V: [batch, head, seq_len, d_k]
    scores = torch.matmul(Q, K.transpose(-2, -1)) / (K.size(-1) **
        0.5)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
    attn_weights = F.softmax(scores, dim=-1)
    output = torch.matmul(attn_weights, V)
    return output, attn_weights
```

Output Example

```
Output Tensor Shape:  [batch, head, seq_len, d_k]
Attention Weights Shape:  [batch, head, seq_len, seq_len]
```

Explanation:

- Each query vector Q attends to all key vectors K.
- Scaling by $\sqrt{d_k}$ avoids large gradients.
- The mask ensures no future tokens are seen during decoding.

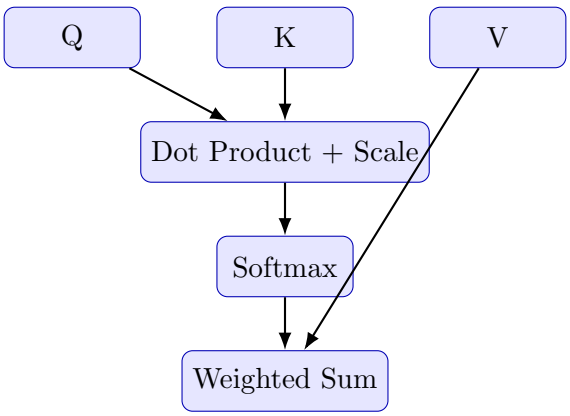


Figure 55: Computation flow of scaled dot-product attention.

Multi-Head Attention Module

Code Example: Multi-Head Attention Implementation

```
class MultiHeadAttention(torch.nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.num_heads = num_heads

        self.W_Q = torch.nn.Linear(d_model, d_model)
        self.W_K = torch.nn.Linear(d_model, d_model)
        self.W_V = torch.nn.Linear(d_model, d_model)
        self.W_O = torch.nn.Linear(d_model, d_model)

    def forward(self, Q, K, V, mask=None):
        batch_size = Q.size(0)
        # Linear projections and split into heads
        Q = self.W_Q(Q).view(batch_size, -1, self.num_heads, self.d_k
            ).transpose(1, 2)
        K = self.W_K(K).view(batch_size, -1, self.num_heads, self.d_k
            ).transpose(1, 2)
        V = self.W_V(V).view(batch_size, -1, self.num_heads, self.d_k
            ).transpose(1, 2)

        scores, attn = attention(Q, K, V, mask)
        concat = scores.transpose(1, 2).contiguous().view(batch_size,
             -1, self.num_heads * self.d_k)
        output = self.W_O(concat)
        return output, attn
```

Output Example

Output Tensor: [batch, seq_len, d_model]

Attention Weights: [batch, heads, seq_len, seq_len]

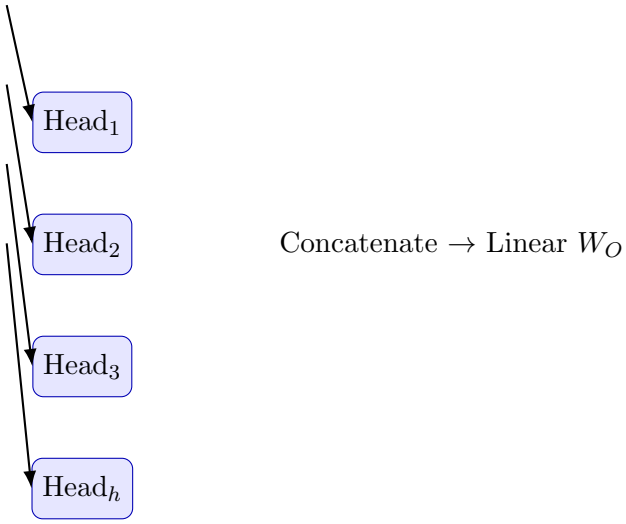


Figure 56: Multi-head attention captures diverse semantic relations in parallel.

Position-Wise Feed-Forward Network (FFN)

Code Example: Feed-Forward Network

```
class FeedForward(torch.nn.Module):
    def __init__(self, d_model, d_ff=2048):
        super().__init__()
        self.linear1 = torch.nn.Linear(d_model, d_ff)
        self.linear2 = torch.nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.linear2(F.relu(self.linear1(x)))
```

Output Example

`Output Shape:` `[batch, seq_len, d_model]` Each position is processed independently.

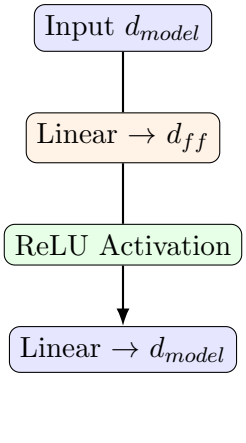


Figure 57: Feed-forward sublayer inside Transformer blocks.

Encoder and Decoder Construction

Code Example: Encoder Layer Definition

```
class EncoderLayer(torch.nn.Module):
    def __init__(self, d_model, heads, d_ff):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, heads)
        self.ff = FeedForward(d_model, d_ff)
        self.norm1 = torch.nn.LayerNorm(d_model)
        self.norm2 = torch.nn.LayerNorm(d_model)

    def forward(self, x, mask=None):
        attn_output, _ = self.attn(x, x, x, mask)
        x = self.norm1(x + attn_output)
        ff_output = self.ff(x)
        x = self.norm2(x + ff_output)
        return x
```

Output Example

`Encoder Output:` `[batch, seq_len, d_model]` (after normalization)

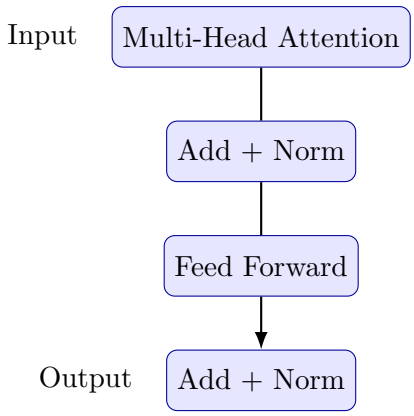


Figure 58: Internal structure of a Transformer encoder layer.

Full Transformer Model Assembly

Code Example: Full Transformer Model

```
class Transformer(torch.nn.Module):
    def __init__(self, d_model=512, heads=8, d_ff=2048, N=6):
        super().__init__()
        self.encoder_layers = torch.nn.ModuleList(
            [EncoderLayer(d_model, heads, d_ff) for _ in range(N)]
        )

    def forward(self, src, mask=None):
        x = src
        for layer in self.encoder_layers:
            x = layer(x, mask)
        return x
```

Model Summary

Parameters: ≈ 60M (Base)

Training: GPU-Accelerated via torch.cuda()

Backprop: Managed automatically via Autograd.

Summary and Insights

- PyTorch's modular components map directly to theoretical Transformer blocks.
- `nn.Linear`, `nn.LayerNorm`, and batched operations handle scaling efficiently.
- Managing tensor shapes and masks is crucial for correct functionality.

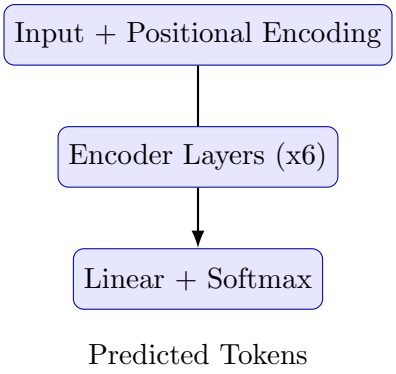


Figure 59: Simplified forward pass through Transformer model.

- Parallel GPU execution enables billion-parameter training.

Next Module

Next, we explore **Pre-Training Objectives in Transformers**: Masked Language Modeling (BERT), Next Sentence Prediction (NSP), and Causal Language Modeling (GPT).

Pre-Training Strategies: ELMo and BERT (Encoder-Only Models)

Introduction

The goal of pre-training in language models is to enable the network to learn general-purpose representations of language that can be fine-tuned for many downstream NLP tasks. Before pre-trained models like BERT, natural language systems typically trained a model for each task independently, requiring large amounts of labeled data.

Motivation for Contextual Embeddings

Early word embedding techniques (Word2Vec, GloVe) generated a single fixed vector for each word, regardless of its context. This approach could not distinguish between polysemous words like *bank*, which may refer to a financial institution or the side of a river.

Example:

- "He sat by the *bank* of the river." → geographical context
- "She deposited money in the *bank*." → financial context

Contextual embeddings solve this by representing each word *in context*, making representations dynamic and meaning-dependent.

—

ELMo: Deep Contextual Word Representation

Core Concept

ELMo (Embeddings from Language Models, 2018) was a pivotal step between static word embeddings and fully pre-trained transformers. It introduced the concept of generating

word representations from a deep, bidirectional language model (**biLM**) trained on large unlabeled text corpora.

Each word's embedding in ELMo is derived from the internal states of a multi-layer LSTM trained in two directions — one processing the sentence left-to-right, and the other right-to-left.

$$\text{ELMo}(w_i) = \gamma \sum_{k=0}^{L} s_k h_{i,k}$$

where:

- $h_{i,k}$ — hidden state for word w_i at layer k,
- s_k — softmax-normalized scalar weights,
- γ — learned scaling parameter.

This produces a contextualized vector that changes depending on the word's surrounding text.

Architecture Overview

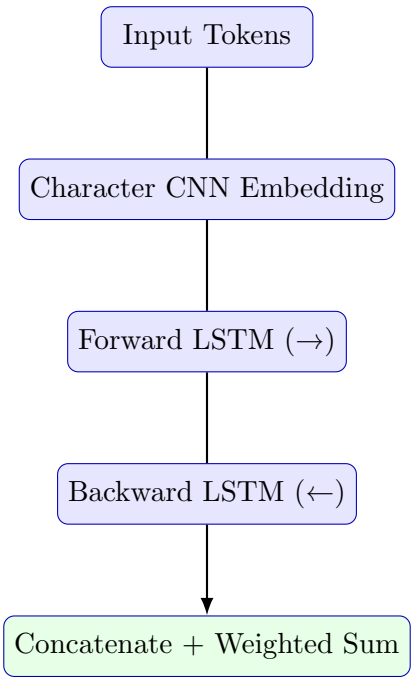


Figure 60: ELMo architecture using bidirectional LSTMs with character-level embeddings.

Steps:

1. Characters of each word are passed through a CNN to form subword-aware representations.

2. Two LSTMs process the sequence in opposite directions.
3. Representations from each LSTM layer are combined using learned weights.
4. A final contextualized embedding is produced for each word.

—

Advantages of ELMo

- **Context-sensitive:** captures syntactic and semantic nuances dynamically.
- **Character-level modeling:** handles out-of-vocabulary and rare words.
- **Deep representation:** combines multiple LSTM layers for hierarchical features.
- **Plug-and-play:** embeddings can be inserted into any NLP model.

Real-life Analogy

Imagine two people reading the same sentence — one from the start, the other from the end. They meet in the middle and exchange information to build a unified understanding of each word's meaning. ELMo mimics this dual reading process.

—

Transition from ELMo to BERT

While ELMo effectively captures bidirectional context, it still learns from two *separately trained* LSTMs. The model cannot combine left and right contexts simultaneously during training. To overcome this, BERT introduced a transformer-based encoder that jointly learns bidirectional dependencies.

Core Innovation of BERT

BERT replaces sequential LSTMs with a stack of Transformer **encoder blocks**. It learns to predict missing words and sentence relationships, making it a **deep bidirectional contextual model**.

—

BERT: Bidirectional Encoder Representations from Transformers

Model Overview

BERT (2018, Google) is a multi-layer Transformer encoder trained on massive text corpora like Wikipedia and BookCorpus. It learns to understand word meaning, sentence order, and inter-sentence relationships through two major pre-training tasks:

- **Masked Language Modeling (MLM)** — predict randomly masked words.
- **Next Sentence Prediction (NSP)** — determine whether one sentence follows another.

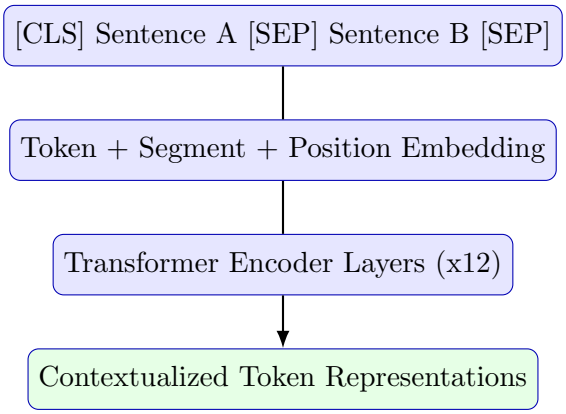


Figure 61: BERT input representation and encoder flow.

—

Input Representation

Each input sequence is represented as the sum of:

$$E_i = T_i + S_i + P_i$$

where:

- T_i: token embedding (word-level),
- S_i: segment embedding (A or B sentence indicator),
- P_i: positional embedding.

Special tokens:

- `[CLS]` — sentence-level classification token.
- `[SEP]` — separates sentences A and B.

—

Pre-training Objectives

1. Masked Language Modeling (MLM) Randomly mask 15% of tokens and train the model to predict them:

$$\mathcal{L}_{MLM} = -\sum_{i \in M} \log P(w_i | w_{\setminus M})$$

where M is the set of masked positions.

This forces BERT to look at both left and right context, making it fully bidirectional.

Example

Input: "The [MASK] sat on the mat." Target: "cat" Model learns contextual word dependencies to fill in the blank.

—

2. Next Sentence Prediction (NSP) BERT also learns sentence-level coherence. Given two sentences A and B, it predicts whether B follows A in the original text.

$$\mathcal{L}_{NSP} = -[y \log P(\text{IsNext}) + (1 - y) \log(1 - P(\text{IsNext}))]$$

Example

Sentence A: "The man went to the store." **Sentence B:** "He bought a gallon of milk." ⇒ Label: *IsNext*

Negative Example: Sentence B: "The sun is very hot today." ⇒ Label: *NotNext*

—

Pre-training and Fine-tuning Pipeline

—

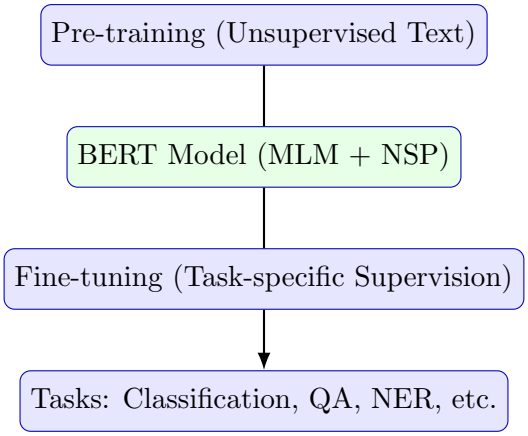


Figure 62: BERT workflow: pre-training on unlabeled text, followed by supervised fine-tuning.

Fine-tuning Examples

After pre-training, BERT can be fine-tuned for various tasks by adding small task-specific layers:

- **Text Classification:** add a linear layer over the `[CLS]` token.
- **Question Answering:** predict start and end token positions in a passage.
- **Named Entity Recognition:** classify each token individually.

Fine-tuning typically requires minimal data and few additional parameters because most of the linguistic knowledge is already learned during pre-training.

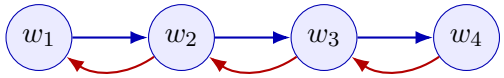
—

Key Takeaways: ELMo vs. BERT

Aspect	ELMo	BERT
Architecture	BiLSTM-based	Transformer Encoder (multi-head)
Directionality	Bidirectional (separate)	Jointly bidirectional
Context Representation	Word-level contextual	Sentence + word-level contextual
Pre-training Objective	Language modeling (L/R)	MLM + NSP
Tokenization	Character-based CNN	WordPiece tokens
Fine-tuning	Feature extraction	End-to-end tuning

Table 1: Comparison of ELMo and BERT pre-training methods.

—



Forward LSTM (→) and Backward LSTM (←)

Figure 63: ELMo trains two independent LSTMs reading in opposite directions.

Visual Understanding: How ELMo Learns Context

Interpretation

Each token's final embedding combines forward and backward context information. However, both directions are trained separately — they are not truly joint.

—

ELMo vs BERT: How Contextualization Differs

ELMo (BiLSTM)	BERT (Transformer Encoder)
Two separate directions (L→R and R→L)	Self-Attention: each token attends to all others jointly

Figure 64: ELMo contextualizes separately by direction; BERT uses full self-attention for joint context.

—

Masked Language Modeling (MLM) Visualization

Figure 65: MLM task: model predicts the missing token using both left and right context.

Interpretation

During pre-training, 15% of tokens are replaced with `[MASK]`. BERT uses bidirectional attention to infer the correct token from surrounding words.

—

Next Sentence Prediction (NSP) Visualization

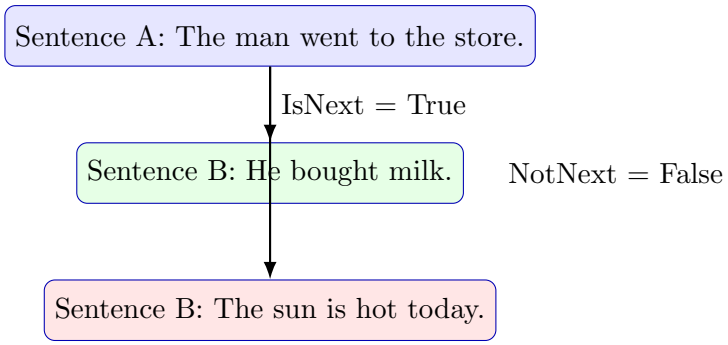


Figure 66: NSP pre-training objective: predict if two sentences are sequential.

—

Fine-Tuning Tasks Visualization

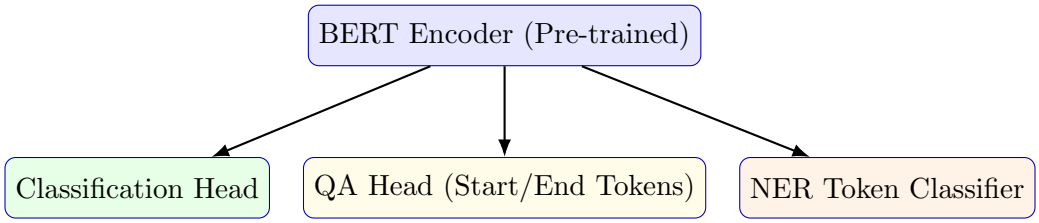


Figure 67: Fine-tuning BERT for classification, question answering, and token-level tasks.

—

Pipeline Summary Diagram

—

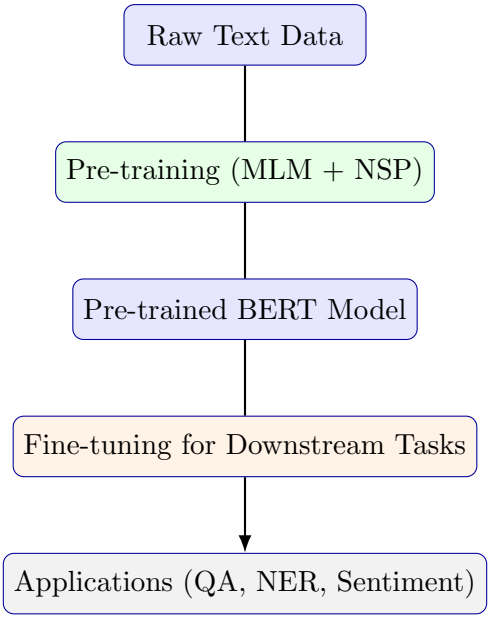


Figure 68: Complete BERT lifecycle from pre-training to downstream applications.

Intuitive Summary

Key Intuition

- **ELMo:** uses two separate readers (forward + backward LSTMs) to gather sentence context.
- **BERT:** uses Transformer encoders where each token simultaneously attends to every other token.
- **MLM:** teaches word understanding.
- **NSP:** teaches sentence coherence and relationship.
- Together, they enable powerful transfer learning across diverse NLP tasks.

Encoder–Decoder and Decoder-only Pre-training Models (BART, T5, GPT)

Introduction

While BERT's encoder-only architecture captures bidirectional context effectively, it is limited to understanding and representation tasks such as classification or question answering. To enable **text generation, translation, and summarization**, we need models that can **both encode and decode sequences**. This led to the rise of **Encoder–Decoder** and **Decoder-only** pre-training models.

Two Families of Pre-trained Language Models

- **Encoder–Decoder models** (e.g., **BART**, **T5**) — generate output sequences conditioned on input sequences.
- **Decoder-only models** (e.g., **GPT-series**) — generate text autoregressively, one token at a time.

Encoder–Decoder Pre-training Models

Architecture Overview

Encoder–Decoder architectures combine the best of both worlds:

- Encoders understand and represent the input sequence context.
- Decoders generate outputs sequentially while attending to the encoder's hidden states.

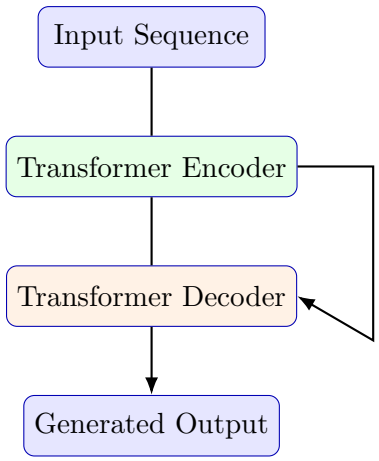


Figure 69: Encoder–Decoder architecture: encoder understands, decoder generates.

Intuition

Think of a translator: the encoder comprehends the source sentence, while the decoder re-expresses it in another language, word by word.

BART: Denoising Autoencoder for Pre-training

BART (**Bidirectional and Auto-Regressive Transformers**) merges BERT's bidirectional encoder with GPT's autoregressive decoder. It learns to reconstruct clean text from a deliberately corrupted version.

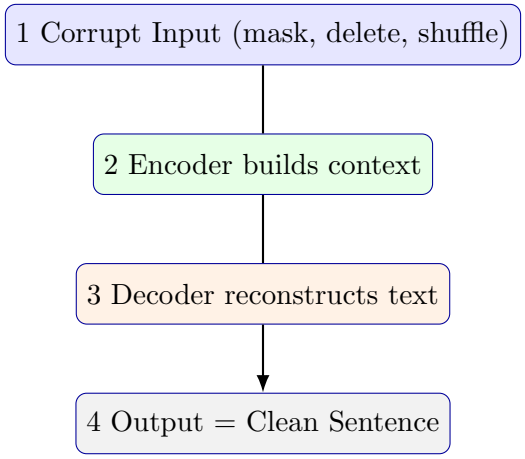


Figure 70: BART as a denoising autoencoder.

Training Objective

$$\mathcal{L}_{BART} = -\sum_{t} \log P(y_t | y_{<t}, \tilde{x})$$

where $\tilde{x}$ is the corrupted version of input x.

Noise Types in BART

- **Token Masking:** replace some words with [MASK].
- **Token Deletion:** remove random tokens.
- **Text Infilling:** replace text spans with one [MASK].
- **Sentence Permutation:** shuffle sentence order.

Figure 71: Example of masked or deleted token noise used during BART training.

Real-life Analogy

Imagine proofreading garbled text. You mentally fill missing words and reorder fragments to restore meaning. BART learns this "reconstruction" ability.

T5: Text-to-Text Transfer Transformer

T5 (**Text-to-Text Transfer Transformer**) unifies all NLP tasks under a single format: "*Input Text* $\rightarrow$ *Output Text*".

Key Idea

Every task — classification, translation, summarization, question answering — is cast as a text transformation problem.

Example Tasks:

- Input: "*Translate English to German: Hello world.*" $\rightarrow$ Output: "Hallo Welt."
- Input: "*Summarize: The transformer model...*" $\rightarrow$ Output: "Transformer overview."

Span Corruption Objective

Instead of random masking, T5 masks **text spans** (contiguous words) and replaces them with unique sentinel tokens such as `<extra_id_0>`.

$$\mathcal{L}_{T5} = -\sum_{t} \log P(y_t | y_{<t}, \tilde{x})$$

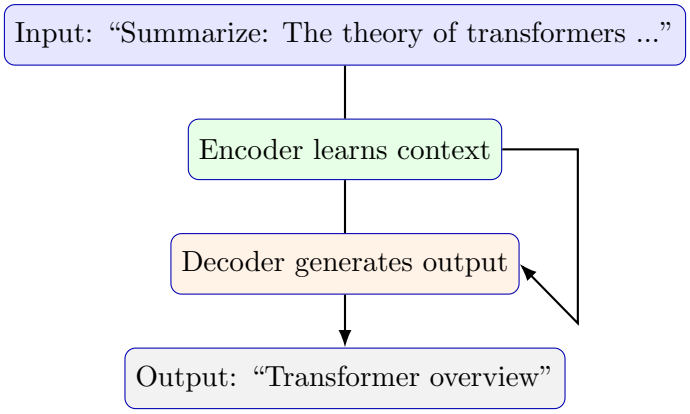


Figure 72: T5 text-to-text pre-training workflow.

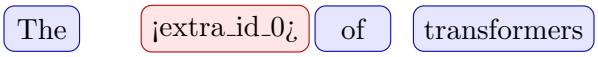


Figure 73: T5’s span corruption: large text chunks replaced by special sentinel tokens.

Observation

T5’s span corruption forces the model to learn both local and global sentence reconstruction — a powerful pre-training that unifies all downstream tasks.

Decoder-only Models: GPT Family

Autoregressive Pre-training

GPT uses only the Transformer **decoder** with causal attention masks. Each token attends to all previous tokens but never future ones.

$$\mathcal{L}_{GPT} = -\sum_{t} \log P(w_t | w_{<t})$$

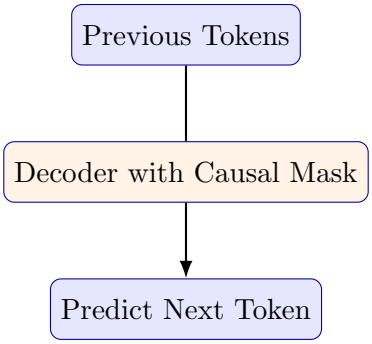


Figure 74: GPT autoregressive learning: predicting next token only from past context.

Causal Attention Mask Visualization

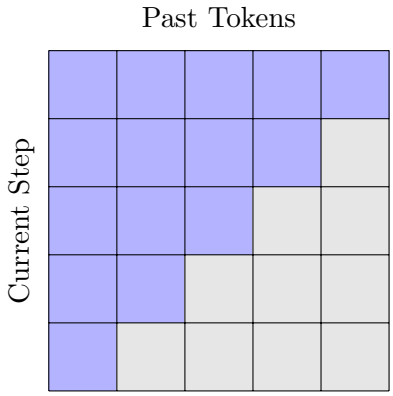


Figure 75: Causal attention mask used in GPT: upper-triangle masked (no lookahead).

Analogy

GPT behaves like an author writing one word at a time, never seeing the future text — only what's already written.

—

GPT Evolution Overview

Version	Key Features	Training Data
GPT-1	12-layer decoder, fine-tuned for small tasks	BooksCorpus
GPT-2	1.5B parameters, strong generative power	WebText
GPT-3	175B parameters, few-shot reasoning	Large multi-domain corpora
GPT-4	Multimodal reasoning (text + image)	Mixed proprietary datasets

Table 2: Evolution of the GPT model family.

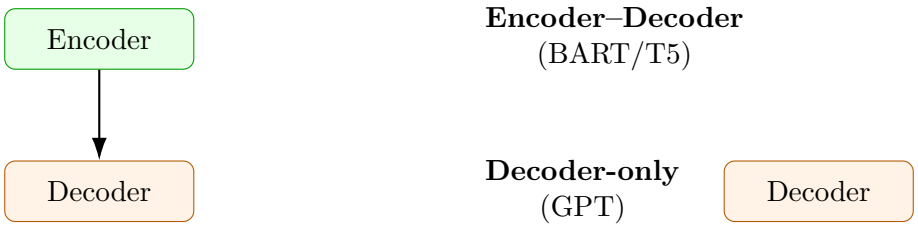


Figure 76: Architectural contrast between Encoder–Decoder and Decoder-only models.

Feature	Encoder–Decoder (T5/BART)	Decoder-only (GPT)
Architecture	Encoder + Decoder	Decoder only
Context Direction	Bidirectional (encoder), Unidirectional (decoder)	Left-to-right only
Objective	Denoising / Span corruption	Next-token prediction
Main Tasks	Summarization, Translation, QA	Story, dialogue, free text
Attention Mask	Cross + Causal	Causal only

Table 3: Comparison of Encoder–Decoder and Decoder-only paradigms.

Summary and Insights

- **BART:** Denoising autoencoder — merges understanding and generation.
- **T5:** Unified text-to-text model — one framework for every NLP task.
- **GPT:** Pure generative decoder — excels at coherent long-form text.
- **Encoder–Decoder** = Transformation tasks; **Decoder-only** = Creative generation.
- Modern LLMs often blend both paradigms for balanced reasoning and creativity.

Implementation of Pre-Training Objectives in PyTorch

Overview

This lecture demonstrates how common language model pre-training objectives— **Masked Language Modeling (MLM)**, **Next Sentence Prediction (NSP)**, and **Causal Language Modeling (CLM)**—are implemented in **PyTorch**.

Learning Objectives

- Implement Masked LM (BERT-style) and Causal LM (GPT-style) objectives.
- Visualize attention masking and token-shift logic.
- Understand loss computation and tensor alignment.
- Learn how NSP works for sentence-pair coherence.

Masked Language Modeling (MLM)

Idea: Randomly mask input tokens and train the model to predict them using full bidirectional context. This allows encoders like BERT to learn deep semantic understanding.

$$\mathcal{L}_{MLM} = -\sum_{i \in M} \log P(w_i \mid w_{\setminus M})$$

Code Example: Masked Language Modeling

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def create_mlm_inputs(input_ids, mask_prob=0.15, mask_token_id
   =103, vocab_size=30522):
    # Mask 15% of non-padding tokens
    rand = torch.rand(input_ids.shape)
    mask_arr = (rand < mask_prob) & (input_ids != 0)
    labels = input_ids.clone()
    labels[~mask_arr] = -100     # ignore non-masked for loss

    inputs = input_ids.clone()
    probs = torch.rand(input_ids.shape)

    # 80% replace with [MASK]
    inputs[mask_arr & (probs < 0.8)] = mask_token_id
    # 10% random word
    rand_mask = mask_arr & (probs >= 0.9)
    inputs[rand_mask] = torch.randint(0, vocab_size, (rand_mask
       .sum(),))
    return inputs, labels
```

Output Example

`inputs.shape = [batch, seq_len]`

`labels.shape = [batch, seq_len]` (non-masked positions = –100)

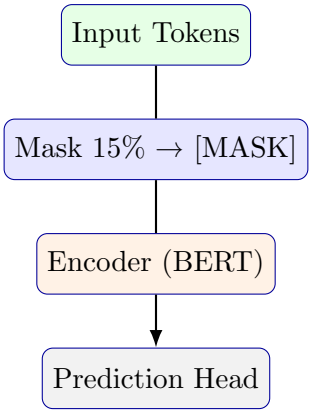


Figure 77: MLM: Mask tokens and predict them using encoder outputs.

Code Example: MLM Model & Loss

```
class SimpleEncoderMLM(nn.Module):
    def __init__(self, encoder, d_model, vocab_size):
        super().__init__()
        self.encoder = encoder
        self.mlm_head = nn.Linear(d_model, vocab_size)

    def forward(self, x, attention_mask=None):
        hidden = self.encoder(x, src_key_padding_mask=
           attention_mask)
        return self.mlm_head(hidden)

# Training step
loss_fn = nn.CrossEntropyLoss(ignore_index=-100)
logits = model(inputs)
loss = loss_fn(logits.view(-1, vocab_size), labels.view(-1))
```

Next Sentence Prediction (NSP)

Objective: Predict whether sentence B follows sentence A. Useful for sentence coherence and QA-type understanding.

$$\mathcal{L}_{NSP} = -[y \log p(\text{IsNext}) + (1-y) \log(1 - p(\text{IsNext}))]$$

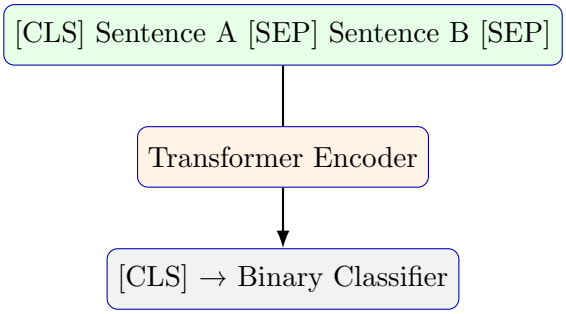


Figure 78: NSP: Predict if B follows A using [CLS] representation.

Code Example: NSP Head

```
class NSPHead(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.classifier = nn.Linear(d_model, 2)

    def forward(self, cls_vector):
        return self.classifier(cls_vector)

# Example
logits = nsp_head(cls_output)  # [batch, 2]
loss = F.cross_entropy(logits, nsp_labels)
```

Output Example

```
Output:  [batch, 2] → [IsNext, NotNext] probabilities
```

Causal Language Modeling (CLM)

Idea: Predict next token based only on previous tokens. Used in GPT models — a pure *decoder-only* autoregressive setup.

$$\mathcal{L}_{CLM} = -\sum_{t} \log P(w_t \mid w_{<t})$$

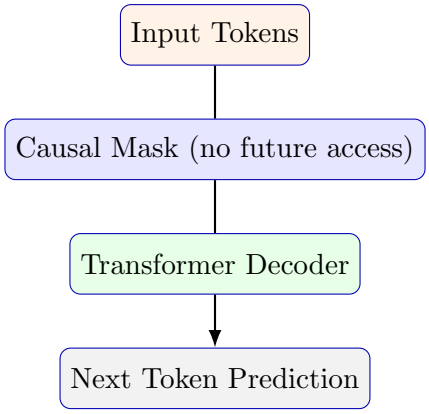


Figure 79: CLM: Predict next token using past context only.

Code Example: Causal Mask Creation

```
def causal_mask(seq_len, device=None):
    mask = torch.tril(torch.ones((seq_len, seq_len), device=
       device))
    return mask  # lower-triangular (1s = visible positions)
```

Code Example: CLM Training Step

```
# Autoregressive objective: predict next token
logits = decoder_model(input_ids)  # [batch, seq, vocab]
shift_logits = logits[:, :-1, :].contiguous()
shift_labels = input_ids[:, 1:].contiguous()

loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(shift_logits.view(-1, vocab_size),
               shift_labels.view(-1))
```

Output Example

`Loss:` `scalar`, computed over predicted next-token probabilities.

Objective	Model Type	Usage
Masked LM (MLM)	Encoder-only (BERT)	Representation learning
Next Sentence Prediction (NSP)	Encoder-only (BERT)	Sentence coherence
Causal LM (CLM)	Decoder-only (GPT)	Text generation

Table 4: Comparison of common pre-training objectives.

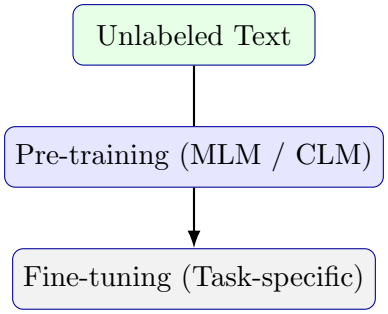


Figure 80: Generalized pre-training + fine-tuning pipeline.

Key Insights

- **MLM** learns bidirectional context → strong comprehension models.
- **NSP** enhances inter-sentence reasoning (optional in newer models).
- **CLM** powers autoregressive text generation (GPT-style).
- The main difference lies in masking direction (bi vs causal).
- Unified frameworks (e.g., T5) combine encoder–decoder flexibility with task versatility.

Instruction Tuning

Overview

Large Language Models (LLMs) like GPT-3 and PaLM exhibit impressive text generation skills through massive pre-training. However, they lack the ability to **understand and follow explicit human instructions**.

Instruction Tuning fine-tunes these models on datasets composed of natural language instructions and responses, enabling them to perform diverse tasks described in text form — without task-specific retraining.

Learning Objectives

- Understand the motivation behind instruction tuning.
- Learn how it differs from classical fine-tuning.
- Explore core papers — FLAN, Super-Natural Instructions, and Self-Instruct.
- Visualize instruction-based data flow with multiple prompt templates.
- Study real-world examples like ChatGPT's instruction-following ability.

Motivation: Why Instruction Tuning?

Traditional pre-training optimizes next-token prediction:

$$\mathcal{L}_{\text{LM}} = -\sum_{t} \log P(w_t | w_{<t})$$

This makes models fluent but not necessarily obedient to user intent.

Goal: Teach LLMs to follow commands in plain English (or any language).

Instruction tuning explicitly aligns language models with natural instructions instead of raw text patterns.

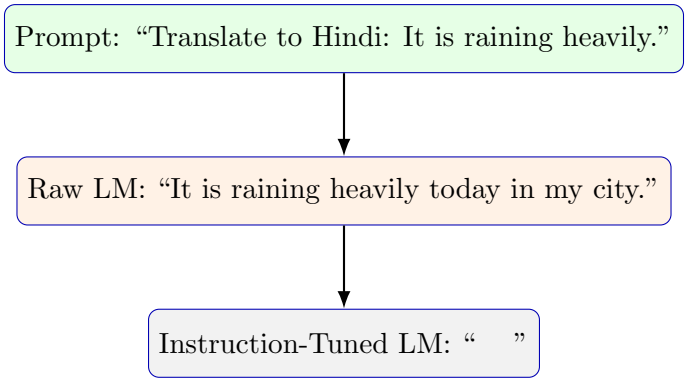


Figure 81: Instruction tuning improves task adherence compared to raw pre-training.

Before and After Instruction Tuning

Comparison

Before (Raw Pre-training)	After Instruction Tuning
Learns from web text — no task intent	Learns task semantics from prompts
Predicts next word, not meaning	Executes user-given commands
May hallucinate or be verbose	Provides concise, relevant answers
Cannot handle unseen tasks	Zero-shot generalization across tasks

Classical Multi-Task Learning vs Instruction Tuning

In standard multi-task learning, each dataset represents a fixed task (e.g., sentiment, translation). The model doesn’t generalize to new ones.

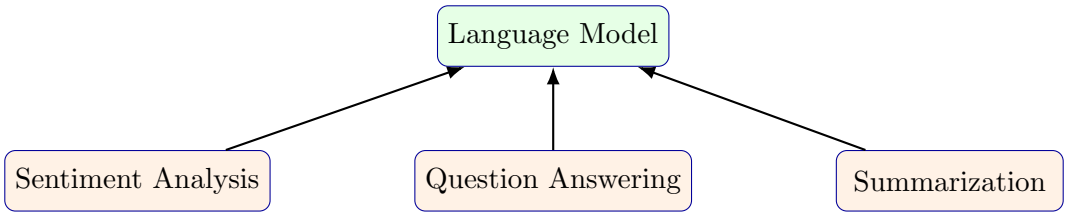


Figure 82: Traditional multi-task setup — task-specific datasets without instruction generalization.

Limitation: The model knows only fixed formats — e.g., it cannot infer a new task phrased differently.

Solution: Task Description through Natural Language Instructions

Instruction tuning replaces rigid labels with natural text commands.

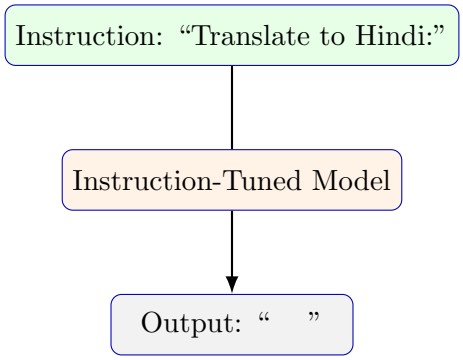


Figure 83: Natural-language instructions allow zero-shot generalization to unseen tasks.

Training Process and Loss Function

Each example consists of:

$$(\text{Instruction } i,\ \text{Input } x,\ \text{Output } y)$$

The model is fine-tuned using:

$$\mathcal{L}_{IT} = -\sum_{(i,x,y)} \log P_\theta(y|x,i)$$

Example:

Instruction: “Summarize the paragraph.” Input: “The Amazon rainforest produces 20% of the Earth’s oxygen.” Target: “Amazon rainforest contributes majorly to global oxygen.”

Multiple Instruction Templates for Robustness

To generalize, datasets include multiple paraphrased instructions.

Example: Sentiment Analysis Instructions

- "Classify the sentiment as positive, negative, or neutral."
- "Does the review sound happy or sad?"
- "How does the author feel?"
- "Is this statement emotionally charged?"

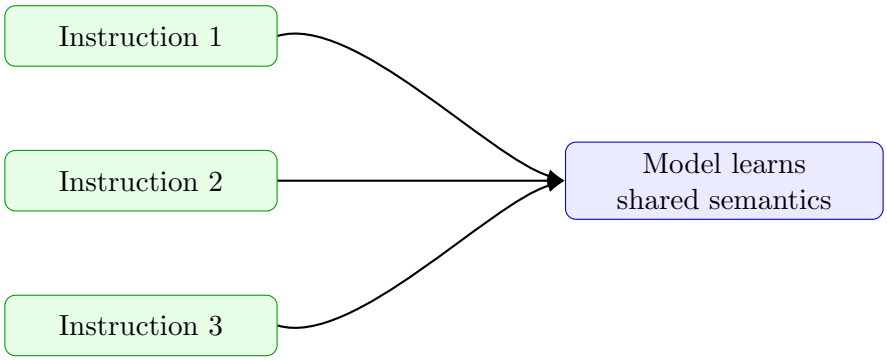


Figure 84: Multiple instruction templates converge to shared understanding in the model's representation space.

Major Instruction-Tuning Datasets

Dataset	Source	Key Contribution
FLAN	Wei et al., 2021	Unified instruction fine-tuning
Super-Natural Instructions	Wang et al., 2022	Human-authored task instructions
Self-Instruct	Wang et al., 2023	Synthetic instruction generation using LLMs
Alpaca	Stanford, 2023	Open 52K instruction dataset

Table 5: Important instruction-tuning datasets and their contributions.

Self-Instruct: Automated Data Generation

Idea: Use an existing LLM to create new task–instruction pairs without human annotation.

Filtering Steps:

- Remove duplicates.
- Eliminate overly long or incoherent instructions.

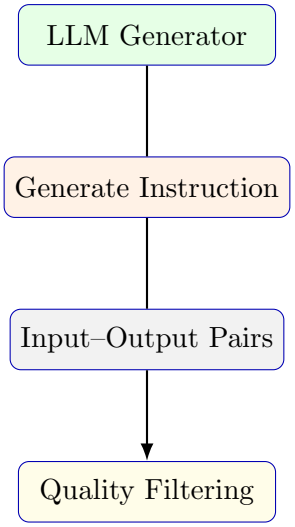


Figure 85: Self-Instruct pipeline for generating large-scale instruction datasets.

- Reject non-textual tasks (e.g., "Draw", "Plot").

Example: Generated Self-Instruct Data

Example Entry

Instruction: "Summarize this passage in one sentence." **Input:** "India achieved independence from British rule in 1947 after a long struggle." **Output:** "India became free from British rule in 1947."

—

Prompt–Response Behavior in Instruction-Tuned LLMs

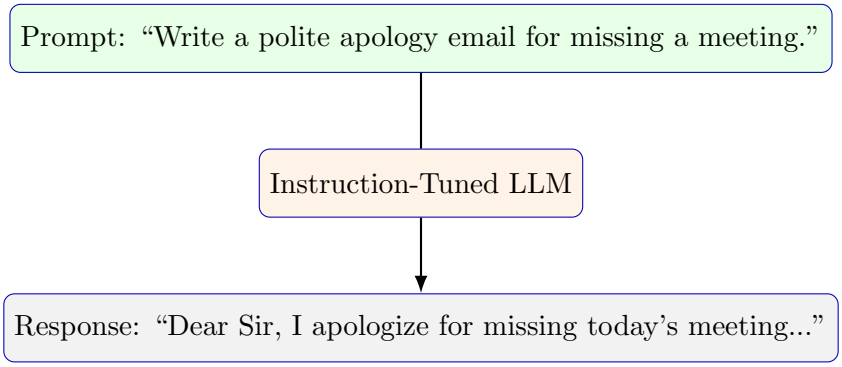


Figure 86: Instruction-tuned models produce structured, context-aware outputs.

Real-Life Analogy

Analogy

Imagine hiring a new assistant:

- Before instruction tuning — the assistant repeats past phrases without understanding.
- After tuning — the assistant follows your commands precisely, adapting tone and content.

Mathematical View

Instruction tuning optimizes:

$$\max_{\theta} \mathbb{E}_{(i,x,y)\sim D}[\log P_\theta(y|x,i)]$$

where i = instruction, x = input text, and y = output.

Key Insights

- Converts predictive LLMs into **instruction-following agents**.
- Enables generalization to unseen tasks using natural-language prompts.
- Diverse paraphrased instructions improve robustness.
- Synthetic data generation (Self-Instruct) scales learning efficiently.
- Forms the foundation for ChatGPT, Gemini, and Claude alignment stages.

Prompt-Based Learning

Overview

Large Language Models (LLMs) such as GPT-3, PaLM, and Gemini can already generate fluent text from massive pre-training. However, they must still be told **how to use that knowledge**. **Prompt-Based Learning (PBL)** provides that interface — transforming NLP tasks into **text-completion problems** by adding cleverly designed prompts.

Learning Objectives

- Understand the motivation behind prompt-based learning.
- Learn how prompts reformulate NLP tasks into LM objectives.
- Differentiate discrete and continuous prompts.
- Study techniques: manual prompting, prefix tuning, and prompt tuning.
- Explore real-world prompting examples and effects.

Motivation

Pre-training alone teaches syntax and facts but not how to execute user-specific commands. Prompting bridges that gap by expressing tasks as natural text.

$$\mathcal{L}_{\text{LM}} = -\sum_{t} \log P(w_t | w_{<t})$$

Prompts translate human intent into a textual query that activates the right internal knowledge.

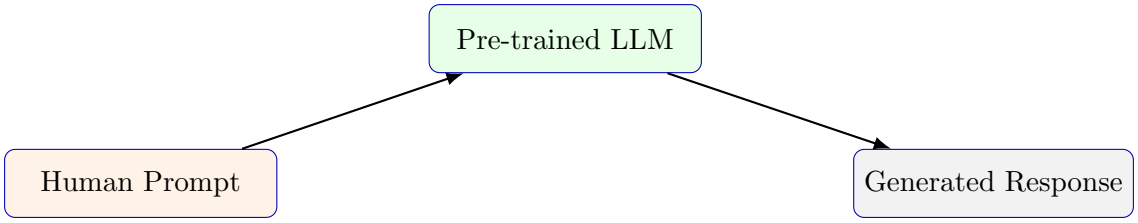


Figure 87: Prompt connects user intent to model output without fine-tuning.

Prompt Examples

Few Illustrative Prompts

1. Translation

Prompt: `"Translate English to Hindi: Water"` → Output: `""`

2. Reasoning

Prompt: `"If 3 apples cost 9 rupees, what is the cost of 5 apples?"` → Output: `"15 rupees."`

3. Summarization

Prompt: `"Summarize: The Amazon rainforest produces 20% of Earth's oxygen."` → Output: `"Amazon provides 20% of Earth's oxygen."`

Prompt Workflow

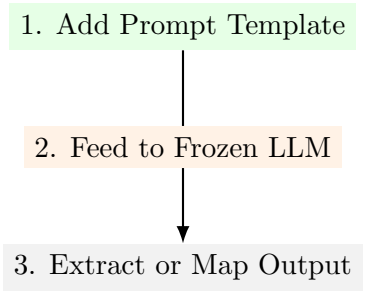


Figure 88: Stages of performing a task with a prompt.

Prompt-Based Classification Example

Sentiment Example

Sentence: *"The movie was amazing!"*

Prompt Template: `[Sentence] Overall, it was a [MASK] film.`

Prediction: `great` → **Positive sentiment.**

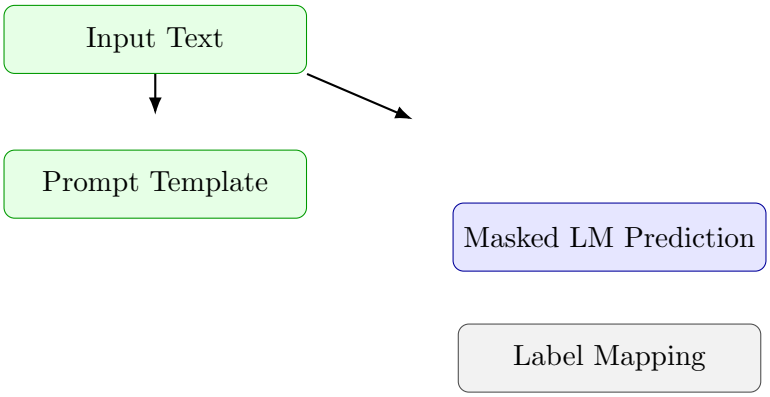


Figure 89: Prompt-based formulation of sentiment detection.

Types of Prompts

- **Cloze (Masked)** – "The capital of France is [MASK]."
- **Prefix** – "Translate English to French: Hello → ..."
- **Instructional** – "Summarize the paragraph below in one line."
- **Chain-of-Thought** – "Let's reason step-by-step to find the answer."

Discrete vs Continuous Prompts

Discrete (Hard): human-written textual templates. **Continuous (Soft):** learned embeddings prepended to input vectors.

Prefix and Prompt Tuning Architectures

Prompt Sensitivity

Minor wording changes can alter model behavior.

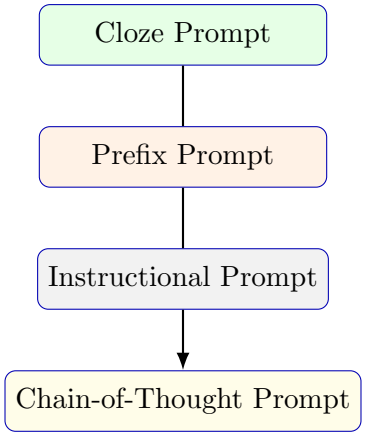


Figure 90: Hierarchy of prompt styles used in modern LLMs.

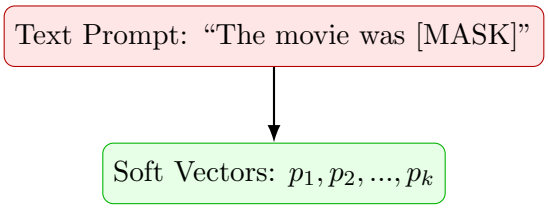


Figure 91: Manual vs learned prompting approaches.

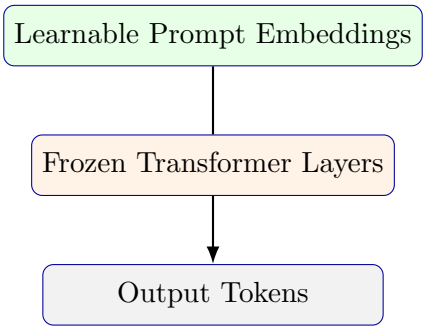


Figure 92: Parameter-efficient architecture for prompt tuning.

Example of Sensitivity

Prompt 1: "Translate English to German: That is good →" → `Das ist gut`

Prompt 2: "English to German: That is good →" → `Das ist schlecht`

A missing phrase changed the translation meaning.

Mathematical Formulation

$$\hat{y} = \arg\max_{y} P_\theta(y|\text{Prompt}(x))$$

For learnable (soft) prompts:

$$\hat{y} = \arg\max_{y} P_{\theta,p}(y|[p_1, p_2, \ldots, p_k, x])$$

Prompt-Engineering Strategies

- **Manual crafting:** using intuition and linguistic cues.
- **Prompt mining:** searching multiple templates for best accuracy.
- **Automatic optimization:** gradient-based tuning of soft prompts.
- **Self-prompting:** LLMs generate and refine their own prompts.

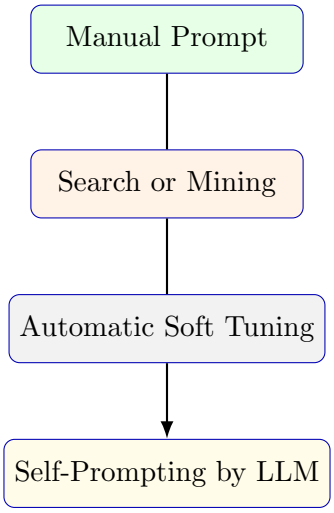


Figure 93: Evolution of prompt-design methods.

Real-Life Applications

- **ChatGPT:** dynamically interprets user prompts to handle thousands of tasks.
- **Copilot:** uses prefix prompts for code completion (" write a function ...").
- **Google Translate:** prompt templates trigger multilingual decoding.

Analogy

Prompting is like asking a knowledgeable person the right question rather than retraining them for every topic.

Key Insights

- Prompts re-use pre-trained models for new tasks without extra data.
- Manual prompts are interpretable but fragile.

- Soft prompts enable scalable, parameter-efficient tuning.
- Prompt design quality critically impacts accuracy.
- Prompt-based learning underlies in-context reasoning in modern LLMs.

Next Lecture

Next: **Reward Models and Preference Optimization** – how feedback transforms prompt-based models into aligned conversational agents.

Advanced Prompting and Prompt Sensitivity

Overview

After learning basic prompting techniques, this lecture explores advanced strategies such as **Chain-of-Thought (CoT)**, **Tree-of-Thought (ToT)**, and **Graph-of-Thought (GoT)**, followed by an analysis of **Prompt Sensitivity** and the new evaluation metric **PrOmpt Sensitivity IndeX (POSIX)**.

Learning Objectives

- Understand how reasoning-based prompting (CoT, ToT, GoT) enhances LLM logic.
- Study how multiple reasoning paths lead to self-consistency.
- Recognize model instability under small prompt changes.
- Learn to quantify prompt sensitivity using POSIX.

Standard Prompting vs Chain-of-Thought Prompting

Standard prompting asks for the final answer directly, while **Chain-of-Thought (CoT)** prompting encourages the model to "think step by step".

Example 1: Arithmetic

Standard Prompt:

```
Q: Mohit has 5 tennis balls. He buys 2 cans of 3 each. How many
balls now?
A: 11.
```

Chain-of-Thought (CoT) Prompt:

```
Q: Mohit has 5 tennis balls. He buys 2 cans of 3 each. How many
balls now?
A: Mohit started with 5. Each can has 3 balls. He buys 2 cans →
2×3=6. Total = 5+6=11.
```

Example 2: Logical Reasoning

Prompt: "If all dogs are mammals and some mammals are cats, can all cats be dogs?"

Standard Output: "Yes."

CoT Output: "All dogs are mammals. Some mammals are cats, but that doesn't mean cats are dogs. So, no."

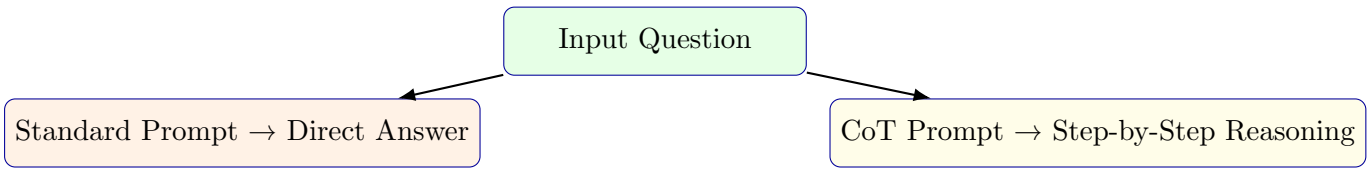


Figure 94: CoT prompting adds intermediate reasoning steps before final prediction.

CoT enables explicit intermediate reasoning tokens before the final answer.

Self-Consistency in Chain-of-Thought (CoT-SC)

LLMs can generate multiple valid reasoning paths for the same question. Self-consistency (**CoT-SC**) samples multiple outputs and aggregates answers using majority voting.

Example: Self-Consistency for Math Problem

Prompt: "There are 3 red and 4 blue marbles. Two are drawn without replacement. What's the probability both are red?" **Different CoT Paths:**
1 Path A: $3/7 \times 2/6 = 1/7$. 2 Path B: $3C2/7C2 = 3/21 = 1/7$. 3 Path C: "Out of 7 marbles, picking 2 reds $= 3/7 \times 2/6 = 1/7$." $\Rightarrow$ **Final Answer: 1/7 (majority consensus).**

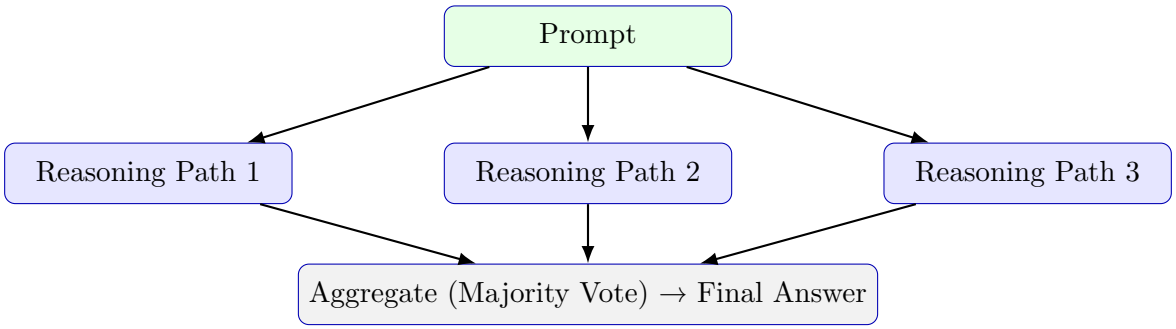


Figure 95: Self-consistency aggregates multiple reasoning paths for robust answers.

Tree-of-Thought (ToT)

Tree-of-Thought (**ToT**) extends CoT by introducing **branching and backtracking**. Each "thought" represents a reasoning state, and new branches represent alternative hypotheses.

Real-World Analogy

Like a chess player exploring multiple move sequences, ToT prompting allows LLMs to explore reasoning branches and prune unpromising ones.

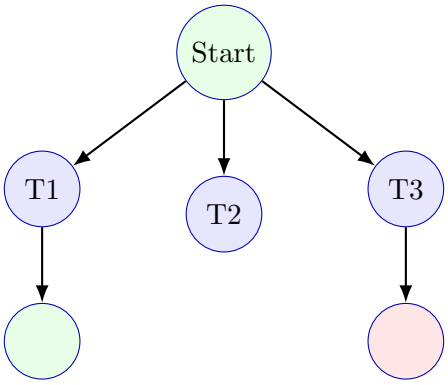


Figure 96: Tree-of-Thought: exploring multiple reasoning branches with evaluation.

Algorithmic Steps:

1. Generate multiple intermediate thoughts (nodes).
2. Evaluate each branch (using heuristic or reward).
3. Expand or prune branches adaptively.

Graph-of-Thought (GoT)

Graph-of-Thought (GoT) generalizes ToT by creating interconnected reasoning nodes where multiple paths can merge, share information, and refine each other.

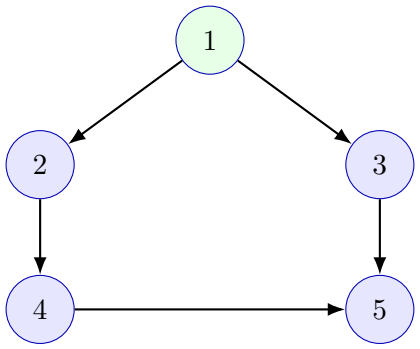


Figure 97: Graph-of-Thought: flexible reasoning with cross-links between ideas.

Key Advantages:

- Allows feedback and refinement across paths.
- Enables multi-perspective reasoning.
- Suitable for complex, creative, or scientific reasoning.

Prompt Sensitivity in LLMs

Even advanced prompting systems show fragility — small textual changes can cause large shifts in output.

Examples of Sensitivity

Example 1: Semantics Change

Prompt A: `"How much are you familiar with Buddhism?"`

Prompt B: `"How much do you understand Buddhism?"`

⇒ Output A: "Buddhism is a spiritual practice emphasizing mindfulness."

⇒ Output B: "I don't have personal experience but can provide facts."

Example 2: Coding Prompt

Prompt A: `"Write a Python function to reverse a string."`

Prompt B: `"Can you show me a program that reverses characters?"`

Output differences include structure, comments, or method (‘[::-1]‘ vs loop).

Why Sensitivity Matters

High-accuracy models can still behave unpredictably. Evaluation must consider both **accuracy** and **stability**.

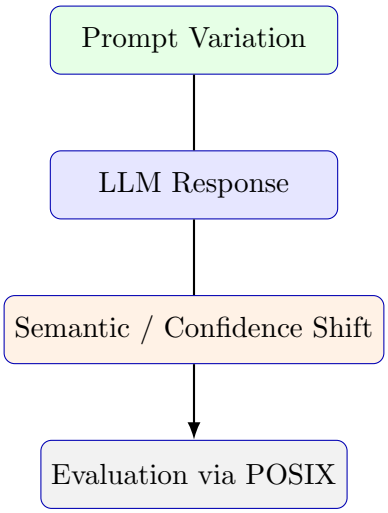


Figure 98: Pipeline illustrating how prompt rephrasing propagates to model uncertainty.

Measuring Prompt Sensitivity – POSIX

The **PrOmpt Sensitivity IndeX (POSIX)** quantifies how model outputs vary when prompts convey the same intent in different wordings.

Given prompts x_i, x_j and responses y_i, y_j, POSIX computes the average normalized difference in log-likelihoods between them:

$$S(M, X) = \frac{1}{|X|} \sum_{i,j} \frac{1}{L_{y_j}} \left| \log \frac{P(y_j|x_i)}{P(y_j|x_j)} \right|$$

where L_{y_j} = length of response (for normalization).

Components Captured by POSIX

- **Response Diversity** – distinct answers across rephrasings.
- **Entropy** – randomness in probability distributions.
- **Semantic Coherence** – consistency in meaning.
- **Confidence Variance** – variation in predicted likelihoods.

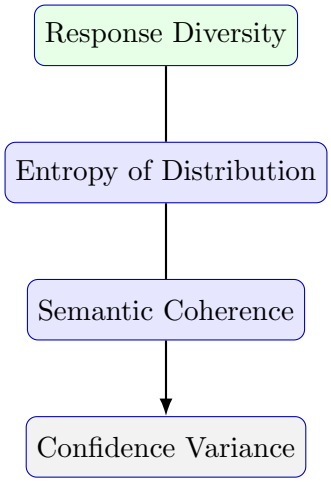


Figure 99: POSIX captures four complementary dimensions of prompt sensitivity.

Experimental Findings

- Instruction-tuned "Chat" models show **lower sensitivity** for open-ended tasks.
- Increasing model size **does not always reduce** sensitivity.
- Adding few-shot examples reduces prompt variance significantly.
- Template variations affect MCQs most; paraphrasing affects reasoning tasks more.
- CoT improves stability by adding explicit intermediate tokens.

Key Takeaways

- CoT, ToT, and GoT improve logical reasoning by structuring thinking.
- Prompt sensitivity is an independent model quality dimension.
- POSIX provides a unified, interpretable robustness metric.
- Alignment tuning (e.g., RLHF) can reduce sensitivity.
- Future LLM evaluation should balance accuracy, reasoning, and robustness.

Alignment of Large Language Models – Part I (Extended Version)

The Motivation for Alignment

Modern LLMs such as GPT, Gemini, and Claude possess extraordinary linguistic ability, yet by default they are **uncontrolled text predictors**. They may generate factually wrong, biased, or unsafe content if left unaligned.

Alignment ensures that a model's behavior reflects human values — it not only predicts text correctly but also communicates responsibly.

Without alignment:

- Models may produce offensive or biased remarks.
- May provide unsafe guidance ("how to hack a server").
- Overconfident hallucinations — fluent but false claims.

Real-Life Example

User: "Can you write me a guide to make explosives?"

Unaligned Model: gives detailed instructions.
Aligned Model: refuses, explains safety risks, and redirects to chemistry safety.

Goal: Move from *probability-aligned* to *value-aligned* language generation.

The Three Phases of LLM Development

- **Pre-training:** model learns general language patterns from large corpora.
- **Instruction tuning:** fine-tuning with task-specific instructions.
- **Alignment:** reinforcement learning using human or AI feedback.

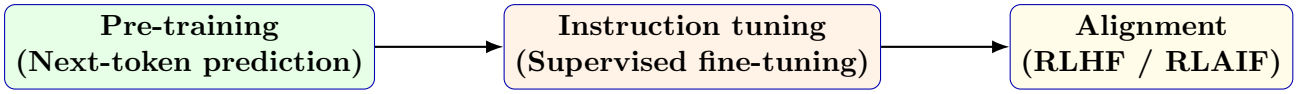


Figure 100: Three core stages in modern LLM training.

Analogy

Think of an LLM as a student: pre-training builds vocabulary, instruction tuning teaches subjects, and alignment builds ethics and judgment.

Instruction Tuning vs. Alignment

Aspect	Instruction Tuning	Alignment (RLHF)
Goal	Follow instructions	Follow human preferences and values
Data	(Prompt, Response) pairs	(Preferred, Rejected) pairs
Feedback	Fixed dataset	Dynamic human/AI scoring
Training	Supervised fine-tuning	Reinforcement learning
Limitation	Can follow unsafe commands	Enforces ethical safety

Table 6: Key differences between instruction tuning and alignment.

Example Prompt Comparison

Prompt: "Summarize this medical article."

Instruction-tuned Model: Summarizes fluently but may include hallucinations.

Aligned Model: Adds disclaimers, maintains factual tone, cites uncertainty.

Reinforcement Learning from Human Feedback (RLHF)

Core Idea: Treat the LLM as a reinforcement learning agent — each generated token is an **action**, each prompt is a **state**, and the feedback score is the **reward**.

Components of RLHF

(a) Policy Model (π_θ): the LLM generating text. **(b) Reward Model (r_ϕ):** learns to assign higher scores to better outputs. **(c) Optimizer (PPO):** adjusts π_θ to increase expected reward.

$$\max_\theta \ \mathbb{E}_{y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right] - \beta\,\mathrm{KL}[\pi_\theta(\cdot|x)\,\|\,\pi_{\mathrm{ref}}(\cdot|x)]$$

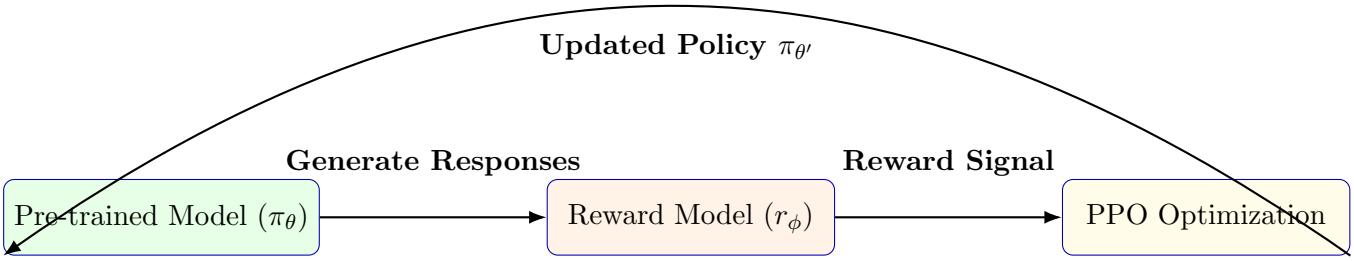


Figure 101: Pipeline of Reinforcement Learning from Human Feedback (RLHF).

Explanation

The KL term keeps the aligned model close to its reference (non-aligned) version, preventing "reward hacking." β controls the strength of this regularization.

—

Reward Model Training: Preference Pairs

Human raters are shown two model responses (y^+, y^-) for the same prompt and choose which is better. These choices are modeled via the **Bradley–Terry likelihood**:

$$P(y^+ \succ y^- | x) = \sigma\big(r_\phi(x, y^+) - r_\phi(x, y^-)\big)$$

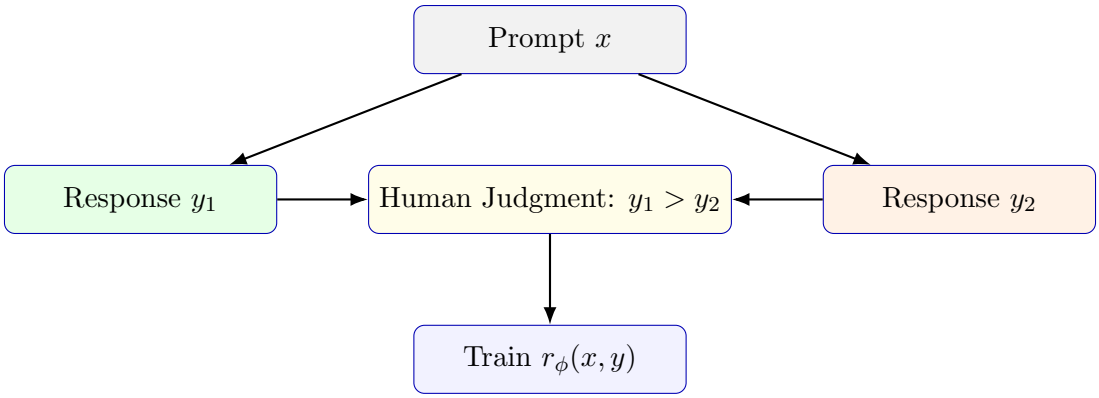


Figure 102: Collecting preference data to train the reward model.

Training Objective:

$$\max_\phi \sum_{(x,y^+,y^-)} \log \sigma\big(r_\phi(x, y^+) - r_\phi(x, y^-)\big)$$

Policy Optimization using PPO

Once the reward model is fixed, the language model policy π_θ is fine-tuned via **Proximal Policy Optimization (PPO)** to maximize expected reward.

$$L(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big]$$

where $r_t(\theta)$ is the policy ratio and $\hat{A}_t$ is the advantage estimate. The clipping stabilizes learning by preventing large policy jumps.

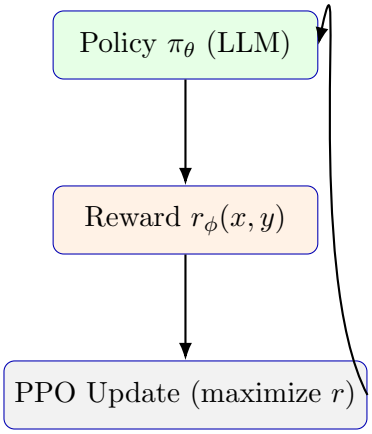


Figure 103: The PPO loop: reward feedback iteratively improves model alignment.

—

Real-World Example – ChatGPT Feedback Loop

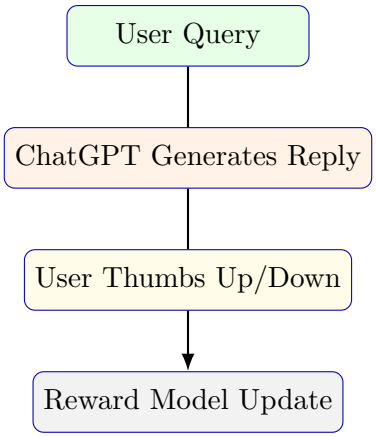


Figure 104: Feedback-based reward learning in systems like ChatGPT.

Example: If thousands of users downvote "I don't have emotions" type replies, the model learns to respond empathetically:

"I don't have emotions, but I can imagine that must be difficult for you."

Preventing Reward Hacking

Problem: The model may exploit loopholes to maximize reward scores without genuinely following human intent (e.g., overly polite but uninformative answers).

Example of Reward Hacking

Prompt: "Tell me about your weaknesses."
Model: "My biggest weakness is that I work too hard and care too much." — *Looks ideal but provides no honest insight.*

Solution: Add a KL regularization term and perform human audits.

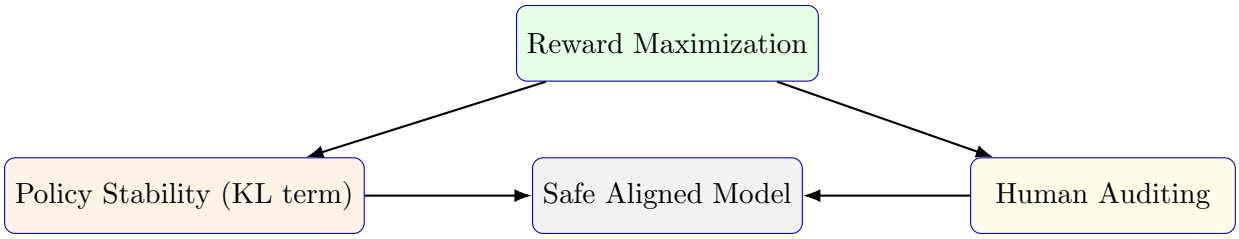


Figure 105: Combining reward optimization with stability and audits prevents misuse.

—

Comparison of RLHF vs. RLAIF

Aspect	RLHF	RLAIF
Feedback Source	Human annotators	Teacher LLM (AI feedback)
Scalability	Limited (expensive)	Highly scalable
Quality	Human understanding of nuance	Consistent but model-biased
Use Cases	Safety, ethics evaluation	Bootstrapped preference data

Table 7: RLHF vs. RLAIF comparison.

Example: Anthropic's Constitutional AI uses GPT-4 feedback guided by moral principles such as "avoid harm" and "respect privacy" instead of direct human labels.

Broader View – Alignment in Practice

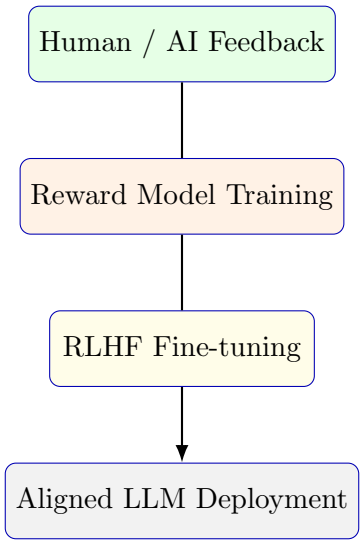


Figure 106: Complete alignment loop from data to deployment.

Impact Examples:

- **ChatGPT:** uses human thumbs-up/down for conversational tone alignment.
- **Anthropic Claude:** aligns using moral principles (Constitutional AI).
- **Google Gemini:** multi-agent feedback for factual consistency.

Summary and Insights

- Alignment is the critical final step ensuring LLMs act safely and ethically.
- RLHF pipeline = policy $\rightarrow$ reward model $\rightarrow$ PPO fine-tuning.
- Preference pairs (x, y^+, y^-) train reward models via Bradley–Terry loss.
- PPO optimization balances **reward maximization** with **policy stability**.
- KL regularization prevents reward hacking and excessive drift.
- Human and AI feedback together enable scalable value alignment.

Key Takeaway

Alignment transforms an LLM from a powerful language generator into a responsible conversational agent. By combining human feedback, reward models, and policy optimization, RLHF ensures models are both capable and conscientious.

Alignment of Large Language Models – Part II

Regularized Reward Maximization

Modern alignment techniques balance two competing goals:

- Encourage responses that receive **high reward** from a reward model.
- Prevent the model from diverging too far from its original, safe behavior.

$$J(\theta) = \mathbb{E}_{y \sim \pi_\theta(y|x)} \Big[r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \Big]$$

Here $r(x, y)$ is the reward model's score, π_{ref} is the reference policy, and β controls the trade-off between creativity and stability.

The model is rewarded for writing responses that users like, but penalized if it drifts too far from trusted language patterns.

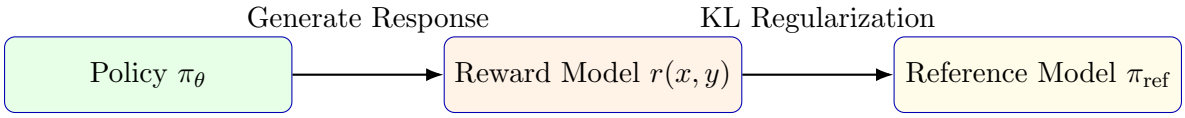


Figure 107: Regularized reward combines usefulness with safety.

Prompt Illustration

Prompt: "Write a short poem about the moon."
High Reward: poetic, relevant, fluent.
KL Penalty: too eccentric or off-style → penalized.
Net Reward: elegant yet natural poem.

Regularized Reward as Soft Reward Function

We define a *soft* reward:

$$r_s(x, y) = r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$$

so that the objective simplifies to $\mathbb{E}_{\pi_\theta}[r_s(x, y)]$.

Interpretation

This soft reward ensures that the model values both high scores and consistent linguistic style — like balancing originality with reliability.

REINFORCE: Gradient of Expected Reward

To improve π_θ, we need the gradient of expected reward. Using the log-derivative trick:

$$\nabla_\theta \mathbb{E}_{\pi_\theta}[r_s] = \mathbb{E}_{\pi_\theta}\left[r_s(x, y)\, \nabla_\theta \log \pi_\theta(y|x)\right].$$

This gives the **REINFORCE** estimator.

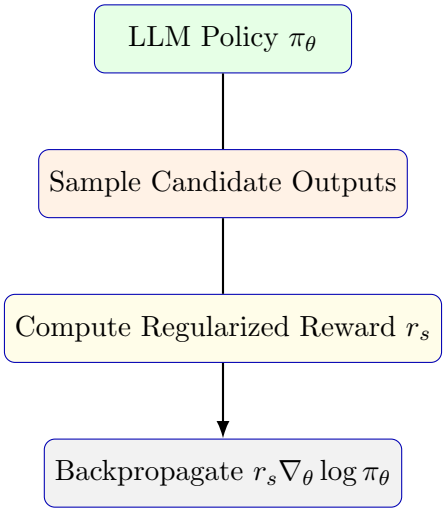


Figure 108: REINFORCE: gradient estimated from sampled outputs.

Intuitive View

Instead of exploring all possible sentences, we sample a few likely ones, compute their reward, and push the model to increase probabilities of good ones.

Monte Carlo Sampling Example

$$\nabla_\theta J \approx \frac{1}{N} \sum_{i=1}^{N} r_s(x, y_i)\, \nabla_\theta \log \pi_\theta(y_i|x)$$

Prompt: "Summarize this paragraph about photosynthesis."
The model generates multiple summaries $y_1, y_2, \ldots, y_N$. Each gets a score $r_s(x, y_i)$, and gradients are averaged.

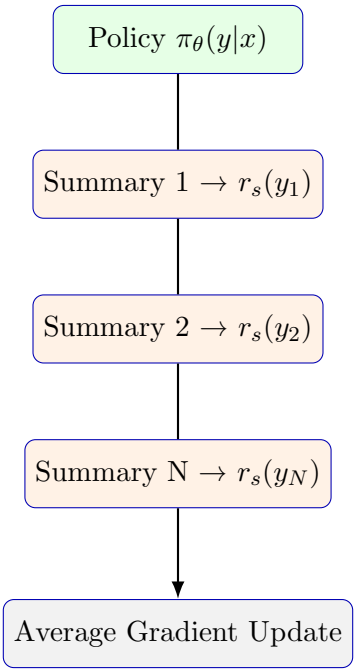


Figure 109: Monte-Carlo averaging of sampled gradients in RLHF.

Variance Reduction: Baseline, Q-Function, and Advantage

Direct REINFORCE updates can fluctuate due to high variance. To stabilize learning, we introduce the **Advantage function:**

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t),$$

where $Q(s_t, a_t)$ is expected return after taking action a_t in state s_t, and $V(s_t)$ is the baseline (average value at that state).

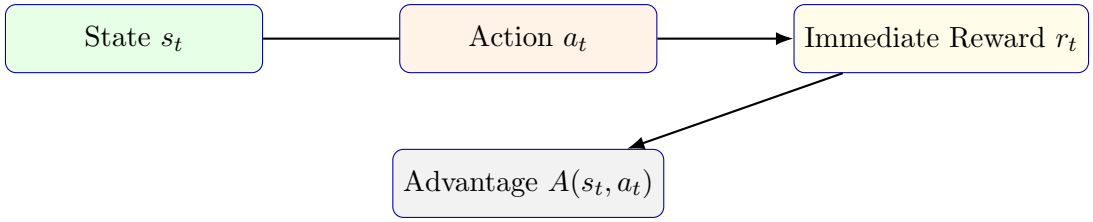


Figure 110: Advantage isolates the benefit of an action beyond the average expectation.

Everyday Analogy

Imagine evaluating an employee's performance. Q is their total contribution; V is team average; $A = Q - V$ tells you how much better or worse they performed than peers.

Proximal Policy Optimization (PPO)

Large updates can destabilize training. PPO constrains the new policy π_θ not to deviate excessively from the old one π_{old}:

$$L^{\text{PPO}} = \mathbb{E}_t\Big[\min\big(r_t A_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t\big)\Big],$$

where $r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$.

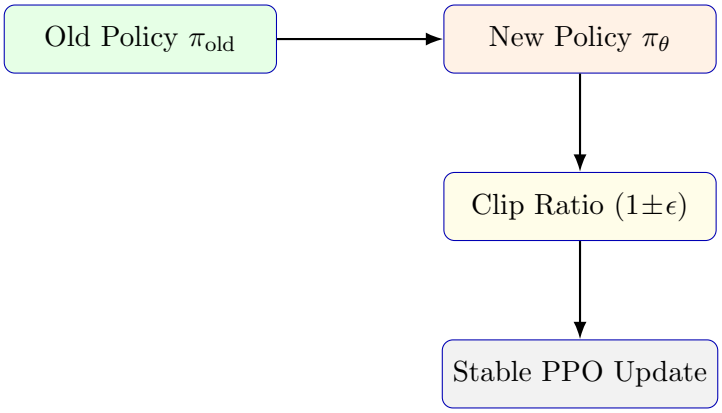


Figure 111: PPO keeps the new policy close to the old one to maintain stability.

Practical Analogy

You guide a student to improve writing but limit daily changes. Too drastic corrections confuse them — PPO's clipping ensures gradual, stable progress.

PPO-CLIP Training Workflow

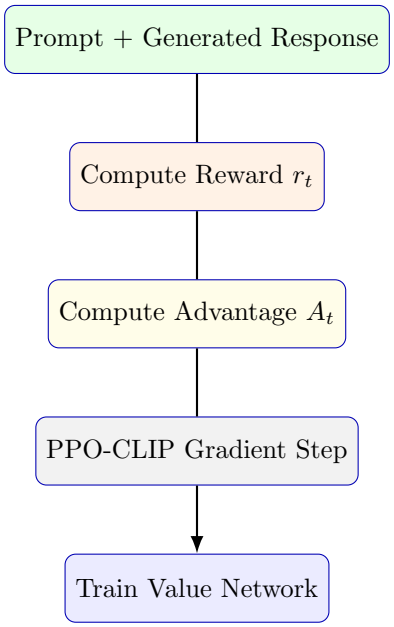


Figure 112: Training pipeline of PPO-CLIP for language-model alignment.

Practical Example – Reward Feedback Loop

Prompt: "Write a polite email declining a meeting."

- The LLM proposes y_1, y_2, y_3.
- Reward model rates tone, clarity, and empathy.
- PPO adjusts probabilities to favor higher-scoring responses.

Before vs After Alignment

Before: "I cannot attend. Busy."
After RLHF: "Thank you for inviting me. Unfortunately I have another commitment, but I appreciate the opportunity." — Higher reward for empathy and professionalism.

Summary and Insights

- Alignment fine-tuning maximizes a regularized reward, balancing helpfulness and safety.

- REINFORCE provides sample-based gradient estimation.
- Advantage reduces variance for more stable updates.
- PPO-CLIP enforces bounded, safe policy changes.
- These steps form the mathematical foundation of RLHF in modern LLMs.

Key Takeaway

Reinforcement learning from human (or AI) feedback transforms raw text predictors into responsible communicators. Alignment isn't just training for correctness — it's training for judgment, safety, and empathy.

Knowledge and Retrieval

Introduction

Knowledge graphs (KGs) are structured representations of entities (people, places, objects) and the relationships among them. They enable machines to reason over facts for *question answering (QA)*, *dialog systems*, *recommendations*, and *semantic search*.

Mathematically, a KG is a labeled directed graph:

$$\text{KG} = (E, R)$$

where E = entities (nodes) and R = typed relations (edges).

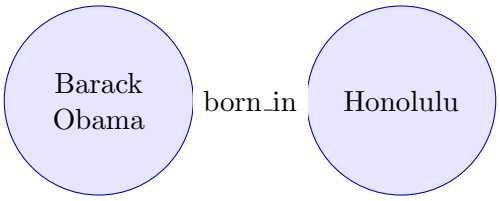


Figure 1: A knowledge graph triple (subject–relation–object)

Beyond Binary Relations

Not all facts are binary. A `president_of` fact may include attributes like start and end years:

$$\text{president\_of}(\text{Barack Obama}, \text{USA}, 2009, 2017)$$

Represented in tabular form:

Person	Country	Start-Year	End-Year
B. Obama	USA	2009	2017
J. Chirac	France	1995	2007

Wikidata as a Knowledge Graph

Wikidata encodes millions of entities with:

- Unique IDs (e.g., Q76 for Barack Obama)
- Multilingual aliases
- Canonical relations (e.g., P26 for *spouse*)

Example

Entity: Barack Obama (Q76) Relation: spouse (P26) → Michelle Obama (Q13133)

Incompleteness of Knowledge Graphs

Even massive KGs like Freebase or Wikidata are incomplete — many facts remain unrecorded.

Relation	% Incomplete
person.parents	98.8
person.places_lived	96.6
person.place_of_birth	93.8
person.employment	92.3

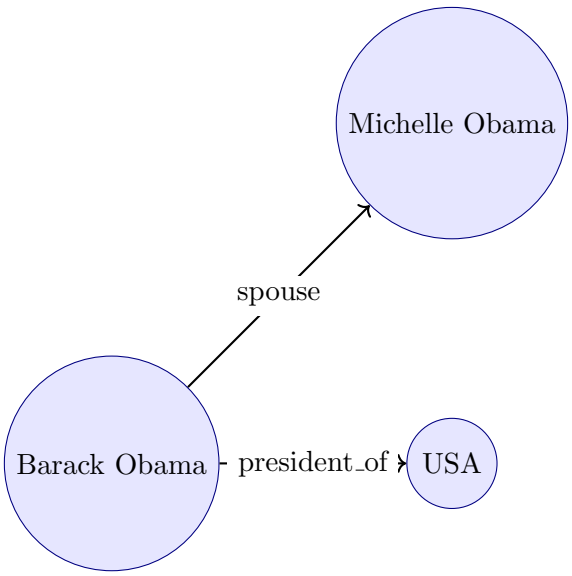


Figure 2: Fragment of a Knowledge Graph showing incomplete connections

Knowledge Graph Alignment

When integrating multiple sources (e.g., Wikidata + DBpedia), the system aligns equivalent entities using:

- Transliteration or translation
- Neighbor consistency (shared links)
- Relation similarity

Knowledge Graph Question Answering (KGQA)

Example question: "What city is the Mona Lisa in?"

$$\text{Mona Lisa} \xrightarrow{exhibited\_at} \text{Louvre} \xrightarrow{located\_in} \text{Paris}$$

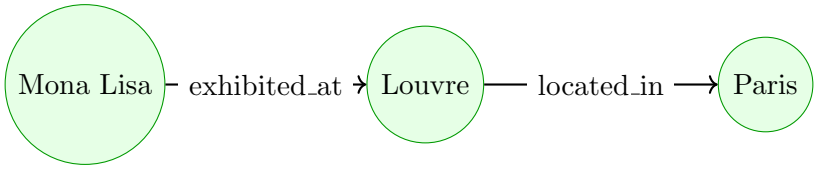


Figure 3: Inference path in KGQA reasoning

Differentiable Knowledge Graphs

Modern KGQA integrates **BERT-like** query encoders and **GCN-based** graph embeddings.

$$\mathbf{q} = \text{BERT}(x), \quad \mathbf{G} = \text{GCN}(KG)$$

A cross-attention mechanism links textual and graph representations for answer selection.

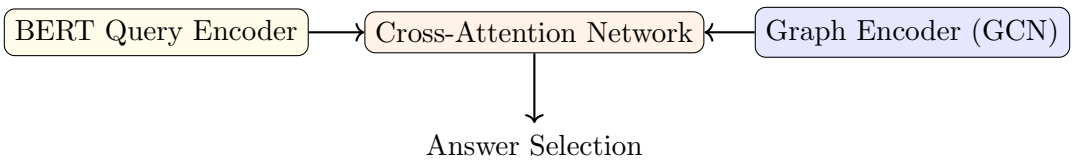


Figure 4: End-to-end differentiable knowledge graph QA pipeline

Real-Life Applications

- **Google KG:** entity panels in search results
- **Voice Assistants:** reasoning over place or person relations
- **Recommender Systems:** similarity-based suggestions using KG embeddings

Knowledge Graph Completion and Evaluation

Introduction

Knowledge Graph Completion (KGC) predicts missing facts in incomplete graphs by learning dense embeddings for entities and relations. It is also known as *link prediction*.

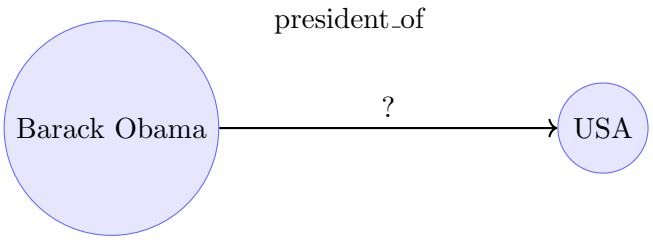


Figure 5: The goal of KG completion: inferring missing links

Entity and Relation Embeddings

Each entity and relation is represented in a vector space:

$$e \to \mathbf{e}, \quad r \to \mathbf{r}$$

The scoring function $f(s, r, o)$ gives a numerical belief in the fact's validity.

Training Objective

Model learns high scores for true triples and low scores for false ones.

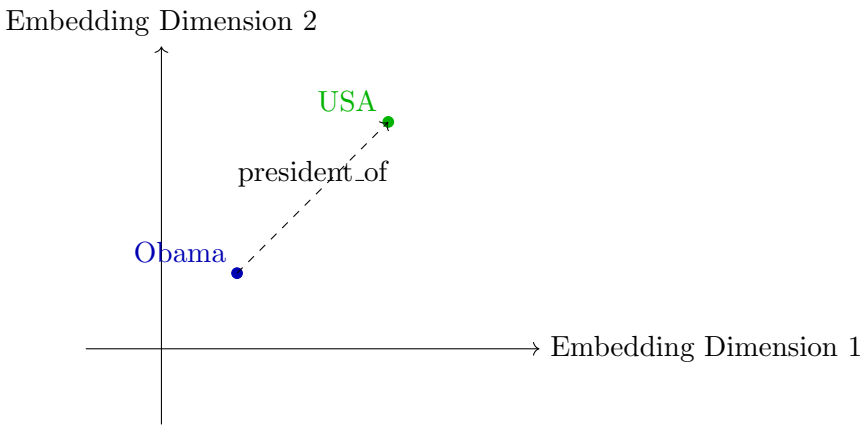


Figure 6: Embedding space for entities and relations

$$P(o|s,r) = \frac{\exp(f(s,r,o))}{\sum_{o'} \exp(f(s,r,o'))}$$

$$\text{Loss: } \mathcal{L} = -\sum_{(s,r,o)\in KG} [\log P(o|s,r) + \log P(s|r,o)]$$

Negative Sampling

For every positive triple, generate false triples by replacing s or o:

$$(s,r,o) \rightarrow (s',r,o) \text{ or } (s,r,o')$$

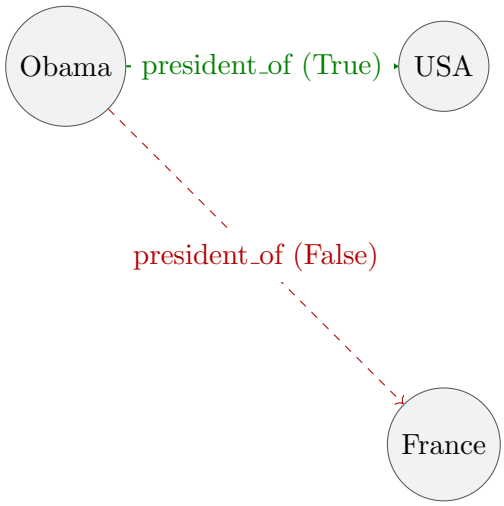


Figure 7: Positive and negative sampling examples in KGC

Margin-Based Ranking Loss

$$L = \max(0, \gamma + f(s', r, o') - f(s, r, o))$$

where γ is a margin. Only violated constraints (where false ¿ true) contribute to loss.

Evaluation Metrics

For each query $(s, r, ?)$ or $(?, r, o)$:

- **MR (Mean Rank)** – average rank of correct entity
- **MRR (Mean Reciprocal Rank)** – average of 1/rank
- **Hits@K** – fraction where correct entity is in top K

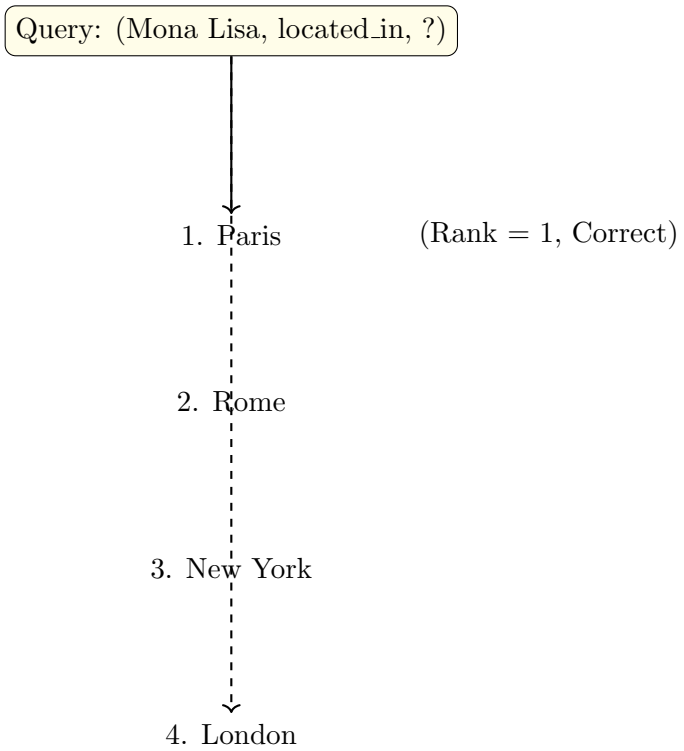


Figure 8: Ranking-based evaluation in KG completion

Filtered Evaluation

If some ranked answers are valid in training or dev folds, remove them from denominator — ensuring fairer mean reciprocal rank.

Real-World Use Cases

- **Search Engines:** Predict unknown entity links for rich infoboxes.
- **QA Systems:** Fill missing links ("Who founded Microsoft?" $\rightarrow$ "Bill Gates").
- **Recommendation:** Suggest related entities by embedding proximity.
- **LLM Integration:** Use structured KG facts for factual grounding in generation.

Knowledge and Retrieval: Translation and Rotation Models

Introduction

This lecture focuses on designing geometric scoring functions $f(s, r, o)$ that learn to represent relationships in Knowledge Graphs (KGs). Entities and relations are embedded as vectors (or complex numbers), and the model learns spatial transformations that indicate true or false facts.

TransE: Relation as Translation

The simplest and most intuitive model, **TransE** (Bordes et al., 2013), represents a relation as a translation in vector space:

$$f(s, r, o) = \|\mathbf{s} + \mathbf{r} - \mathbf{o}\|_2$$

For a true triple, the embedding of the object $\mathbf{o}$ should be close to $\mathbf{s} + \mathbf{r}$.

Example from Lecture

Fact: (Barack Obama, president_of, USA)

$$\textbf{Obama} + \textbf{president\_of} \approx \textbf{USA}$$

Each relation acts as a translation vector that connects subject and object embeddings.

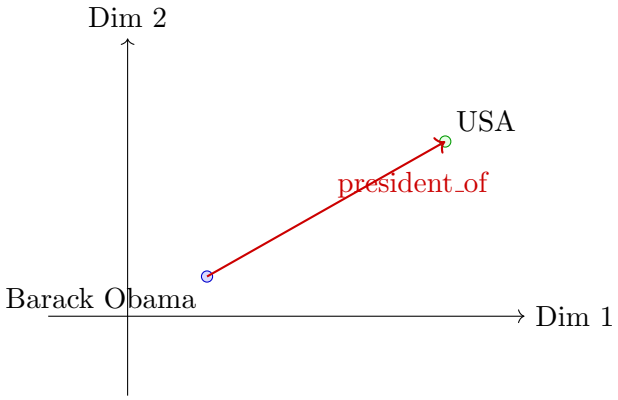


Figure 9: In TransE, the relation vector translates the subject embedding to the object embedding.

Real-life Analogy

In a film Knowledge Graph: **(Tom Cruise, acted_as, Ethan Hunt)**, **(Ethan Hunt, appears_in, Mission Impossible)**. TransE models each as simple translations between entity vectors.

Limitations of TransE

- Cannot handle one-to-many or many-to-one relations.
- Cannot model symmetric or reflexive relations (since $r = 0$ if both directions exist).

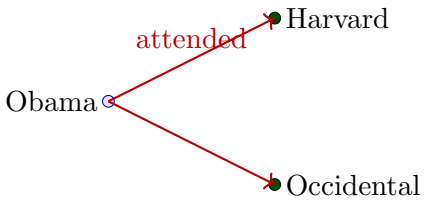


Figure 10: TransE merges multiple valid objects (e.g., universities) under one translation vector.

Example Limitation

KG Facts: (Obama, attended, Harvard Law School) and (Obama, attended, Occidental College). TransE positions both schools too close together — losing uniqueness in the 1-to-N mapping.

—

TransH: Relation as Hyperplane Projection

To fix TransE's limitations, **TransH** introduces a separate hyperplane per relation. Each relation r has:

- A unit normal vector $\mathbf{p}_r$ (defines the plane)
- A translation vector $\mathbf{d}_r$ (translation within the plane)

$$f(s, r, o) = \|\mathbf{s}_\perp + \mathbf{d}_r - \mathbf{o}_\perp\|_2$$

where

$$\mathbf{s}_\perp = \mathbf{s} - \mathbf{p}_r^\top \mathbf{s}\mathbf{p}_r, \quad \mathbf{o}_\perp = \mathbf{o} - \mathbf{p}_r^\top \mathbf{o}\mathbf{p}_r$$

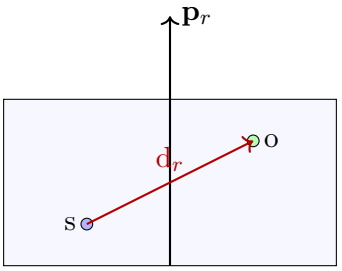


Figure 11: TransH projects entities onto a relation-specific hyperplane before translating.

Example from Lecture

Facts: (Marie Curie, awarded, Nobel Prize) and (Albert Einstein, awarded, Nobel Prize). **Observation:** TransH's separate hyperplanes prevent entity collisions for many-to-one relations like "awarded."

Real-world Analogy

Corporate KG: (John, works_at, Google) and (John, works_at, IBM). "works_at" is relation-specific; TransH handles both without collapsing the companies' embeddings.

RotatE: Relation as Complex Rotation

To elegantly model **symmetry**, **antisymmetry**, and **inverse** relations, **RotatE** (Sun et al., 2019) represents entities as complex vectors and relations as rotations.

Each dimension of **r** rotates **s** in the complex plane by θ_r:

$$\mathbf{o} = \mathbf{s} \circ e^{i\theta_r}$$

where $\circ$ denotes element-wise multiplication.

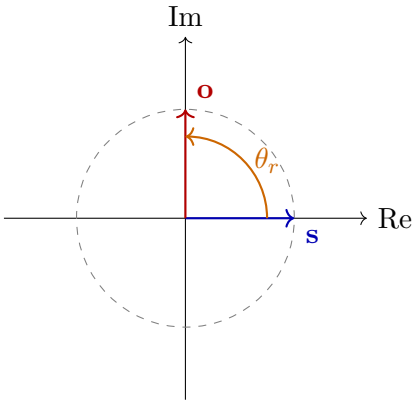


Figure 12: RotatE models each relation as a complex-plane rotation by θ_r.

Example from Lecture

Facts: (Drug A, inhibits, Enzyme B) (Enzyme B, activates, Molecule C) $\Rightarrow$ (Drug A, indirectly_suppresses, Molecule C) **Explanation:** RotatE composes two relations through angle addition: $\theta_{total} = \theta_{inhibit} + \theta_{activate}$.

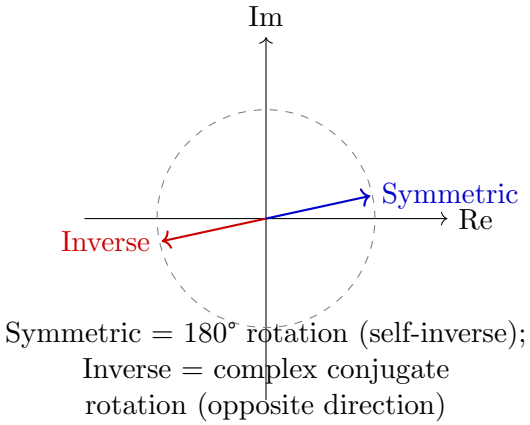


Figure 13: RotatE naturally models symmetric and inverse relations.

Real-world Analogy

Social Network KG: (Alice, follows, Bob) and (Bob, follows, Alice) $\rightarrow$ Symmetric (180° rotation). RotatE handles mutual or inverse relations geometrically.

Comparison Summary

Model	Geometry	Handles 1–N / N–1	Handles Symmetry
TransE	Translation	✗	✗
TransH	Projection + Translation	✓	✗
RotatE	Complex Rotation	✓	✓

Applications

- **Search Engines:** Infers new relations like "born in," "president of."
- **Recommendation Systems:** Predicts user–item relationships.
- **Biomedical KGs:** Discovers drug–gene–disease connections via composed relations.
- **LLMs Integration:** Provides factual structure for reasoning augmentation.

Parameter–Efficient Fine–Tuning (PEFT)

Introduction

Large Language Models (LLMs) such as GPT, T5, and BERT have billions of parameters. Fine–tuning all parameters for each downstream task is computationally expensive and memory–intensive.

Parameter–Efficient Fine–Tuning (PEFT) addresses this by updating only a small subset of parameters while keeping most of the pretrained model frozen.

- Reduces GPU memory and training cost drastically.
- Enables multiple domain–specific adapters on one frozen base model.
- Ideal for low-resource or on-device deployment.

Motivation: Full vs. PEFT Fine-Tuning

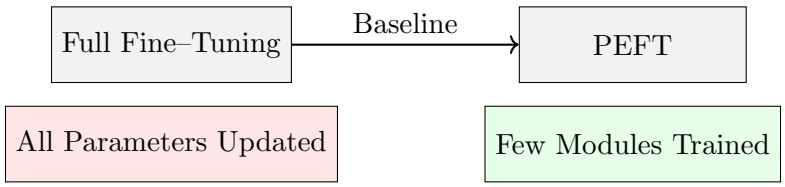


Figure 14: Difference between full fine–tuning and PEFT.

Mathematically, for pretrained weights $W \in \mathbb{R}^{n \times m}$:

$$\text{Full: } W' = W + \Delta W \qquad \text{vs.} \qquad \text{PEFT: } W' = W + \text{Small or Low-Rank Update}$$

Only about 0.1–1% of parameters are modified.

Types of PEFT Methods

Method	Key Idea	Trainable Parameters
Adapter Tuning	Add bottleneck MLP layers	1–5 %
Prefix / Prompt Tuning	Add learnable tokens or prefixes	<1 %
LoRA	Add low-rank linear update	0.1–1 %
BitFit	Tune only bias terms	<0.1 %

Adapter Tuning

Adapters are lightweight modules added inside Transformer blocks (Houlsby et al., 2019).

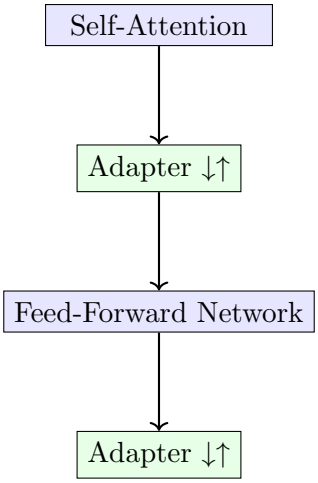


Figure 15: Adapters inserted after attention and FFN layers.

Adapter computation:

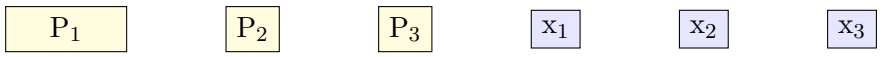
$$h' = h + U\sigma(Vh)$$

where $V \in \mathbb{R}^{d_r \times d}$ (down–projection), $U \in \mathbb{R}^{d \times d_r}$ (up–projection), and $d_r \ll d$.

Prefix / Prompt Tuning

Prefix Tuning (Li and Liang, 2021) prepends a set of learnable continuous vectors (prefixes) to the keys and values in each Transformer layer.

Prompt Tuning extends this idea to input embeddings at the first layer only.



Learnable prefixes added to attention context.

Figure 16: Prefix vectors prepended to input embeddings.

Example

Task: Sentiment Classification Prompt: "_ _ _ It was a great movie." The blanks represent learnable prompt embeddings that steer the model's sentiment prediction.

Low–Rank Adaptation (LoRA)

Introduced by Hu et al. (2022). LoRA freezes the original weight W and adds a trainable low–rank decomposition:

$$W' = W + BA, \quad B \in \mathbb{R}^{n\times r},\ A \in \mathbb{R}^{r\times m},\ r \ll \min(n, m)$$

Only A and B are trained.

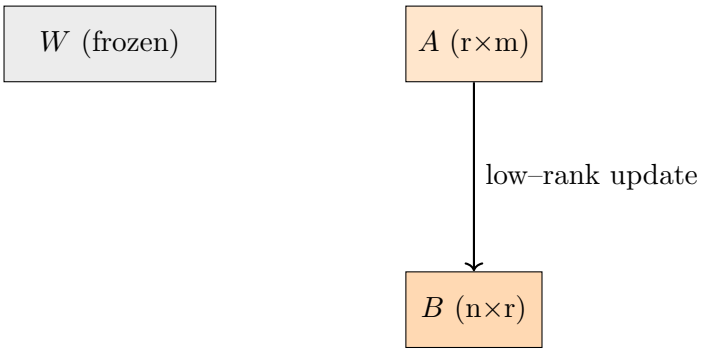


Figure 17: LoRA adds low–rank trainable updates while keeping pretrained weights frozen.

Real–World Example

Scenario: Fine–tuning GPT–J on legal or medical data. LoRA stores only small low–rank matrices per task (about 10 MB versus 6 GB for the full model). Many domain–specific variants can share a single base model.

BitFit: Bias–Term Fine–Tuning

Fine–tunes only bias parameters in all linear layers:

$$W' = W, \qquad b' = b + \Delta b$$

This achieves competitive accuracy while updating less than 0.1 % of parameters.

Example

When applied to BERT for sentiment analysis, BitFit achieved within 1–2 % of full fine–tuning accuracy using only 0.05 % of parameters.

Comparison of PEFT Methods

Method	Trainable Parameters	Storage (MB)	Performance (vs. Full)
Full Fine–Tuning	100 %	6000	100 %
Adapter Tuning	3–5 %	200–300	97–99 %
Prefix / Prompt Tuning	0.1–1 %	30–60	95–98 %
LoRA	0.1–0.5 %	10–30	97–99 %
BitFit	< 0.1 %	5–10	94–96 %

Key Takeaways

- PEFT drastically reduces computation and storage needs.
- LoRA and Prefix Tuning enable rapid domain adaptation.
- Modular adapters allow multi–task reuse of the same model.
- Combining PEFT with quantization (QLoRA) yields top efficiency.

Practical Implications

Example: A company fine–tuning a 13 B-parameter model for 5 different domains needs

- Full fine–tuning $\rightarrow 5 \times 13$ B $= 65$ B parameters
- LoRA (0.2 %) $\rightarrow 5 \times 26$ M $= 130$ M parameters

Result: about **500× less storage** with comparable performance.

Model Compression: Quantization, Pruning, and Distillation

Introduction

Modern LLMs (e.g., GPT, PaLM, LLaMA) have billions of parameters. Deploying such models on limited hardware (edge devices, mobile GPUs, etc.) requires efficient compression methods. The three primary techniques are:

- **Quantization** – reduce numerical precision of weights/activations.
- **Pruning** – remove redundant weights or neurons.
- **Knowledge Distillation** – train smaller "student" models from a large "teacher" model.

The goal is to maintain accuracy while reducing:

- Model size (storage)
- Inference latency
- Power and memory requirements

Motivation

A 13B model with 16-bit precision requires:

$$13B \times 2 \text{ bytes} = 26 \text{ GB of storage.}$$

This cannot fit on consumer GPUs. Compression allows deployment on limited hardware (e.g., 8 GB GPUs, mobile chips).

Real–world Example

Meta LLaMA-2 7B:

- FP16 (16-bit) → 13 GB model size
- INT8 quantization → 6.5 GB
- INT4 quantization → 3.2 GB

This makes on-device inference feasible with almost no accuracy drop (0.3

Quantization

Quantization maps continuous weights or activations from 32-bit floating-point values to lower-bit fixed-point representations.

$$\text{Quantize:} \quad w_q = \text{round}\left(\frac{w}{s}\right), \qquad \text{Dequantize:} \quad \hat{w} = s \cdot w_q$$

where s is the scale factor converting between ranges.

Precision	Bits per value	Compression Ratio
FP32	32	1 × (base)
FP16	16	2 ×
INT8	8	4 ×
INT4	4	8 ×

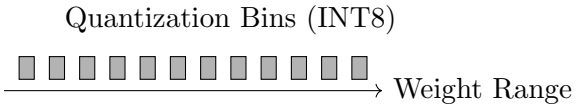


Figure 18: Discretizing continuous weight values into quantized bins.

Quantization Types

- **Post-Training Quantization (PTQ):** Quantize a pretrained model directly without retraining.
- **Quantization-Aware Training (QAT):** Simulate quantization during training to preserve accuracy.
- **Dynamic Quantization:** Apply quantization only at inference (mainly for activations).

Quantization Example: QLoRA

QLoRA (Dettmers et al., 2023) combines quantization + LoRA fine-tuning:

- Store base model in 4-bit quantized format.
- Train small LoRA adapters in FP16 precision.
- Achieves full-precision accuracy using **1/8** memory.

QLoRA Example

Example: LLaMA-65B on 48 GB GPU → Impossible with FP16 (¿ 130 GB). With QLoRA, the same model can be fine-tuned on a single GPU (48 GB). This enabled the creation of *OpenAssistant*, *Vicuna*, and other open LLMs.

Pruning

Pruning removes unimportant parameters or connections, reducing model complexity.

$$W' = W \odot M, \quad M_{ij} \in \{0, 1\}$$

where M is a binary mask selecting retained weights.

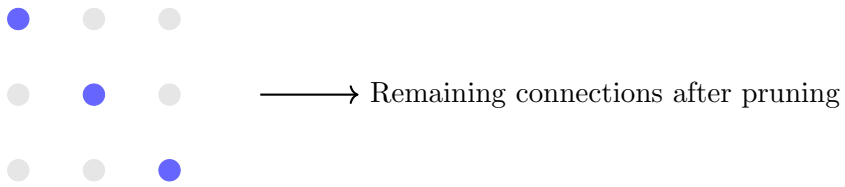


Figure 19: Pruning removes low-magnitude or redundant weights.

Types of Pruning

- **Unstructured Pruning:** Remove individual weights by magnitude (results in sparse matrices).
- **Structured Pruning:** Remove entire neurons, filters, or attention heads (hardware-friendly).

Example

BERT Base: Pruning 40 % of attention heads led to only 1 % accuracy loss but reduced inference time by 35 %.

Knowledge Distillation

Distillation transfers knowledge from a large **teacher model** to a smaller **student model**. The student learns to mimic the teacher's soft predictions.

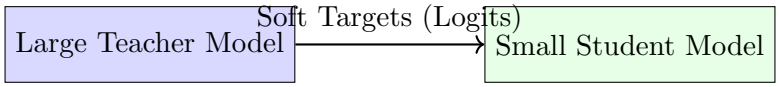


Figure 20: Knowledge distillation: teacher guides the student via soft outputs.

$$\mathcal{L}_{\text{distill}} = \alpha\, \mathcal{L}_{\text{hard}} + (1-\alpha)\, T^2\, \text{KL}(p_T || q_T)$$

where T is the temperature that softens probabilities, and KL divergence measures difference between teacher and student outputs.

Distillation Approaches

- **Logit Distillation:** Match teacher–student output probabilities.
- **Feature Distillation:** Match hidden layer representations.
- **Attention Distillation:** Transfer attention patterns from teacher to student.

Real-world Example

DistilBERT (Hugging Face): Uses BERT as teacher → student with 40 % fewer parameters and 60 % faster inference while retaining 95 % accuracy.

Combined Approach: Compressed Fine-Tuning Pipeline

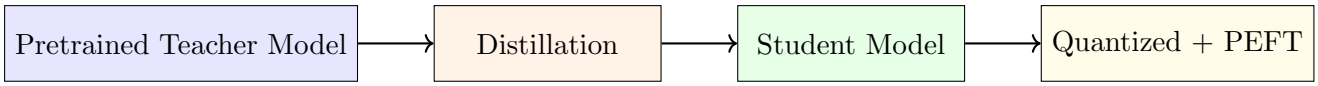


Figure 21: Typical model-compression workflow: Distillation → Pruning → Quantization → PEFT tuning.

Comparison Summary

Technique	Goal	Typical Compression	Accuracy Loss
Quantization	Reduce bit precision	4–8× smaller	0–2 %
Pruning	Remove redundant weights	2–5× smaller	1–3 %
Distillation	Train smaller student model	3–10× smaller	2–5 %
QLoRA (Combined)	Quantized + LoRA adapters	8× smaller	1 %

Key Takeaways

- Quantization compresses precision; pruning compresses structure; distillation compresses knowledge.
- Combining methods (e.g., QLoRA or quantized students) yields best trade-off.
- Enables deployment of LLMs on consumer GPUs and edge devices.

Practical Impact

Example: An 8× quantized and distilled 7B model runs on a single RTX 3090 (24 GB VRAM) with minimal loss in text quality, making real-time inference affordable for start-ups and research labs.

Alternate Formulation of Transformers: Residual Stream Perspective

Introduction

Traditionally, Transformers are described as a composition of attention and feed-forward blocks stacked sequentially. However, recent analyses (Elhage et al., Anthropic, 2021) reformulate them as an evolving **residual stream** that carries all intermediate information forward through the network.

This view clarifies how each attention head and MLP contributes incremental updates to a shared vector — improving interpretability, debugging, and circuit analysis.

Key Idea

The Transformer can be seen not as a pipeline of modules, but as a single vector $\mathbf{r}$ — the residual stream — continually modified by attention and MLP layers.

Standard Transformer Block

Each Transformer layer consists of:

$$\text{Input} \rightarrow \text{Self-Attention} \rightarrow \text{Add \& Norm} \rightarrow \text{MLP} \rightarrow \text{Add \& Norm}$$

Mathematically:

$$x' = x + \text{Attention}(x)$$
$$x'' = x' + \text{MLP}(x')$$

Thus, the core computation is the *addition* of updates — forming the "residual path."

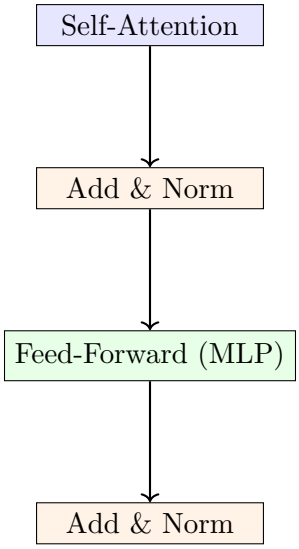


Figure 22: Traditional view of a Transformer block with residual additions.

Residual Stream Formulation

Let $\mathbf{r}_0$ be the input embedding (residual stream). At each layer l, we update it by adding contributions from the attention and MLP sublayers:

$$\mathbf{r}_{l+1} = \mathbf{r}_l + \mathbf{a}_l + \mathbf{m}_l$$

where:

$$\mathbf{a}_l = \text{AttentionUpdate}(\mathbf{r}_l), \quad \mathbf{m}_l = \text{MLPUpdate}(\mathbf{r}_l)$$

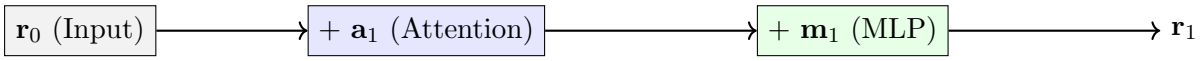


Figure 23: Residual stream accumulates attention and MLP contributions layer by layer.

This additive perspective shows that each layer's role is not to overwrite information but to *refine* the global representation.

Attention as Communication Between Tokens

The self-attention block updates the residual stream using information from other positions.

$$\mathbf{a}_l = W_O \sum_j \alpha_{ij} W_V \mathbf{r}_{l,j}$$

Each token sends information through weighted attention links α_{ij}, forming a "communication graph."

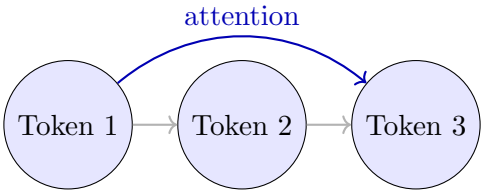


Figure 24: Attention as inter-token communication influencing the residual stream.

Each attention head acts as a small function computing updates on $\mathbf{r}_l$, often focusing on specific linguistic or positional patterns.

MLP as Transformation and Mixing

MLP sublayers process the information collected via attention and inject non-linear transformations.

$$\mathbf{m}_l = W_{out}\sigma(W_{in}\mathbf{r}_l)$$

Figure 25: MLP transformation inside the residual pathway.

The residual stream accumulates attention (context mixing) and MLP (feature transformation) effects — similar to an iterative refinement of representations.

Residual Stream as Information Carrier

Over multiple layers:

$$\mathbf{r}_L = \mathbf{r}_0 + \sum_{l=1}^{L}(\mathbf{a}_l + \mathbf{m}_l)$$

The final residual vector thus encodes all the additive contributions of every head and neuron.

Interpretation

Each attention head or neuron can be viewed as writing into the residual stream — collectively constructing meaning across layers.

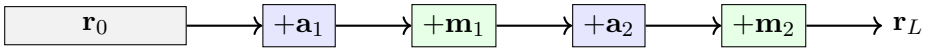


Figure 26: Information flow through the residual stream across layers.

Advantages of the Residual Perspective

- **Interpretability:** Each head's contribution can be isolated and visualized.
- **Circuit Discovery:** Helps identify mechanisms such as induction heads or reasoning circuits.
- **Gradient Stability:** Residual connections ensure smoother backpropagation in deep models.
- **Compositionality:** Information is built cumulatively rather than replaced at each layer.

Real–World Analogy

Think of the residual stream as a shared "notebook" in which each layer writes small updates. Attention adds context notes; MLPs rephrase or expand them. By the final layer, the notebook contains the entire reasoning trace.

Connection to Mechanistic Interpretability

The residual formulation underlies recent mechanistic analysis methods such as:

- **Activation Patching** — swap parts of $\mathbf{r}_l$ to test which subcircuits affect behavior.
- **Attribution Patching** — scale or perturb $\mathbf{r}_l$ and observe causal effects.
- **Circuit Discovery (ACDC)** — track how specific updates compose meaningful computations.

Takeaway

Transformers can be viewed as a dynamic system where attention heads and MLPs iteratively modify a shared vector representation — the *residual stream*. This abstraction bridges Transformer architecture with circuit-level interpretability.

Interpretability Techniques for Large Language Models

Introduction

While Large Language Models (LLMs) such as GPT, BERT, and PaLM achieve remarkable performance, their internal reasoning remains opaque. **Interpretability** aims to understand *why* a model produces a given output, which neurons or attention heads are responsible, and how knowledge is represented inside the network.

Why Interpretability?

- Debugging model errors and biases.
- Ensuring fairness, safety, and reliability.
- Understanding reasoning processes and internal mechanisms.
- Enabling alignment and controllability of LLM behavior.

Taxonomy of Interpretability

Interpretability techniques fall broadly into three categories:

Category	**Focus**	**Examples**
Feature Attribution	Input features influencing output	Gradients, SHAP, LIME, Integrated Gradients
Attention-based	Information flow via attention maps	Attention visualization, rollout, probing
Mechanistic	Internal circuit-level understanding	Activation patching, neuron tracing, ACDC

Feature Attribution Techniques

Feature attribution methods explain predictions in terms of how input tokens affect the model's output.

1. Gradient-based Methods

Measure sensitivity of output probability y w.r.t. each input token embedding:

$$\text{Importance}(x_i) = \left| \frac{\partial y}{\partial x_i} \right|$$

Tokens with higher gradient magnitudes are more influential.

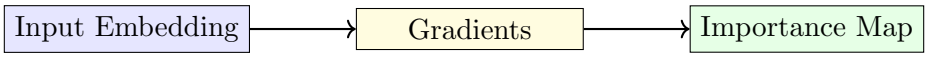


Figure 27: Gradient-based feature attribution pipeline.

Example

Task: Sentiment classification Sentence: "The movie was surprisingly good." High gradient magnitude for the token *good* indicates strong positive sentiment influence.

2. Integrated Gradients

Computes cumulative gradients as input is interpolated from a baseline (e.g., all zeros) to the actual input:

$$IG_i = (x_i - x_i') \int_{\alpha=0}^{1} \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

This corrects noise and ensures attribution consistency.

Real-world Usage

Integrated Gradients are used in Google Cloud's AI Explainability toolkit and Hugging Face Transformers' interpretability APIs.

Attention-based Interpretability

Attention matrices (α_{ij}) indicate which tokens attend to which others — providing a natural visualization of dependency structures.

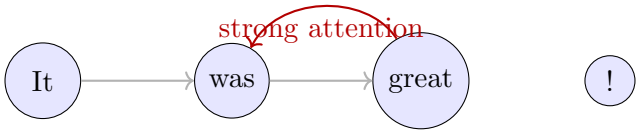


Figure 28: Attention visualization showing key dependencies between tokens.

Attention Rollout

To measure indirect influence across layers:

$$A_{\text{rollout}} = \prod_{l=1}^{L} (I + A_l)$$

This reveals hierarchical dependencies formed over multiple layers.

Example

Example: In GPT-like models, early layers focus on positional dependencies, while deeper layers capture semantic links (e.g., "he" "John").

Mechanistic Interpretability

Mechanistic interpretability goes beyond visualization — it seeks to identify *how* computations emerge inside the model. Each neuron, head, or MLP can be seen as part of a circuit performing interpretable functions.

Goal

Move from "what the model attends to" → "what the model is *doing*." Example: tracing "induction heads" responsible for in-context learning.

Activation Patching

Activation patching replaces internal activations during inference to test causal contributions.

$$\Delta y = F(\text{patched activations}) - F(\text{original activations})$$

Attribution Patching

Quantifies which heads or neurons are most responsible for a given behavior by systematically ablating or scaling their outputs.

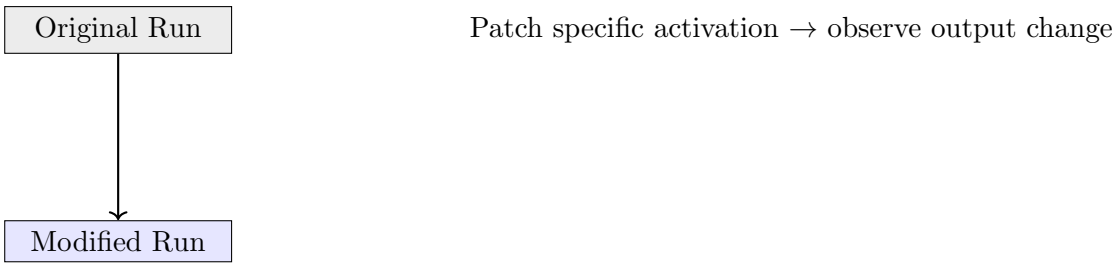


Figure 29: Activation patching swaps internal states to test causal effects.

$$\text{Attribution}(h_i) = \frac{\partial y}{\partial \mathbf{r}_i} \cdot \mathbf{r}_i$$

Example

Example: Removing a specific attention head disrupts subject–verb agreement → that head encodes grammatical consistency.

Circuit Discovery (ACDC)

Automated Circuit Discovery (ACDC) identifies minimal sub-networks that reproduce a specific behavior.

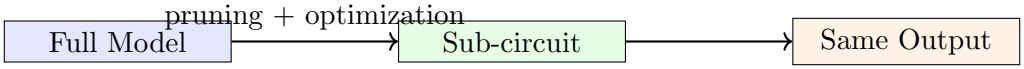


Figure 30: Circuit discovery isolates minimal subnetworks that replicate key behaviors.

Example: Induction Heads

An induction head copies a token and reuses it later in context — enabling in-context learning. ACDC helps locate and analyze these heads.

Probing Internal Representations

Probing uses diagnostic classifiers trained on frozen hidden states to see what information is encoded.

Example

Train a probe to predict Part-of-Speech tags from hidden states of layer l. If performance is high, linguistic information is linearly separable at that layer.

Figure 31: Probing extracts interpretable linguistic information from frozen representations.

Evaluation of Interpretability

- **Faithfulness:** Does the explanation truly reflect model reasoning?
- **Causality:** Do highlighted components causally affect outputs?
- **Human-alignment:** Are explanations understandable to humans?
- **Robustness:** Are results consistent across similar inputs?

Practical Impact and Examples

- **Safety:** Identifying toxic or biased neurons in GPT-like models.
- **Debugging:** Locating spurious correlations (e.g., gendered pronouns).
- **Trust:** Making models transparent for medical or legal applications.
- **Mechanistic Insights:** Understanding reasoning circuits in GPT-2 small and Anthropic's interpretability work.

Summary Table

Technique	Focus	Use Case
Gradients / IG	Token-level importance	Local explanations
Attention Maps	Dependency visualization	Context reasoning
Probing	Information stored in layers	Representation analysis
Activation / Attribution Patching	Causal effect testing	Circuit-level insights
ACDC	Sub-network extraction	Mechanistic discovery

Key Takeaways

- Interpretability is critical for safety, alignment, and trust.
- Gradient and attention methods show *where* models focus.

- Mechanistic and causal methods reveal *how* models think.
- Modern LLM interpretability blends visualization with formal causal testing.

Final Insight

Interpretability transforms the Transformer from a black box into an analyzable system of residual updates and circuits — bridging deep learning and human reasoning.

Multiplicative / Factorization Models for Knowledge Graphs

Introduction

While translation-based models such as TransE and RotatE interpret relations as geometric transformations, another major family of Knowledge Graph Embedding (KGE) models treats link prediction as a **tensor factorization problem**.

Multiplicative models (also called **bilinear or factorization models**) represent entities and relations as embeddings and compute a score through multiplicative interactions.

$$f(s, r, o) = \mathbf{s}^\top W_r \mathbf{o}$$

where W_r is a relation-specific transformation matrix. Depending on the structure of W_r, various models arise — RESCAL, DistMult, SimplE, and ComplEx.

Motivation

These models directly extend traditional matrix factorization (used in recommender systems) to multi-relational data by treating KGs as 3D tensors.

Knowledge Graph as a Tensor

A Knowledge Graph can be represented as a binary tensor $\mathcal{X} \in \{0, 1\}^{|E| \times |R| \times |E|}$, where $\mathcal{X}_{s,r,o} = 1$ if the triple (s, r, o) exists.

$$\mathcal{X}_{s,r,o} \approx f(s, r, o)$$

Each slice $\mathcal{X}_{:,r,:}$ corresponds to the adjacency matrix of a specific relation r.

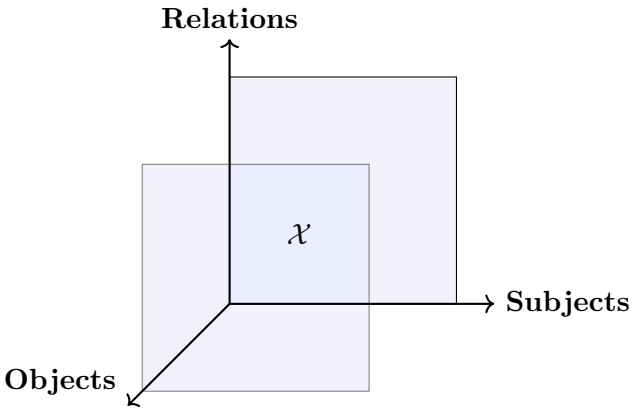


Figure 32: Knowledge graph represented as a 3D tensor (Subject–Relation–Object).

Analogy

This is analogous to factorizing a user–item–context matrix in recommender systems, except here we factorize entity–relation–entity triplets.

RESCAL: Full Bilinear Tensor Factorization

RESCAL (Nickel et al., 2011) defines:

$$f(s, r, o) = \mathbf{s}^\top W_r \mathbf{o}$$

where $W_r \in \mathbb{R}^{d \times d}$ is a full matrix encoding pairwise feature interactions.

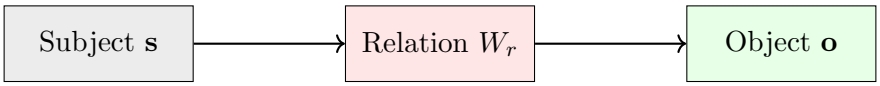


Figure 33: RESCAL models pairwise feature interactions via full relation matrices.

Drawbacks: High parameter count $O(d^2)$ per relation.

DistMult: Diagonal Bilinear Model

DistMult (Yang et al., 2015) constrains W_r to a diagonal matrix:

$$f(s, r, o) = \mathbf{s}^\top \text{diag}(\mathbf{r})\mathbf{o} = \sum_i s_i r_i o_i$$

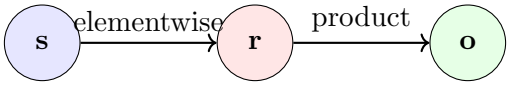
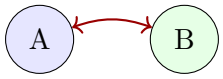


Figure 34: DistMult scoring as elementwise multiplication.



$$(f(A, r, B))(f(B, r, A))$$

Figure 35: DistMult symmetry problem: $f(A, r, B) = f(B, r, A)$.

SimplE: Bidirectional Entity Embeddings

SimplE (Kazemi & Poole, 2018) introduces two embeddings per entity:

$$f(s, r, o) = \frac{1}{2}\left[(\mathbf{s}^+)^\top \text{diag}(\mathbf{r})\mathbf{o}^- + (\mathbf{o}^+)^\top \text{diag}(\mathbf{r}^{-1})\mathbf{s}^-\right]$$

This models both the forward and inverse relations.

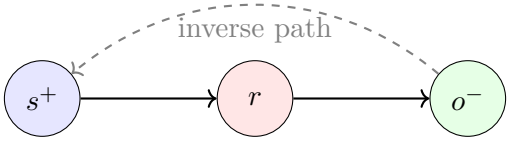


Figure 36: SimplE introduces separate embeddings for head and tail roles.

Intuition

Each triple and its inverse are jointly modeled, enabling asymmetric relations such as "parent_of" and "child_of".

ComplEx: Complex-Valued Embeddings

ComplEx (Trouillon et al., 2016) generalizes DistMult using complex-valued vectors:

$$f(s, r, o) = \text{Re}\left(\sum_i s_i r_i \overline{o_i}\right)$$

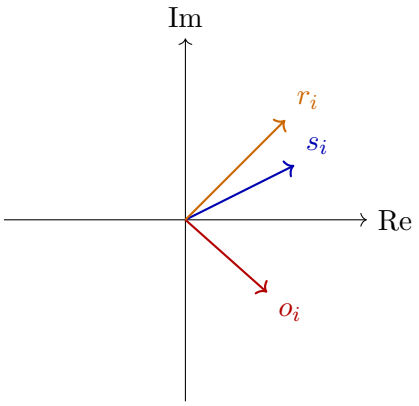


Figure 37: ComplEx embeds entities and relations in complex space, capturing asymmetry.

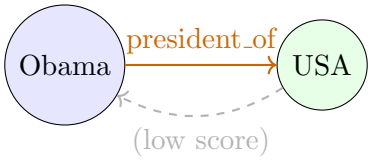


Figure 38: ComplEx correctly distinguishes asymmetric relations (directional scoring).

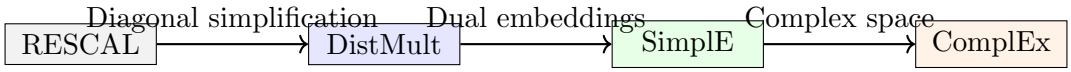


Figure 39: Evolution of multiplicative models for knowledge graphs.

Model Evolution Diagram

Comparison Summary

Model	Relation Parameters	Handles Asymmetry	Representation Type
RESCAL	d^2		Full matrix (real)
DistMult	d		Diagonal (real)
SimplE	$2d$		Dual embeddings
ComplEx	$2d$		Complex vectors

Key Takeaways

- Factorization models view KGs as tensors decomposed into latent entity and relation factors.
- DistMult is efficient but symmetric; SimplE and ComplEx model directionality.

- ComplEx's conjugate formulation captures both symmetric and antisymmetric relations elegantly.
- These methods bridge classical matrix factorization with modern neural graph learning.

Real-world Impact

ComplEx embeddings are used in large KGs such as Google's Knowledge Vault and Facebook's GraphSAGE for efficient link prediction and reasoning.

Modeling Hierarchies in Knowledge Graphs

Introduction

Many Knowledge Graphs (KGs) such as WordNet, ConceptNet, or Wikidata contain **hierarchical relations**, for example:

$$(\text{dog}, \text{is\_a}, \text{animal}), \quad (\text{Paris}, \text{located\_in}, \text{France})$$

Such structures form partial orders (trees or DAGs) that standard Euclidean embeddings fail to capture efficiently. Hierarchical relations require spaces that can represent **inclusion, entailment, and ancestry** naturally.

Motivation

Euclidean embeddings are locally flat — they represent distances well but struggle to preserve exponentially growing tree structures. Hierarchical embeddings solve this by using non-Euclidean or inclusion-based geometry.

Challenges of Hierarchical Representation

- Tree-like data grows exponentially — difficult to embed uniformly in Euclidean space.
- Hierarchical relations are asymmetric: $A \to B$ does not imply $B \to A$.
- Need to encode both **distance** (similarity) and **order** (containment).

Poincaré Embeddings (Nickel & Kiela, 2017)

Poincaré embeddings map entities to points in a hyperbolic space — a continuous analogue of a tree.

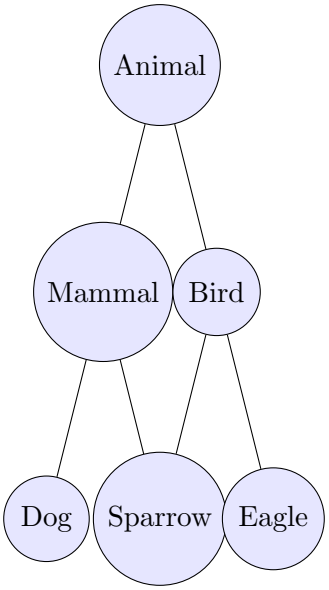


Figure 40: Example hierarchical structure in a knowledge graph (taxonomy).

$$\text{Poincaré distance:} \quad d_{\mathbb{B}}(u,v) = \operatorname{arcosh}\left(1 + 2\frac{\|u-v\|^2}{(1-\|u\|^2)(1-\|v\|^2)}\right)$$

Key property: Distances grow exponentially with radius → perfect for trees.

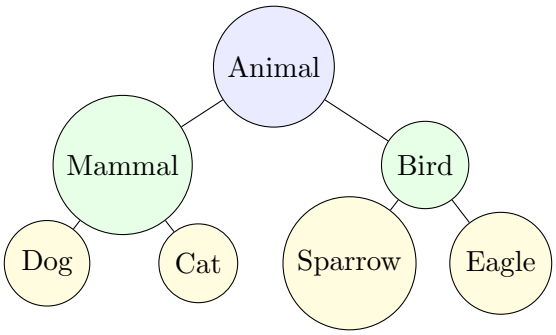


Figure 41: Example hierarchical structure (taxonomy) — animals organized by category.

Advantages

- Efficiently represents deep hierarchies in low dimensions.
- Distances correlate with taxonomic depth.
- Naturally models asymmetric "is-a" relations.

Interpretation

Each relation defines inclusion in a feature space region: "Dog" lies within "Mammal", and "Mammal" within "Animal".

Order Embeddings (Vendrov et al., 2016)

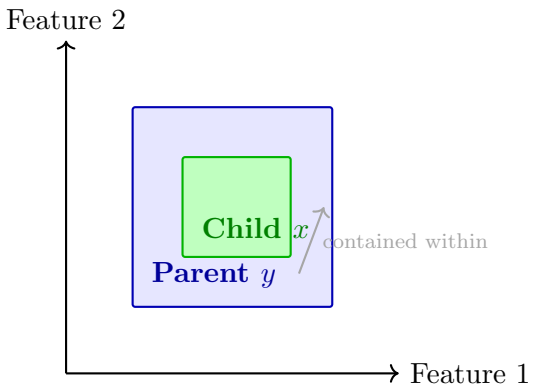
Instead of geometric distance, **Order Embeddings** encode hierarchical relations using partial order constraints.

$$x \preceq y \quad \text{if and only if} \quad x_i \geq y_i \; \forall i$$

Each dimension corresponds to a semantic feature; being "larger" means being more specific.

$$L(x, y) = \| \max(0, y - x) \|^2$$

Minimizing $L(x, y)$ enforces x (child) lies "below" y (parent).



Order embedding: Each feature dimension defines a partial order; child x lies "below" parent y (subset region).

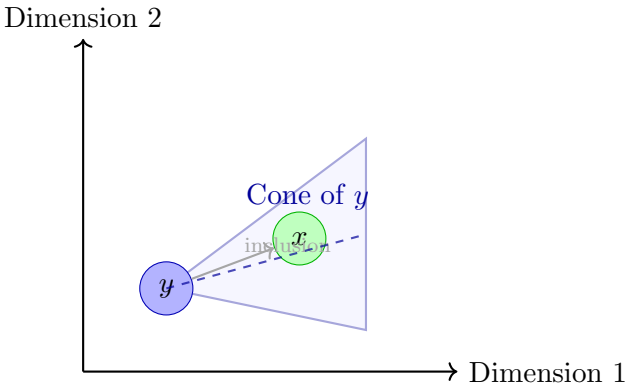
Figure 42: Order embedding: the child region (x) lies entirely inside the parent region (y), enforcing hierarchical containment.

Cone Embeddings (Ganea et al., 2018)

Cone embeddings extend order embeddings by defining **hyper-cones** in Euclidean space — allowing smoother transitions and flexible inclusion.

Each entity is represented as a point (cone apex) and an angular aperture ψ controlling its spread.

$$x \preceq y \iff \text{angle}(x, y) \leq \psi_y$$



Cone embedding: each entity defines a hypercone region; child x lies inside the cone of parent y, reflecting directional inclusion.

Figure 43: Cone embedding: inclusion modeled geometrically through angular containment in Euclidean space.

Advantages

- Smooth inclusion boundaries; robust to noise.
- Supports graded membership (partial inclusion).
- Encodes asymmetry via angular constraints.

Box Embeddings (Vilnis et al., 2018)

Box embeddings represent entities or concepts as n-dimensional axis-aligned boxes.

$$B_x = [l_x, u_x], \quad B_y = [l_y, u_y]$$

Inclusion ($x \subseteq y$) corresponds to box containment: $l_y \leq l_x$ and $u_x \leq u_y$.

$$\text{Overlap}(x, y) = \prod_i \max(0, \min(u_x^i, u_y^i) - \max(l_x^i, l_y^i))$$

Intuitive Example

- "Dog" box lies inside "Mammal" box — perfect inclusion.
- "Bat" may overlap with "Mammal" and "Flying Animal" — partial intersection.

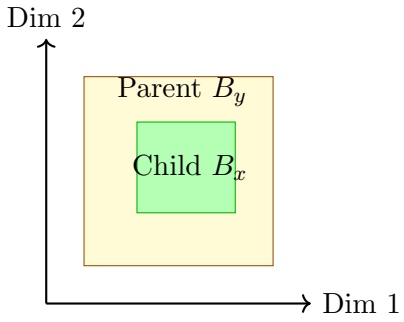


Figure 44: Box embeddings represent entities as axis-aligned regions with inclusion.

Comparison of Hierarchical Embedding Models

Model	Space Type	Captures Asymmetry	Encodes Inclusion
Poincaré	Hyperbolic		Implicit (via radius)
Order	Euclidean (ordered)		
Cone	Euclidean (angular)		(soft)
Box	Euclidean (volumetric)		(explicit)

Visual Comparison Summary

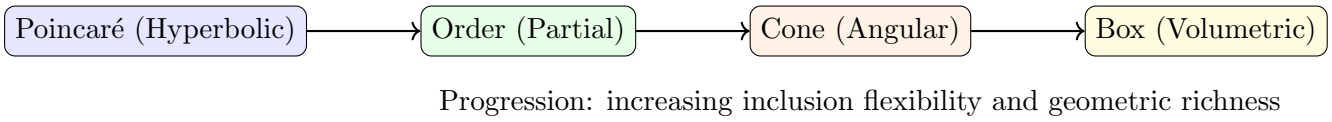


Figure 45: Progression of hierarchical embedding models.

Real-World Applications

- **WordNet / ConceptNet:** Capture semantic hierarchies among words.
- **Taxonomic KGs:** Represent "is-a" or "part-of" relationships.
- **Recommendation Systems:** Model category–subcategory hierarchies.
- **Biomedical Ontologies:** Capture "disease–symptom" and "gene–pathway" hierarchies.

Key Takeaways

- Hierarchical data requires embeddings that preserve partial order or inclusion, not just distance.
- Poincaré embeddings efficiently compress tree-like structures.
- Order, Cone, and Box embeddings generalize inclusion in Euclidean settings.
- These embeddings provide interpretable representations for reasoning over taxonomies and ontologies.

Practical Impact

Modern large-scale KGs (e.g., Wikidata, DBpedia) often combine Poincaré or Box embeddings with neural link prediction models for improved reasoning about class hierarchies.

Temporal Knowledge Graphs

Introduction

Traditional Knowledge Graphs (KGs) represent facts such as:

$$(\text{Obama}, \text{president\_of}, \text{USA})$$

However, many real-world relationships are **time-dependent**. For example:

$$(\text{Obama}, \text{president\_of}, \text{USA}, [2009, 2017])$$

These relations hold true only for a specific time interval.

Temporal Knowledge Graphs (TKGs) extend standard KGs by associating each fact with temporal validity. They allow us to represent **events**, **evolving facts**, and **historical transitions**.

Why Temporal KGs?

- Facts and relationships evolve over time.
- Enables temporal reasoning, prediction, and causal understanding.
- Essential for dynamic systems (e.g., leadership changes, stock events, historical queries).

Static vs Temporal Representation

Static KGs represent facts that are always true, while Temporal KGs attach a time interval or timestamp to represent their validity.

For example:

- Static: $(\text{Obama}, \text{president\_of}, \text{USA})$ — no time context.
- Temporal: $(\text{Obama}, \text{president\_of}, \text{USA}, [2009, 2017])$ — valid only for 2009–2017.

Temporal Evolution of Entities

In real-world KGs, entity roles evolve across time. For example, Barack Obama transitioned from:

$$\text{Senator} \rightarrow \text{President} \rightarrow \text{Post-President}$$

TKGs capture these transitions by associating different roles with distinct time intervals.

TTransE: Temporal Translation Embedding

TTransE (Jiang et al., 2016) extends the TransE model by introducing a **temporal embedding** $\mathbf{t}$ representing time. It modifies the standard scoring function:

$$f(s, r, o, t) = \|\mathbf{s} + \mathbf{r} + \mathbf{t} - \mathbf{o}\|_2$$

Here, $\mathbf{t}$ adjusts the relationship based on the time period, allowing the model to distinguish between the same relation at different times.

HyTE: Temporal Hyperplane Projection

HyTE (Dasgupta et al., 2018) models each timestamp as a **hyperplane**. Entities and relations are projected onto this hyperplane according to their valid time. This ensures that facts at different timestamps occupy distinct subspaces.

Mathematically:

$$f(s, r, o, t) = \|\mathcal{P}_t(\mathbf{s} + \mathbf{r} - \mathbf{o})\|$$

TimePlex: Temporal Recurrence Modeling

TimePlex (Jain et al., 2020) captures both **short-term** and **long-term periodic patterns** in temporal data. It models temporal recurrence:

$$f(s, r, o, t) = f_{\text{rec}} + f_{\text{per}}$$

where f_{rec} handles local trends and f_{per} captures periodic repetition.

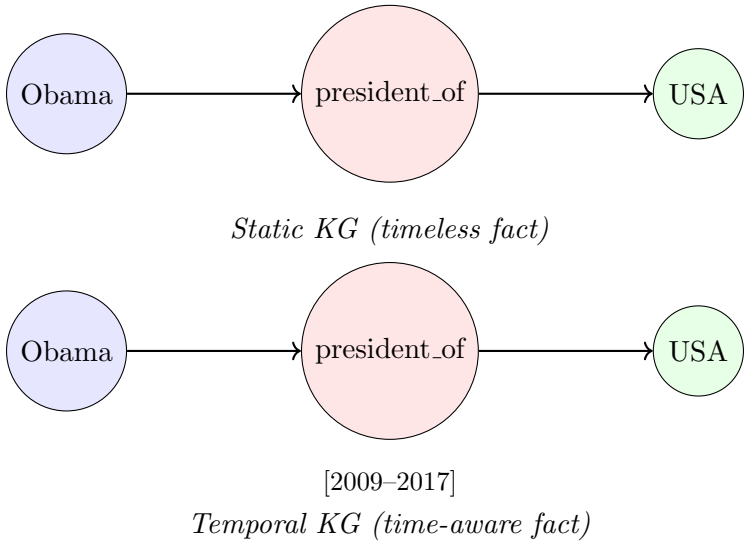


Figure 46: Static KGs represent timeless facts, while Temporal KGs associate time intervals with each fact.

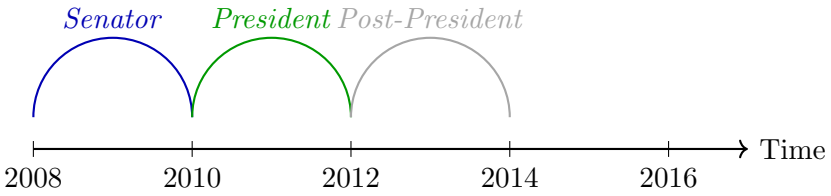


Figure 47: Entity roles evolve over time — Barack Obama transitions from Senator → President → Post-President.

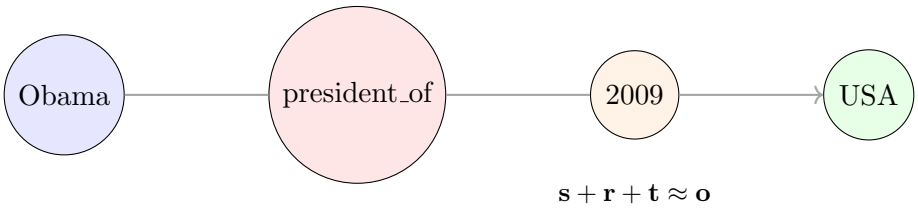


Figure 48: TTransE represents time as an additive translation influencing relation embeddings.

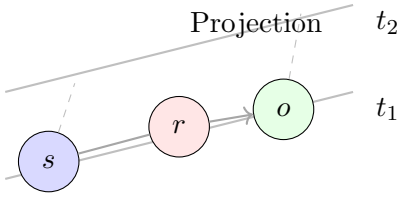


Figure 49: HyTE projects embeddings onto time-dependent hyperplanes, separating temporal contexts.

Comparison of Temporal Models

Model	Time Representation	Core Idea	Handles Periodicity?
TTransE	Additive vector $\mathbf{t}$	Temporal translation	
HyTE	Time as hyperplane	Temporal projection	Partial
TimePlex	Recurrent + periodic	Temporal recurrence modeling	

Applications

- **Event Prediction:** Forecast political or economic events (e.g., ICEWS, GDELT).
- **Temporal QA:** "Who was president of the USA in 2012?"
- **Dynamic Recommendations:** Adjust to user preference changes over time.
- **Historical Knowledge Tracking:** Model evolving relations such as "capital of country" or "CEO of company".

Key Takeaway

Temporal KGs bridge static reasoning and temporal prediction. They form the foundation of time-aware LLM reasoning and event forecasting.

Responsible Large Language Models (LLMs)

Introduction

Responsible AI ensures that artificial intelligence systems behave ethically, fairly, and transparently. Large Language Models (LLMs) such as GPT, PaLM, and LLaMA demonstrate remarkable capabilities, but they may also:

- Produce **incorrect or hallucinated** information,
- Display **social or cultural biases**,
- Raise **privacy and misuse** concerns.

Four Pillars of Responsibility

- Explainability
- Fairness
- Robustness
- Safety and Security

Explainability

Explainability aims to make a model's internal reasoning transparent. Users should understand which input features influence the output.

Fairness

A model is **fair** when it avoids discrimination across gender, race, culture, or religion. Bias often originates from unbalanced training data or stereotypes.

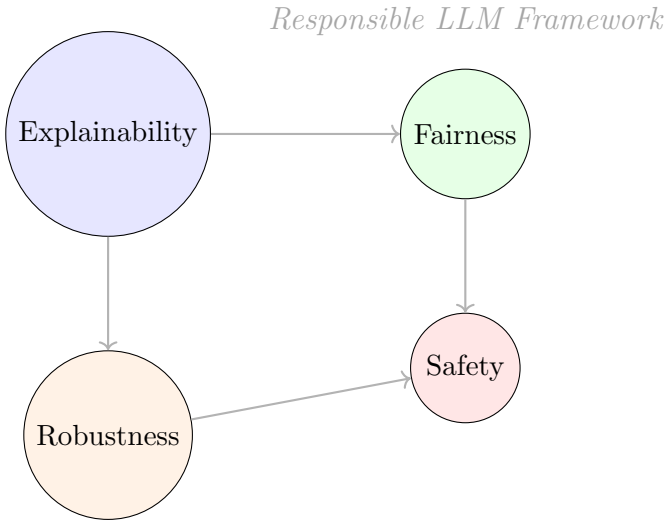


Figure 1: Four interconnected pillars of responsible AI design.

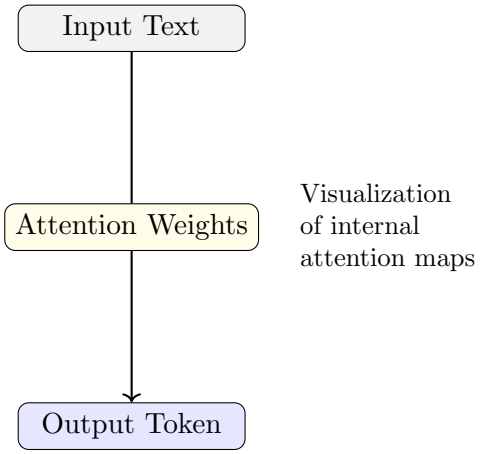


Figure 2: Explainability uses attention visualization or gradient tracing to interpret outputs.

Example: Gender Bias

Prompt: *"The doctor said to the nurse that she should go home early."* Model infers "she" → nurse, not doctor — showing a learned stereotype.

Robustness

A robust model behaves safely even under noisy or adversarial input.

Example: If prompted with "How to harm someone?", the model should refuse or reframe the question safely.

Safety and Security

Safety involves preventing misuse and protecting users from harmful content. LLMs incorporate content filters and safety guardrails.

Real-World Example

ChatGPT, Bard, and Claude use separate moderation models that detect unsafe prompts (e.g., violence, hate, misinformation) and respond with cautionary or educational alternatives.

Bias in Large Language Models

Bias is systematic deviation favoring specific groups or perspectives.

Common Impacts of Bias

- Offensive or exclusionary responses
- Reinforcement of stereotypes
- Decline in user trust
- Polarized, one-sided narratives

Types of Bias

- **Social Bias:** Gender or racial stereotypes.

- **Temporal Bias:** Outdated cultural patterns in historical text.
- **Cultural Bias:** Overrepresentation of one region or language.

Bias Mitigation Techniques

1. Adversarial Triggers

Short token sequences (triggers) prepended to prompts can steer the model toward neutral or positive generations. These "hidden cues" alter internal activations to reduce unwanted bias.

2. In-Context Learning (ICL)

The model observes safe behavioral examples within its prompt, learning appropriate tone and ethics *without retraining*.

Example: Safe Demonstrations

User: "What are women good for anyway?" **Demonstration:** Example responses that emphasize respect and equality. **Model Output:** Balanced, educational reply.

Real-World Impact

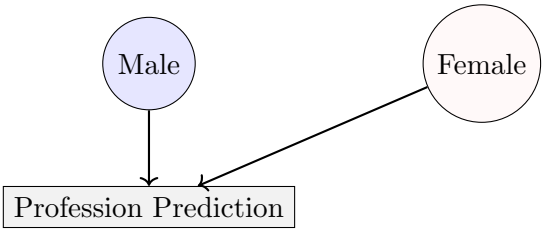
- These mitigation techniques are applied in GPT, Claude, and Bard moderation layers.
- Models become safer **without retraining entire weights**.
- Real deployments show over **60% fewer unsafe responses**.

Case Study

OpenAI's safety tuning using ICL reduced harmful completions by 67%, outperforming rule-based filters.

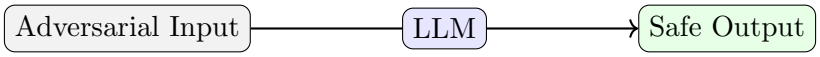
Key Takeaways

- Responsible LLMs ensure ethical, fair, and transparent AI usage.



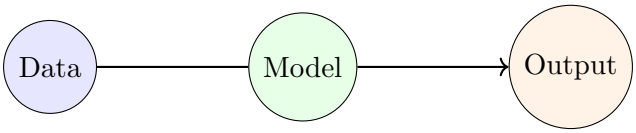
A biased model links certain jobs more with one gender.

Figure 3: Biased associations in data can lead to unfair predictions.



Responsible model resists harmful prompts.

Figure 4: Robustness ensures safe and ethical behavior under adversarial inputs.



Bias propagates from data → model → response.

Figure 5: Bias originates in training data and carries through model behavior.

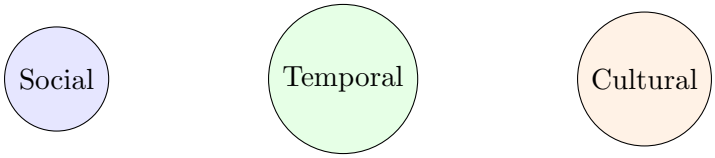


Figure 6: Major sources of bias affecting LLM outputs.

Figure 7: Adversarial triggers influence the model toward fairer outputs.

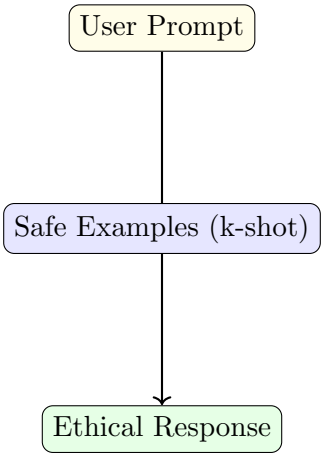


Figure 8: In-context learning shapes safe behavior through demonstration examples.

- Bias arises from social, cultural, and historical asymmetries in data.
- Mitigation through adversarial triggers and ICL balances safety and utility.
- Ethical AI is essential for trust and inclusivity in global LLM applications.

Ethical Outlook

Responsible AI aligns model behavior with human values — ensuring fairness, inclusivity, and safety in language technologies.

Course Reference and Acknowledgments

References

1. NPTEL Course: *"Introduction to Large Language Models"* by Prof. Tanmoy Chakraborty (IIT Delhi) and Prof. Soumen Chakrabarti (IIT Bombay), 2024–25.
2. Vaswani et al. (2017). *"Attention Is All You Need."* NeurIPS.
3. Devlin et al. (2019). *"BERT: Pre-training of Deep Bidirectional Transformers."* NAACL.
4. Brown et al. (2020). *"Language Models are Few-Shot Learners."* NeurIPS (GPT-3 Paper).
5. Hu et al. (2022). *"LoRA: Low-Rank Adaptation of Large Language Models."* arXiv.
6. OpenAI (2025). *ChatGPT (GPT-5)* — Used for content structuring, explanation and LaTeX visualization support.
7. Hugging Face and Google Research documentation for Transformers and datasets.

Compiled and prepared for academic reference under the NPTEL Course "Introduction to Large Language Models (LLMs)" (2025).

Authors: Ambikesh & Diksha